



**UNIVERSITÉ
DE GENÈVE**

FACULTÉ DES SCIENCES

Département d'informatique

Saisie d'objets en environnement pour un bras robotique assisté par vision

Travail de Fin d'Études — Bachelor en Sciences Informatiques

Étudiant Maxence Chanier

Encadrant Dr. G. Bologna

Année académique 2025-2026

02 Juillet 2026

Résumé

Ce travail porte sur la saisie autonome d’objets par un bras robotique à bas coût guidé par la vision. Sur un SO-101 équipé de trois caméras — une paire stéréo *eye-to-hand* et une caméra *eye-in-hand* montée au poignet —, nous construisons une *pipeline* complète et modulaire allant du pixel à la prise : détection de l’objet, reconstruction de sa position tridimensionnelle, planification de la saisie, puis contrôle du bras. L’ensemble est *open-source* et reproductible, et chacune de ses étapes est conçue et expliquée séparément plutôt que traitée comme une boîte noire.

La planification de saisie règle l’ouverture et l’orientation des mâchoires sur la géométrie mesurée de l’objet et choisit un angle d’attaque par balayage. Pour situer cette approche explicite, nous implémentons sur le *même* bras et la *même* tâche une seconde méthode — une politique apprise de bout en bout par imitation (*Action Chunking Transformer*) —, puis nous comparons les deux paradigmes sur un banc commun.

Deux campagnes de mesures — 78 essais pour le pipeline, 30 pour la politique apprise, assorties d’intervalles de confiance de Wilson — établissent des taux de réussite de 67 à 77 % pour le pipeline sur le cube et de 67 à 78 % sur le cylindre (selon le détecteur employé), et de 70 % pour la politique apprise sur le cube. À égalité sur l’objet entraîné, le pipeline est plus général et plus transparent, la politique apprise plus rapide à mettre en œuvre. Nous énonçons enfin les limites du travail : un bras à cinq degrés de liberté (soit un de moins que les six d’une pose quelconque dans l’espace), l’absence délibérée d’évitement d’obstacles, et un périmètre d’objets et de conditions explicitement déclaré.

Remerciements

Ce travail est né d’une initiative personnelle. Au printemps 2025, avant même qu’un sujet ne soit défini, j’ai découvert le SO-101 et entrepris d’en imprimer et d’en assembler le bras de mes propres mains, par curiosité et par goût du défi. Le projet de recherche s’est ensuite construit autour de cette plateforme.

Je remercie chaleureusement mon encadrant, le Dr. Guido Bologna, d’avoir accueilli cette démarche, de m’avoir accordé sa confiance et de m’avoir guidé tout au long de ce travail. Nos échanges ont permis de transformer une intuition de départ en un travail de recherche structuré.

Ma reconnaissance va également aux communautés *open-source* sans lesquelles ce projet n’aurait pas vu le jour — l’écosystème LeRobot de Hugging Face, ainsi que les contributeurs des projets SO-101 et XLeRobot —, dont le travail met une robotique de qualité à la portée d’un étudiant.

Je remercie enfin mes proches, ma famille et mes amis, pour leur soutien et leur patience durant ces mois de travail.

Table des matières

1	Introduction	5
1.1	Contexte	5
1.2	Problématique	6
1.3	Objectifs	6
1.4	Matériel	7
1.5	Contributions et plan du mémoire	8
2	État de l’art	10
2.1	Calibration caméra–robot (<i>hand-eye</i>)	10
2.2	Géométrie projective et triangulation stéréo	11
2.3	Détection d’objets : du seuillage couleur aux modèles <i>open-vocabulary</i>	11
2.4	Planification de saisie (<i>grasp planning</i>)	11
2.5	Planification de trajectoires	12
2.6	Apprentissage par imitation (ACT)	12
2.7	Positionnement de ce travail	13
3	Architecture	14
3.1	Vue d’ensemble	14
3.2	Interfaces et patrons de conception	15
3.3	Repères et conventions	16
3.4	Lien matériel–logiciel	18
3.5	Modes d’exécution : <i>live</i> , <i>dry-run</i> et <i>replay</i>	19
4	Perception	21
4.1	Pipeline de perception	21
4.2	Calibration et modèles caméra	22
4.3	Détection 2D — version V1 (HSV)	28
4.4	Détection 2D — version V2 (<i>open-vocabulary</i>)	30
4.5	Reconstruction 3D	31
4.6	Synchronisation multi-caméras	33
4.7	Validation expérimentale	33
5	Planification de saisie	36
5.1	Définition du problème et sous-actionnement	36

5.2	Stratégie : saisie géométrique par balayage de l'angle d'attaque	37
5.3	Construction géométrique de la prise	38
5.4	Périmètre : balayage sagittal et limite de la prise latérale	39
5.5	Interface avec le contrôle et validation	40
6	Contrôle	41
6.1	Cinématique directe	41
6.2	Cinématique inverse	43
6.3	Génération de trajectoire	46
6.4	Contrôle moteur	48
6.5	Pipeline intégré et boucle fermée	49
7	Évaluation expérimentale	52
7.1	Protocole expérimental	52
7.2	Résultats : pipeline modulaire	54
7.3	Résultats : apprentissage par imitation (ACT)	55
7.4	Comparaison des deux paradigmes	56
8	Discussion	59
8.1	Synthèse des limites identifiées	59
8.2	Choix discutables et sous-actionnement	60
8.3	Limites assumées : évitement d'obstacles et occlusion	62
8.4	Pistes d'amélioration	63
9	Conclusion	65
9.1	Bilan au regard des objectifs	65
9.2	Contributions	66
9.3	Perspectives	67
9.4	Note sur l'utilisation d'outils d'intelligence artificielle	67
	Bibliographie	68
A	Détails techniques et reproductibilité	71
A.1	Structure du code	71
A.2	Repère de base : mesure expérimentale	73
A.3	Procédure de calibration	74
A.4	Campagne de mesure de la structure d'environnement	76
A.5	Capture des données et paramètres	78

Chapitre 1

Introduction

1.1 Contexte

Les bras robotiques à bas coût sont devenus largement accessibles. Des plateformes open-source comme le SO-101 [1], soutenues par des communautés telles que Hugging Face et son écosystème LeRobot [2], mettent aujourd’hui un bras manipulateur fonctionnel à la portée d’un étudiant, à un coût bien inférieur à celui des bras robotiques professionnels. Disposer d’un bras que l’on peut commander ne suffit pourtant pas à lui faire *saisir* un objet de façon autonome : il faut enchaîner plusieurs étapes non triviales, à savoir percevoir la scène, localiser la cible en trois dimensions, décider comment l’attraper, puis exécuter le mouvement.

Deux approches permettent de construire cet enchaînement. La première, *modulaire et explicite*, décompose le problème en étapes que l’on conçoit et comprend séparément : calibration des caméras, détection de l’objet, reconstruction 3D, planification de la prise, puis contrôle du bras. La seconde, *de bout en bout*, confie l’ensemble à une politique apprise qui imite directement des démonstrations humaines, sans décomposition explicite.

La motivation de ce travail est de **construire entièrement un pipeline du premier type** (complet, transparent, open-source et reproductible) sur du matériel à bas coût, en maîtrisant et en expliquant chacune de ses étapes plutôt qu’en traitant la saisie comme une boîte noire.

Le SO-101 illustre d’ailleurs ce contraste : il est d’abord conçu pour l’apprentissage par imitation (deux bras identiques, un « meneur » manipulé à la main et un « suiveur » qui le reproduit, servent à enregistrer des démonstrations). Sur cette plateforme, l’imitation est l’approche la plus directe et la plus rapide. Construire à la place un pipeline géométrique explicite, l’approche la moins immédiate pour ce matériel, est précisément le choix retenu et la principale difficulté de ce travail. Pour mettre ce choix en perspective, nous implémentons aussi l’approche de bout en bout (une politique *Action Chunking Transformer*, ACT) sur le *même* bras et la *même* tâche ; l’imitation y sert alors de point de comparaison plutôt que de finalité, et cette comparaison revient tout au long du mémoire.

Ce travail s’inscrit dans un but plus large, fixé par le cahier des charges : la saisie en *environnement encombré*, où les occlusions, les points de vue partiels et les risques de collision interdisent de se contenter d’une trajectoire figée ou d’une stratégie de prise unique. Ce but structure les objectifs (section 1.3). Le présent travail en traite la partie principale — l’architecture perception–planification–contrôle multi-caméras et la saisie adaptative à la géométrie de l’objet

— et discute, comme limites et perspectives, les aspects non couverts tels que l'évitement actif d'obstacles.

1.2 Problématique

Prise isolément, chacune de ces étapes dispose de solutions connues : on sait calibrer une caméra, détecter un objet coloré ou décrire une cible par du texte, ou encore résoudre une cinématique inverse. La difficulté de la saisie autonome ne réside donc pas dans une brique isolée, mais dans leur *orchestration* : faire coopérer de façon fiable perception, reconstruction 3D, planification de la prise et contrôle, sur un matériel à bas coût dont les imprécisions (jeu mécanique, bruit de détection, incertitude de calibration) se cumulent le long de la chaîne.

Cette difficulté s'accroît dès que l'objet cible n'est plus seul. En *environnement encombré*, la perception devient sujette aux occlusions et aux points de vue partiels (une caméra fixe peut ne plus voir la cible) et la géométrie de l'objet n'est connue qu'imparfaitement. De plus, le chemin le plus direct vers l'objet n'est pas toujours praticable, en raison des risques de collision. Une trajectoire figée et une stratégie de prise unique ne suffisent alors plus.

La question centrale de ce travail est donc la suivante : *comment décider, à partir de plusieurs sources visuelles, où regarder et comment saisir un objet, tout en exécutant le mouvement de façon robuste ?* Nous l'aborderons dans une logique *perception-action*, où le résultat dépend non seulement de la qualité de chaque brique, mais aussi de la décision et de la coordination de l'ensemble. Les objectifs qui en découlent, et le périmètre effectivement traité, sont précisés à la section 1.3.

1.3 Objectifs

La finalité de ce travail, telle que fixée par le cahier des charges, est de *concevoir, implémenter et évaluer une architecture permettant à un bras robotique équipé de caméras eye-to-hand et eye-in-hand d'interagir de manière autonome avec un environnement encombré*. Cette finalité se décline en six objectifs, que nous reprenons ici en indiquant honnêtement leur degré de réalisation dans le présent travail.

Objectif (cahier des charges)	Statut	Ch.
1. Architecture perception-planification-contrôle multi-caméras	Atteint	3, 4
2. Sélection de points de vue contre les occlusions	Partiel	4, 8
3. Trajectoires évitant les obstacles	Hors périmètre	8
4. Replanification en boucle fermée par la perception	Partiel	4, 6
5. Stratégies de saisie adaptées à la géométrie et à l'accessibilité	Atteint	5
6. Évaluation expérimentale par métriques	Partiel	7

TABLE 1.1 – Les six objectifs du cahier des charges et leur degré de réalisation dans ce travail.

Deux objectifs constituent la partie principale réalisée du travail : l'architecture multi-caméras (objectif 1) et la saisie adaptative à la géométrie de l'objet (objectif 5). Deux autres sont traités partiellement : la perception bénéficie d'un raffinement par la caméra montée au poignet, qui

atténue certaines occlusions et corrige la position en boucle fermée (objectifs 2 et 4), sans toutefois constituer une sélection active de points de vue ni un asservissement continu. L'évitement d'obstacles (objectif 3) n'est pas abordé : c'est un choix délibéré, dont le chapitre 8 détaille le motif. Enfin, l'évaluation (objectif 6) repose sur deux campagnes de mesures ; la métrique des « collisions évitées » présuppose ce module d'évitement non développé et reste donc sans objet.

Le cahier des charges précise par ailleurs que, « dans un premier temps, la prise de décision reposera sur des règles heuristiques et des critères géométriques », les approches par apprentissage étant « envisagées comme extensions ». C'est exactement la trajectoire suivie : le pipeline repose sur des heuristiques géométriques, et l'apprentissage par imitation est réalisé comme seconde méthode, à titre de comparaison.

1.4 Matériel

La plateforme matérielle réunit un bras manipulateur, trois caméras, une pince adaptée et une scène de saisie imprimée en 3D, le tout piloté depuis un ordinateur portable (figure 1.1).

Bras robotique. Le robot est un **SO-101**, un bras open-source à bas coût de l'écosystème LeRobot. Chaque bras comporte **six servomoteurs Feetech STS3215** : cinq commandent les articulations rotatives (rotation de base, épaule, coude, flexion et rotation du poignet) et le sixième actionne la pince. La plateforme se compose de deux bras identiques (un « meneur » et un « suiveur ») ; le pipeline modulaire n'utilise que le suiveur, tandis que l'apprentissage par imitation exploite la paire, le meneur servant à enregistrer les démonstrations. Ne disposer que de *cinq* articulations pour positionner *et* orienter la pince — soit une de moins que les six degrés de liberté d'une pose quelconque dans l'espace — est une contrainte centrale, analysée au chapitre 5.

Caméras. Trois caméras USB de résolution 1920×1080 équipent le système. Deux d'entre elles, **cam_0** et **cam_1**, sont fixes et placées face à la scène : montées *eye-to-hand* (déportées du bras), elles forment une paire stéréo qui reconstruit la position tridimensionnelle de l'objet. La troisième, **cam_2**, est montée *eye-in-hand*, au poignet : embarquée, elle suit la pince et sert au raffinement de la prise en boucle fermée.

Pince. La pince d'origine a été remplacée par un modèle imprimé en **TPU** (filament souple) issu des extensions communautaires du projet *XLeRobot* : le *Compliant gripper (new)*, à nombreuses lamelles internes. Une version antérieure, le *Compliant gripper*, plus souple, avait également été essayée, de même que différentes caméras de poignet, avant de retenir la configuration actuelle (figure 1.2). Deux adaptations améliorent la tenue de l'objet : un **grip de tennis** collé sur les mâchoires augmente l'adhérence et réduit le glissement, et la souplesse du TPU permet aux mâchoires de se **déformer pour épouser la forme** de l'objet saisi. Cette compliance facilite la prise, mais introduit une incertitude sur le point de contact, discutée au chapitre 5.

Scène de saisie. L'environnement de saisie est une structure conçue et imprimée en 3D. Ses dimensions résultent d'une campagne de conception (caractéristiques visées puis cotes retenues), détaillée à l'annexe A. Elle se compose de quatre pièces distinctes, dont deux imprimées en double.

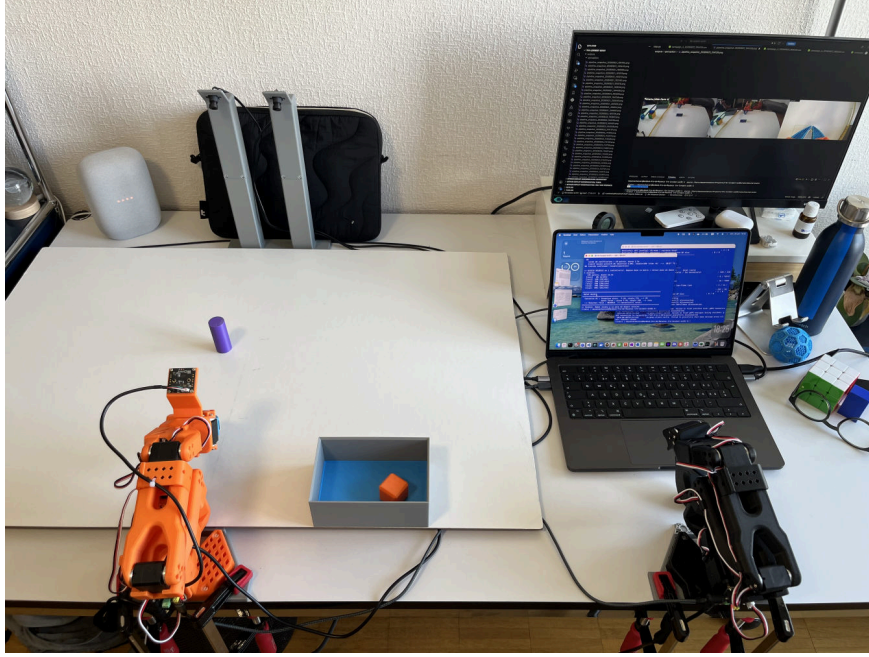


FIGURE 1.1 – Vue d’ensemble de la plateforme.

Les fichiers de modélisation sont publiés sur le dépôt du projet plutôt que reproduits dans ce mémoire.

Ordinateur et logiciels. L’ensemble est piloté depuis un MacBook M4 (24 Go). Le code est écrit en Python et s’appuie sur l’écosystème open-source Hugging Face / LeRobot, sur OpenCV pour la vision et NumPy pour le calcul ; le modèle cinématique du bras provient du fichier URDF `so101_new_calib.urdf`. Une première politique d’imitation a été entraînée localement, sur le processeur graphique intégré du Mac (MPS). Faute de puissance et de temps, la version finale a toutefois été entraînée sur Google Colab Pro (GPU A100), via la plateforme Hugging Face. Les détails du protocole d’entraînement sont donnés au chapitre 7.

1.5 Contributions et plan du mémoire

Les contributions de ce travail sont les suivantes. (i) Un pipeline complet et modulaire de perception, planification et contrôle, entièrement open-source et reproductible, mise en œuvre sur le SO-101 (un bras à bas coût). (ii) Une stratégie de saisie *adaptive à la géométrie* de l’objet, fondée sur un balayage de l’angle d’attaque, qui en constitue l’apport central. (iii) La mise en perspective de cette approche explicite avec une politique d’imitation de bout en bout (ACT) entraînée sur le même bras et la même tâche, et leur comparaison expérimentale. (iv) Une évaluation conduite par deux campagnes de mesures, dont le périmètre et les limites sont déclarés explicitement.

Le mémoire est organisé comme suit. Le chapitre 2 situe le travail par rapport à l’état de l’art. Le chapitre 3 présente l’architecture logicielle et matérielle. Le chapitre 4 décrit la perception — détection puis reconstruction 3D —, le chapitre 5 la planification de saisie, et le chapitre 6 le contrôle du bras. Le chapitre 7 rapporte l’évaluation expérimentale des deux méthodes ; le



FIGURE 1.2 – Pince imprimée en TPU, équipée d'un grip de tennis.

chapitre 8 en discute les limites et les choix, et le chapitre 9 conclut. Enfin, l'annexe A rassemble les éléments de reproductibilité.

Chapitre 2

État de l’art

2.1 Calibration caméra–robot (*hand-eye*)

Pour exploiter une caméra dans une tâche robotique, il faut connaître la transformation rigide entre le repère de la caméra et celui du robot. Déterminer cette transformation est le problème de la *calibration œil-main* (*hand-eye*) [3]. Deux configurations sont possibles : la caméra peut être montée sur l’organe terminal du bras (*eye-in-hand*), ou rester fixe et observer l’espace de travail (*eye-to-hand*) [4]. La première fournit une vue locale et précise qui suit la pince, la seconde une vue globale mais moins précise ; les deux sont complémentaires [4].

La formulation classique ramène la calibration à l’équation matricielle homogène $AX = XB$, où X est la transformation recherchée [3]. Tsai et Lenz en proposent une résolution efficace, qui découple l’estimation de la rotation puis de la translation [5]. Horaud et Dornaika montrent qu’une résolution *simultanée* de la rotation et de la translation, par optimisation non linéaire, est plus robuste au bruit de mesure [3].

Cette calibration *extrinsèque* suppose une caméra dont les paramètres *intrinsèques* (distance focale, point principal, distorsion) sont déjà connus. Leur estimation repose le plus souvent sur la méthode de Zhang, qui exploite plusieurs vues d’une mire planaire (un damier) [6] ; cette calibration intrinsèque est une étape distincte de la calibration œil-main, qu’elle précède. La qualité d’une calibration s’évalue usuellement par l’*erreur de reprojection* [7].

En pratique, la calibration œil-main s’appuie sur un objet de référence observé dans plusieurs poses du bras : le plus souvent un *damier* (comme pour la calibration intrinsèque) ou des *marqueurs fiduciaux* de type ArUco [8]. Des travaux récents cherchent à s’affranchir de ces mires, par exemple en relevant la base du robot sous forme de *nuage de points* que l’on recalcule sur son modèle 3D connu [9].

Dans ce travail, nous employons conjointement les deux configurations : une paire stéréo *eye-to-hand* et une caméra *eye-in-hand*. La calibration œil-main repose sur la méthode de Horaud (via la fonction `calibrateHandEye` d’OpenCV) ; la procédure et ses résultats chiffrés sont détaillés au chapitre 4.

2.2 Géométrie projective et triangulation stéréo

Une caméra projette le monde tridimensionnel sur une image bidimensionnelle ; cette projection, décrite par le modèle *sténopé* (*pinhole*), perd l'information de profondeur [10, 11]. Retrouver la position 3D d'un point à partir des images est donc un problème inverse, qui ne peut être résolu à partir d'une seule vue.

La solution classique est la *triangulation* : en observant le même point depuis au moins deux caméras de positions connues, on intersecte les deux rayons optiques pour retrouver le point 3D [10]. C'est le principe de la vision *stéréo*, qui exploite une paire de caméras calibrées. En pratique, à cause du bruit et des imprécisions de calibration, les deux rayons ne se croisent jamais exactement : on cherche alors le point qui minimise l'écart aux deux rayons, et la méthode algébrique de référence est la *transformation linéaire directe* (DLT) [10].

Dans ce travail, nous obtenons la position 3D de l'objet cible par triangulation à partir de la paire stéréo *eye-to-hand* (`cam_0` et `cam_1`). La dérivation détaillée de la triangulation DLT, ainsi que l'analyse de la sensibilité de la profondeur au bruit de détection, sont présentées au chapitre 4.

2.3 Détection d'objets : du seuillage couleur aux modèles *open-vocabulary*

L'approche classique pour localiser un objet dans une image est le *seuillage couleur* : on isole les pixels dont la couleur correspond à celle de l'objet. La conversion dans l'espace *HSV* (teinte, saturation, valeur) facilite ce seuillage en séparant la teinte de la luminosité [11]. Simple, rapide et sans apprentissage, cette méthode reste sensible à l'éclairage et inopérante sur les objets peu colorés, multicolores ou transparents.

Les détecteurs modernes sont *appris*. La famille la plus répandue est à *vocabulaire fermé* (*closed-vocabulary*) : un réseau comme YOLO [12] détecte un ensemble *fixe* de classes décidé à l'entraînement ; reconnaître un nouvel objet impose de le *ré-entraîner* sur des images annotées. Rapides et précis sur les classes connues, ces détecteurs restent donc rigides.

Une famille plus récente lève cette contrainte : les détecteurs à *vocabulaire ouvert* (*open-vocabulary*) localisent un objet à partir d'une *description textuelle quelconque*, fournie à l'usage, sans ré-entraînement. OWL-ViT [13] et sa version passée à l'échelle OWL-ViTv2 [14] en sont représentatifs, et permettent de cibler un objet par le langage (par exemple « le cylindre violet »).

Dans ce travail, nous employons deux détecteurs : le seuillage couleur HSV (V1) pour les primitives colorées, et OWL-ViTv2 (V2) pour le ciblage par le langage. La comparaison détaillée des espaces colorimétriques (HSV, RGB, HSL) et l'évaluation des deux détecteurs sont présentées au chapitre 4.

2.4 Planification de saisie (*grasp planning*)

Planifier une saisie, c'est déterminer la configuration de la pince (position, orientation, ouverture, direction d'approche) qui permet de tenir l'objet de façon stable. On distingue

classiquement deux familles d’approches [15] : les méthodes *analytiques*, qui raisonnent sur la géométrie et la mécanique du contact (par exemple la fermeture de force), et les méthodes *pilotées par les données* (*data-driven*), qui apprennent à proposer des prises à partir d’exemples. La synthèse de Bohg et al. les organise aussi selon la connaissance de l’objet : connu, familier ou inconnu [15].

Pour des objets de forme simple et connue — des primitives géométriques —, une approche *géométrique et heuristique* suffit : on dérive la prise directement de la géométrie mesurée de l’objet (position, dimensions, axe principal), sans apprentissage. Elle est plus légère qu’une méthode apprise et, surtout, *transparente* : chaque choix de prise se justifie par une règle explicite.

Dans ce travail, nous adoptons une planification de saisie géométrique : l’ouverture et l’orientation de la pince sont dérivées de la géométrie mesurée de l’objet, tandis que l’angle d’attaque est choisi selon la *position* de l’objet (distance et hauteur) plutôt que selon sa forme. La stratégie complète et les limites de cette adaptation sont détaillées aux chapitres 5 et 8.

2.5 Planification de trajectoires

Une fois la prise planifiée, il faut amener la pince à la configuration voulue par un mouvement réalisable. On distingue deux niveaux *complémentaires*. La *planification de chemin* ne traite que la *géométrie* : trouver une suite de configurations menant du départ à l’arrivée en évitant les obstacles, sans *dimension temporelle* (indépendamment de la vitesse à laquelle on le parcourt) ; les méthodes par échantillonnage, comme les arbres d’exploration aléatoire (RRT), y font référence [16]. La *génération de trajectoire* ajoute la *dimension temporelle* : elle associe à chaque point du chemin un instant de passage, ce qui fixe la vitesse et l’accélération du mouvement, de façon lisse et dans les limites du robot [17].

Dans ce travail, en l’absence d’obstacles, aucune planification de chemin n’est nécessaire : le mouvement est une interpolation polynomiale *quintique* directe dans l’espace articulaire (lisse, à vitesse et accélération bornées). L’évitement d’obstacles n’est pas traité (objectif 3, cf. chapitre 8) ; la dérivation de la quintique est présentée au chapitre 6.

2.6 Apprentissage par imitation (ACT)

À l’opposé de la chaîne explicite, l’*apprentissage par imitation* (*imitation learning*) apprend directement une politique — de l’image aux commandes moteur — à partir de démonstrations humaines, sans décomposer perception, planification et contrôle. Plusieurs architectures récentes relèvent de ce paradigme : ACT (*Action Chunking with Transformers*) [18], qui prédit des *séquences* (*chunks*) d’actions à l’aide d’un réseau Transformer ; la *Diffusion Policy* [19], qui génère les actions par un processus de débruitage ; ou les modèles vision-langage-action tels que SmolVLA [20]. Ces approches reproduisent bien les gestes dans la distribution des démonstrations, mais restent sensibles aux écarts à cette distribution.

Dans ce travail, nous mettons en œuvre l'apprentissage par imitation avec une politique *ACT*, comme seconde méthode mise en regard du pipeline explicite, sur le même bras et la même tâche. Parmi les politiques légères supportées par LeRobot, nous retenons *ACT*, référence courante pour le SO-101 et entraînable sur du matériel modeste ; un modèle vision-langage-action comme SmolVLA serait plus lourd et ferait double emploi avec le ciblage par le langage déjà assuré par OWL-ViT côté pipeline. Le protocole d'entraînement et les résultats sont présentés au chapitre 7.

2.7 Positionnement de ce travail

Ce travail se situe à la croisée de ces domaines. Nous assemblons une *pipeline modulaire* (des étapes indépendantes : calibration œil-main, détection par couleur et par langage, triangulation stéréo, planification de saisie géométrique, contrôle) et *explicite* (chaque décision suit une règle ou une équation interprétable), sur un bras à bas coût, et nous la mettons en regard d'une politique apprise de bout en bout (*ACT*) sur le même matériel. L'accent ne porte pas sur une brique isolée, déjà bien couverte par la littérature, mais sur leur *orchestration* et sur la *comparaison* des deux paradigmes. Conformément au cahier des charges, la prise de décision repose sur des règles géométriques et heuristiques, l'apprentissage étant exploré comme extension ; les aspects non traités — évitement d'obstacles, scène fortement encombrée — sont assumés et discutés au chapitre 8.

Chapitre 3

Architecture

3.1 Vue d'ensemble

Faire saisir un objet à un bras robotique à partir d'images se décompose naturellement en trois étapes : *percevoir* la scène pour y localiser la cible, *planifier* une prise et le mouvement qui y mène, puis *exécuter* ce mouvement sur le bras. Cette décomposition modulaire — la lignée « percevoir, planifier, agir » de la robotique classique — constitue le premier des deux paradigmes étudiés dans ce mémoire. Le second, l'apprentissage par imitation (chapitre 2), remplace ces étapes par une politique unique apprise à partir de démonstrations. Le présent chapitre décrit l'architecture logicielle qui réalise le pipeline modulaire, puis précise comment les deux approches se situent l'une par rapport à l'autre au niveau du système.

Le pipeline modulaire est orchestré par une classe centrale, `PickAndPlacePipeline`, qui enchaîne les trois étages sur un cycle complet de *pick-and-place*. À chacun correspond un sous-ensemble du code source : la *perception* (`src/perception`) capture les caméras et reconstruit la position tridimensionnelle de la cible ; la *planification* (`src/planning`) en déduit une prise, que la *cinématique inverse* traduit en angles articulaires ; le *contrôle* (`src/control`) engendre la trajectoire et la joue sur les moteurs. Deux modules transverses complètent l'ensemble : la *calibration* (`src/calibration`), exécutée hors ligne, fournit les paramètres géométriques que le pipeline consomme ; un module d'*utilitaires* (`src/utils`) regroupe les opérations sur les transformations rigides — le langage commun des changements de repère, sur lequel repose tout le reste.

Dans ce travail, un cycle complet se déroule ainsi : les trois caméras et l'état articulaire du bras sont capturés de façon synchronisée ; un détecteur 2D (seuillage couleur HSV ou détecteur appris) localise les objets dans chaque image ; leur position 3D est reconstruite par triangulation stéréo dans le repère de base du robot, et la cible désignée est sélectionnée dans la scène. La prise est ensuite planifiée — ouverture et orientation des mâchoires d'après la géométrie mesurée de l'objet, angle d'attaque d'après sa position — puis convertie en angles articulaires par cinématique inverse. Le bras rejoint une pose d'approche par une trajectoire *quintique*, après quoi la caméra de poignet (*eye-in-hand*, `cam_2`) corrige la prise en boucle fermée avant la descente finale ; la pince se referme sous asservissement du couple, et la tenue est vérifiée à la levée avant la dépose et le retour. Ces étapes sont détaillées aux chapitres 4 à 6.

Ce qui tient l'ensemble n'est pas tant l'orchestrateur que la *frontière de types* entre les étages. Les modules ne communiquent qu'à travers un petit nombre de structures de données explicites : une image calibrée (**Frame**), une détection dans le plan image (**Detection2D**), un objet reconstruit en 3D (**ObjectInstance**), l'état instantané de la scène (**Scene**) et une prise candidate (**GraspPose**). Chaque étage ne consomme que les types produits par le précédent, ce qui rend les frontières nettes et le système vérifiable étage par étage. Ces interfaces, et les patrons de conception qui les accompagnent, font l'objet de la section 3.2.

L'apprentissage par imitation réutilise le même matériel — le même bras, le même bus moteur et les mêmes caméras — mais court-circuite cette chaîne explicite : une politique *ACT* apprend à produire directement les commandes articulaires à partir des images et de l'état du bras, sans reconstruction 3D ni cinématique inverse. Sa propre chaîne — téléopération pour enregistrer des démonstrations, entraînement, puis évaluation — s'appuie sur l'écosystème LeRobot et est décrite au chapitre 7. Confronter ces deux architectures sur la même tâche et le même bras est l'un des fils conducteurs du mémoire.

La figure 3.1 résume ce flux. La suite du chapitre précise les interfaces et patrons de conception (section 3.2), les repères et conventions géométriques qui circulent entre les modules (section 3.3), la correspondance entre logiciel et matériel (section 3.4), et enfin les trois modes d'exécution — *live*, *dry-run* et *replay* (section 3.5).

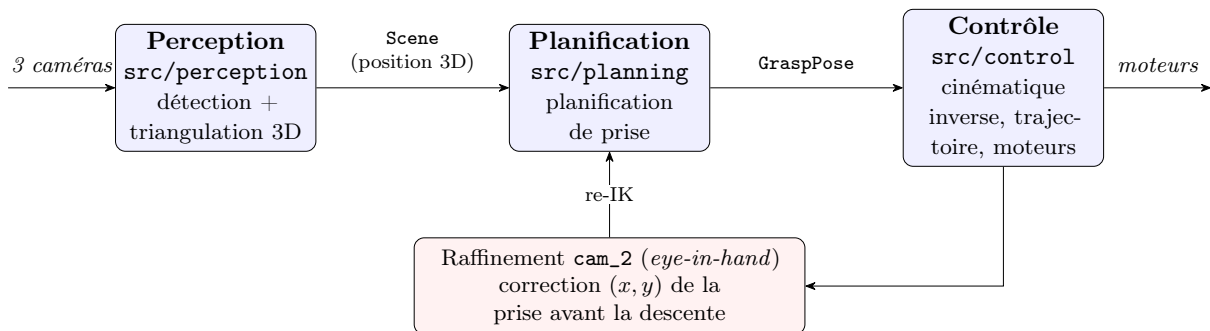


FIGURE 3.1 – Vue d'ensemble du pipeline modulaire perception–planification–contrôle.

3.2 Interfaces et patrons de conception

La séparation nette annoncée à la section précédente ne tient pas d'elle-même : elle repose sur quelques choix de conception délibérés. Quatre méritent d'être explicités — un modèle de données partagé, des algorithmes interchangeables derrière des interfaces abstraites, un comportement piloté par configuration, et une abstraction de l'état du robot.

Un modèle de données partagé. Les étages n'échangent qu'un petit vocabulaire de structures de données. Une **Frame** est une image calibrée : les pixels, accompagnés de la matrice intrinsèque, des coefficients de distorsion et de la pose de la caméra dans le repère de base. Une **Detection2D** est une détection dans le plan image : un label, un centre, une boîte englobante ou un contour, un score. Une **ObjectInstance** est l'objet reconstruit en 3D dans le repère de base : sa position, une covariance qui en mesure la fiabilité, son encombrement et, le cas échéant, son orientation.

Une **Scene** rassemble les objets — et d'éventuels obstacles — à un instant donné ; une **GraspPose** décrit une prise : les trois poses successives de la pince (approche, saisie, retrait) et ses ouvertures. Une règle structure ce modèle : une fois l'objet reconstruit en 3D, tout l'aval raisonne en mètres dans le repère de base et ne revient jamais aux pixels. Ces structures portent une convention géométrique commune, détaillée à la section 3.3.

Des algorithmes interchangeables (patron *stratégie*). Deux étapes admettent plusieurs réalisations : la détection d'objets et la planification de la prise. Chacune est isolée derrière une interface abstraite — **ObjectDetector**, qui transforme une **Frame** en détections, et **GraspStrategy**, qui transforme un objet en prise. Les variantes concrètes en dérivent : le détecteur par seuillage **HSVDetector** et le détecteur appris **HFDetector** d'un côté, la stratégie de saisie géométrique de l'autre. C'est le patron de conception dit *stratégie* : l'orchestrateur manipule l'interface, jamais l'implémentation. Nous pouvons ainsi substituer un détecteur à l'autre sans toucher au reste du pipeline, ce qui rend immédiate la comparaison expérimentale des deux détecteurs (chapitre 7) : même pipeline, seule la stratégie change. À l'exécution, ce choix se réduit à une option de ligne de commande :

```
python scripts/pick_and_place.py --target orange_cube --detector hf
```

La stratégie de saisie effectivement employée est la stratégie *géométrique*, dont la prise verticale (*top-down*) n'est qu'un cas particulier ; elle est détaillée au chapitre 5.

Un comportement piloté par configuration. Les réglages du pipeline ne sont pas codés en dur mais rassemblés dans une configuration. Un objet **PipelineConfig** regroupe les hyperparamètres d'une exécution — l'objet cible, le détecteur à utiliser, les seuils et marges, les options d'affichage — tandis que les données issues de la calibration et la description de la scène résident dans des fichiers JSON sous **configs/** : intrinsèques et calibration œil-main de chaque caméra, géométrie de la scène, spécifications des détecteurs. Changer de cible, de détecteur ou de réglage ne demande donc pas de modifier le code, et les paramètres géométriques produits par la calibration (chapitre 4) sont simplement chargés au démarrage.

Une abstraction de l'état du robot. Enfin, le pipeline ne lit jamais les moteurs directement : il passe par un **RobotStateProvider**, qui fournit l'état du bras — les angles articulaires et la pose de l'*effecteur* (la pince), cette dernière calculée par cinématique directe — à partir de trois sources interchangeables : la lecture en temps réel du bus moteur Feetech, un jeu d'angles fixes (pour les tests), ou des positions d'encodeur rejouées depuis un enregistrement. Les trois produisent le même type d'objet, si bien que tout l'aval du pipeline est identique, que le robot soit branché ou non. C'est exactement ce qui rend possibles les modes d'exécution *live*, *dry-run* et *replay* décrits à la section 3.5.

3.3 Repères et conventions

Les structures de données de la section précédente transportent toutes des grandeurs géométriques — positions, orientations, poses — qui n'ont de sens que rapportées à un *repère*. Cette

section fixe les repères du système, la convention qui nomme les transformations entre eux, et la petite boîte à outils qui les manipule. C’est le langage géométrique commun, que nous retrouvons — cette fois démontré — dans la perception (chapitre 4) et la cinématique (chapitre 6).

Les repères. Six repères interviennent : la *base* du robot — repère fixe, racine de tous les autres —, les deux caméras fixes `cam_0` et `cam_1` (*eye-to-hand*), la caméra de poignet `cam_2` (*eye-in-hand*), la *pince* et l’*objet* cible. Tout ce que le pipeline calcule est, en définitive, exprimé dans le repère de base.

La convention de nommage. On note $T_{A \leftarrow B}$ — dans le code, `T_A_B` — la pose du repère B exprimée dans le repère A ; c’est aussi la matrice qui convertit les coordonnées d’un point du repère B vers le repère A . Cette convention unique est employée à l’identique dans la perception, la planification et la calibration. Elle se compose en enchaînant les repères,

$$T_{A \leftarrow C} = T_{A \leftarrow B} T_{B \leftarrow C},$$

où le repère intermédiaire B « s’annule » visuellement, et s’inverse en échangeant les deux repères : $T_{B \leftarrow A} = T_{A \leftarrow B}^{-1}$.

Représentation : matrices homogènes. Une pose réunit l’*orientation* et la *position* d’un repère : une rotation R et une translation t . La rotation est une matrice 3×3 *orthonormale* de déterminant $+1$ — ses colonnes forment un trièdre direct de vecteurs unitaires, ce qui en fait une rotation *pure*, sans mise à l’échelle ni effet miroir. On regroupe les deux dans une unique matrice 4×4 dite *homogène*,

$$T = \begin{bmatrix} R & t \\ \mathbf{0} & 1 \end{bmatrix},$$

obtenue en adjoignant aux points une quatrième coordonnée égale à 1. L’intérêt est le suivant : sous cette forme, la rotation *et* la translation s’expriment par une *seule* multiplication matricielle (la translation seule serait une addition). Enchaîner plusieurs poses revient alors à multiplier leurs matrices, et déplacer un point se fait par un simple produit. Les positions sont en mètres. La boîte à outils `src/utils/transforms.py` implémente exactement ces opérations — composition, et inversion (qui combine la transposée R^T de la rotation et la translation $-R^T t$) — ainsi que les conversions vers les représentations utilisées aux frontières : le couple axe-angle (`rvec`, `tvec`) d’OpenCV pour le code de calibration, et les angles *roll-pitch-yaw* de l’URDF pour le modèle cinématique. Ces fondements — pourquoi ces matrices, comment elles agissent sur les points — sont développés au chapitre 4.

Les deux chaînes clés. Deux transformations reviennent constamment et illustrent la convention. Pour une caméra *fixe* (*eye-to-hand*), la pose dans la base $T_{\text{base} \leftarrow \text{cam}}$ est *constante* : c’est exactement le résultat de la calibration œil-main (chapitre 4). Pour la caméra de *poignet* (*eye-in-hand*), la pose n’est pas constante — la caméra suit le bras — et se reconstruit en composant la

pose courante de la pince avec la transformation (fixe) de la caméra sur la pince :

$$T_{\text{base} \leftarrow \text{cam}2} = T_{\text{base} \leftarrow \text{pince}} T_{\text{pince} \leftarrow \text{cam}2},$$

où $T_{\text{base} \leftarrow \text{pince}}$ provient elle-même de la *cinématique directe* appliquée aux angles articulaires courants (chapitre 6). La même algèbre traverse tout le système : la triangulation place l’objet dans le repère de base (chapitre 4), la planification y exprime la pose visée de la pince (chapitre 5), et la cinématique inverse retraduit cette pose en angles articulaires (chapitre 6). Ces changements de repère — les matrices T — relient tous les étages du projet ; la figure 3.2 en donne une vue d’ensemble.

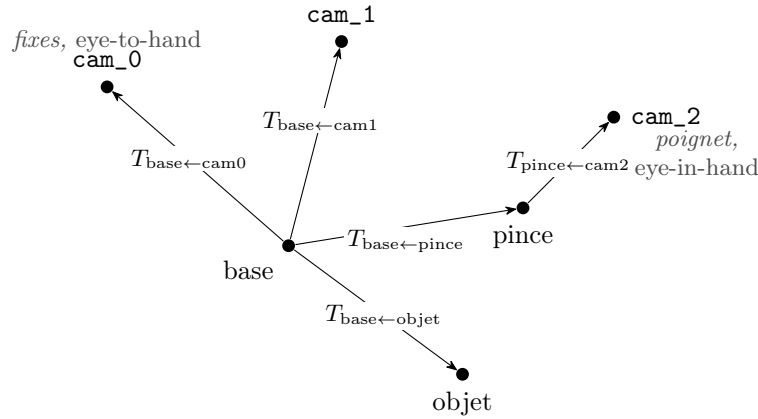


FIGURE 3.2 – Repères du système et transformations $T_{A \leftarrow B}$ qui les relient. Les poses des caméras fixes (*eye-to-hand*) sont constantes ; celle de la caméra de poignet (*eye-in-hand*) se compose via la pince, $T_{\text{base} \leftarrow \text{cam}2} = T_{\text{base} \leftarrow \text{pince}} T_{\text{pince} \leftarrow \text{cam}2}$.

3.4 Lien matériel–logiciel

L’architecture présentée jusqu’ici est *logique* : des modules, des types, des repères. Cette section la rattache aux dispositifs physiques. Le matériel lui-même — bras, caméras, pince, enceinte — a été présenté en introduction ; nous précisons seulement ici comment le logiciel s’y connecte. Deux modules jouent le rôle de *pilotes*, et une couche de calibration fait le pont entre les mesures brutes du matériel et les grandeurs géométriques que le pipeline manipule (table 3.1).

Deux pilotes. Le module **MotorController** commande le bus moteur : il associe les six noms logiques de moteurs (**shoulder_pan**, **shoulder_lift**, **elbow_flex**, **wrist_flex**, **wrist_roll** et **gripper**) aux identifiants des servomoteurs Feetech, envoie des consignes d’angles et d’ouverture de la pince, suit une trajectoire en respectant ses horodatages, et lit en retour les positions ainsi que la *charge* du moteur — une mesure de couple que nous utilisons, faute de capteur d’effort dédié, pour détecter qu’un objet est bien tenu (chapitre 6). Il s’appuie sur l’implémentation du protocole Feetech fournie par LeRobot. Le module **MultiCamera**, lui, pilote les trois caméras USB — la paire stéréo fixe (*eye-to-hand*) et la caméra de poignet (*eye-in-hand*) — et restitue les images synchronisées sous forme de **Frame**.

La calibration comme pont. C’est le point d’architecture décisif : les mesures brutes du matériel — des pixels, des comptes d’encodeur — ne deviennent les grandeurs métriques du pipeline qu’à travers les données de *calibration* rangées dans *configs/*. Les intrinsèques et la calibration œil-main donnent à chaque **Frame** sa matrice K et sa pose $T_{\text{base} \leftarrow \text{cam}}$; la calibration moteur convertit les comptes d’encodeur en angles articulaires, donc — par cinématique directe — en pose de la pince. Ces fichiers sont produits une fois par les procédures de calibration (chapitre 4 et annexe A) et sont propres à *ce* robot : c’est par eux que la géométrie de la section 3.3 se trouve « câblée » au matériel réel.

Une configuration centralisée. Enfin, les éléments susceptibles de changer d’une machine ou d’un branchement à l’autre — ports USB, index et résolution des caméras — sont rassemblés dans un unique fichier de configuration. Les noms logiques (*cam_0*, *cam_1*, *cam_2*) restent fixes ; seuls les index matériels varient. Rebrancher le matériel ou porter le projet sur un autre ordinateur ne demande donc d’éditer qu’un seul endroit.

Matériel	Module (pilote)	Calibration (<i>configs/</i>)
5 articulations + pince (bus Feetech)	MotorController	calibration_follower.json
Paire stéréo fixe (<i>eye-to-hand</i>)	MultiCamera	calibration_cam_0/1.json, handeye_cam_0/1.json
Caméra de poignet (<i>eye-in-hand</i>)	MultiCamera	calibration_cam_2.json, handeye_cam_2.json

TABLE 3.1 – Correspondance entre dispositifs matériels, modules pilotes et fichiers de calibration.

3.5 Modes d’exécution : *live*, *dry-run* et *replay*

Les abstractions de la section 3.2 — une entrée caméra et une source d’état du robot interchangeables, produisant toujours les mêmes types — ont une conséquence pratique : le *même* code de pipeline peut s’exécuter de trois manières, qui ne diffèrent que par l’origine des données et par le fait de commander ou non le robot (table 3.2).

Live. C’est le mode de production. Les trois caméras USB sont capturées en temps réel, l’état du bras est lu sur le bus moteur, et les trajectoires calculées sont effectivement envoyées aux moteurs : le robot perçoit et agit. C’est ce mode qui réalise un cycle de *pick-and-place* réel.

Dry-run. Ici, tout est calculé — perception, planification, cinématique inverse, trajectoires — mais *aucune commande n’est envoyée aux moteurs*, et l’état du bras est pris comme une pose de référence fixe plutôt que lu en continu. Ce mode sert à valider la chaîne de décision sans risquer un mouvement ; il s’active aussi automatiquement, en repli, lorsque le bus moteur ne peut être ouvert.

Replay. Enfin, l’architecture rend possible de rejouer une séquence *enregistrée* : au lieu des caméras live, le pipeline lit des images pré-enregistrées accompagnées des angles du bras au moment de la capture. Ni le robot ni les caméras n’ont alors besoin d’être connectés — le bras

n'étant pas commandé, ni la liaison au PC ni l'alimentation moteur ne sont requises. L'intérêt est double : itérer sur la perception — par exemple comparer deux détecteurs — hors-ligne, et obtenir des résultats *exactement* reproductibles, une capture figée ne variant pas d'une exécution à l'autre. Ce mode est une capacité offerte par la conception plus qu'un passage obligé du développement.

Dans les trois cas, le code du pipeline est identique : seules changent la source des données et la décision de commander ou non les moteurs. C'est le bénéfice concret de la frontière de types (section 3.2), et cela clôt la description de l'architecture. Les chapitres suivants entrent dans chacun des étages : la perception (chapitre 4), la planification de saisie (chapitre 5) et le contrôle (chapitre 6).

Mode	Images	État du bras	Moteurs	Usage
<i>live</i>	caméras USB	lu sur le bus	oui	exécution réelle (production)
<i>dry-run</i>	caméras USB	pose fixe	non	valider sans mouvement ; repli si le bus est injoignable
<i>replay</i>	fichiers enregistrés	rejoué (manifeste)	non	perception hors-ligne, reproductible

TABLE 3.2 – Les trois modes d'exécution du pipeline. Le code applicatif est identique ; seuls changent la source des données et la commande des moteurs.

Chapitre 4

Perception

4.1 Pipeline de perception

La perception est le premier étage du pipeline : sa tâche est de transformer les images brutes des caméras en une information exploitable par la planification — avant tout, la *position tridimensionnelle* de l’objet cible dans le repère de base du robot. Elle s’exprime entièrement dans les types de données du chapitre 3 : des **Frame** en entrée, une **Scene** peuplée d’**ObjectInstance** en sortie.

Le traitement enchaîne quatre opérations. D’abord la *capture* : les trois caméras sont lues de façon synchronisée, chacune accompagnée de ses paramètres de calibration et de sa pose dans le repère de base (section 4.2). Ensuite la *détection 2D* : dans chaque image, un détecteur localise les objets connus et renvoie, pour chacun, une boîte englobante ou un contour dans le plan image ; deux détecteurs interchangeables sont étudiés, un seuillage couleur (section 4.3) et un détecteur appris piloté par le langage (section 4.4). Puis la *reconstruction 3D* : les détections d’un même objet dans les deux caméras fixes sont appariées, puis *triangulées* pour retrouver sa position dans le repère de base (section 4.5). Enfin le *filtrage* : les positions hors de l’espace de travail, ou tombant dans une zone d’exclusion connue, sont écartées avant de constituer la scène.

Dans ce travail, la reconstruction 3D repose sur la paire stéréo fixe `cam_0–cam_1` ; la caméra de poignet `cam_2` sert surtout au raffinement de la prise (chapitre 6). Le filtrage s’appuie sur une description de la scène qui évite, par exemple, de confondre le filament orange du robot lui-même avec le cube orange visé. Un repli monoculaire par *PnP* est prévu lorsque la stéréo échoue, mais il reste peu opérant sur les formes étudiées — ce point est discuté à la section 4.5.

Au-delà de la position, la perception estime aussi l’*encombrement* de l’objet — une boîte englobante tridimensionnelle approchée — ainsi que des indices d’orientation. Ces grandeurs alimentent la planification de saisie, qui en déduit l’ouverture et l’alignement des mâchoires (chapitre 5).

La suite du chapitre détaille ces éléments dans l’ordre où ils se composent : les modèles de caméra et la calibration qui fondent toute la géométrie (section 4.2), les deux détecteurs 2D (sections 4.3 et 4.4), la reconstruction 3D par triangulation et sa sensibilité au bruit (section 4.5), la synchronisation des caméras (section 4.6), et enfin la validation expérimentale de la perception (section 4.7).

4.2 Calibration et modèles caméra

Toute la géométrie de la perception repose sur deux ingrédients : un *modèle* qui relie un point de l'espace au pixel où la caméra le voit, et une *calibration* qui en mesure les paramètres pour chaque caméra. Cette section établit d'abord le modèle sténopé et sa distorsion (les paramètres *intrinsèques*, internes à l'optique), puis la calibration œil-main qui place la caméra dans le repère du robot — les paramètres *extrinsèques* —, et enfin la chaîne de repères qui en résulte.

Le modèle sténopé. Une caméra est modélisée comme un *sténopé* (*pinhole*) : tout rayon lumineux passe par un unique centre optique avant de frapper le plan image. Concrètement, la lumière émise par un point de la scène traverse ce centre et vient frapper le *capteur* — une grille de pixels — en un seul endroit ; c'est la position de ce point d'impact, en pixels, que le modèle cherche à calculer. Comme tous les rayons convergent vers le même centre, un objet éloigné se projette sur une plus petite portion du capteur qu'un objet proche : c'est l'effet de *perspective*, que traduit la division par la profondeur. Un point $P_{\text{cam}} = (X, Y, Z)$ exprimé dans le repère de la caméra — l'axe Z pointant vers la scène — se projette alors en deux temps (figure 4.1). D'abord une *division perspective*, qui traduit qu'un objet paraît d'autant plus petit qu'il est loin :

$$x = \frac{X}{Z}, \quad y = \frac{Y}{Z},$$

où (x, y) sont les coordonnées sur le plan image *normalisé*. Ensuite une mise à l'échelle par les distances focales, puis une translation par le point principal pour exprimer le résultat en pixels :

$$u = f_x x + c_x, \quad v = f_y y + c_y,$$

où (u, v) est le pixel obtenu, f_x et f_y les *distances focales* (en pixels) et (c_x, c_y) le *point principal*. Les deux étapes se regroupent sous forme matricielle, en coordonnées homogènes, à l'aide de la matrice *intrinsèque* K :

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}, \quad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \quad s = Z.$$

Le facteur d'échelle $s = Z$ indique que la projection *perd* la profondeur : une seule image ne permet pas de retrouver Z — c'est l'objet de la reconstruction 3D (section 4.5). Les paramètres de K se lisent physiquement : les distances focales prennent deux valeurs f_x et f_y parce qu'un pixel n'est pas nécessairement carré. Le point principal (c_x, c_y) est l'intersection de l'axe optique avec le plan image : comme les pixels se comptent depuis un *coin* de l'image, ses coordonnées avoisinent la moitié des dimensions (d'où des valeurs proches de (960, 540) pour une image 1920×1080). Il ne coïncide pas pour autant avec ce centre géométrique : c'est une grandeur *mesurée* par la calibration, et non le centre supposé de l'image. Enfin, ces paramètres sont *internes* à la caméra : ils ne dépendent pas de l'endroit où elle se trouve et valent pour toutes ses images.

Dans ce travail, chaque caméra est calibrée à la résolution de 1920×1080 pixels. Pour `cam_0`, la calibration (décrite plus bas) donne des distances focales $f_x \approx 1226$ et $f_y \approx 1225$ pixels — des pixels donc quasi carrés — et un point principal $(c_x, c_y) \approx (979, 610)$, voisin du centre géométrique de l'image (960, 540).

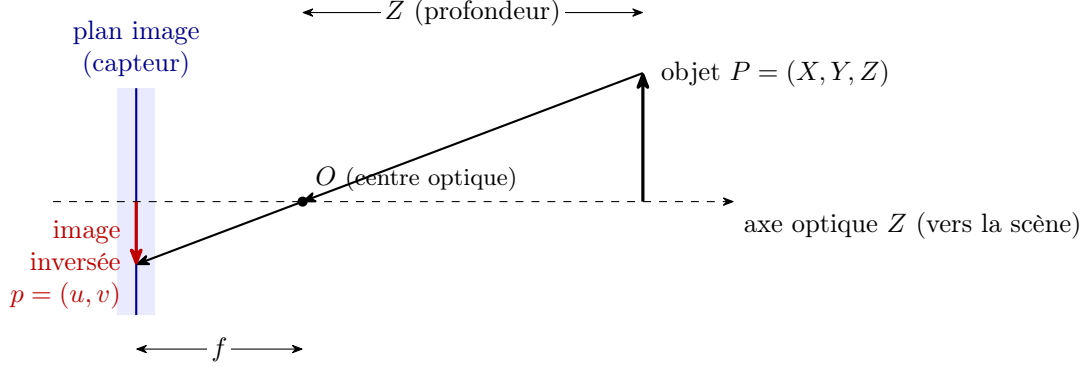


FIGURE 4.1 – Le modèle de caméra sténopée : la lumière d'un point P de la scène traverse le centre optique O et forme une image *inversée* sur le capteur, placé derrière. La division par la profondeur Z produit l'effet de perspective.

La distorsion. Le modèle sténopée idéalise l'optique : il suppose un trou ponctuel parfait. Une vraie caméra emploie en réalité une *lentille* — une pièce de verre *courbe* — pour capter davantage de lumière, et cette courbure ne dévie pas tous les rayons aussi régulièrement qu'un trou idéal : l'écart grandit à mesure que le rayon frappe la lentille loin de son centre, là où il traverse le verre plus obliquement (c'est-à-dire vers les bords de l'image). Visuellement, les lignes droites de la scène apparaissent alors légèrement *bombées*, surtout près des bords — c'est la *distorsion*. Sa composante principale est *radiale* (les bords « gonflent » : effet barillet ; ou « se creusent » : effet coussinet) ; s'y ajoute une faible distorsion *tangentielle*, due à un léger défaut de parallélisme entre la lentille et le capteur. On la modélise sur les coordonnées normalisées (x, y) , avant le passage en pixels. En posant $r^2 = x^2 + y^2$ (le carré de la distance au centre optique), le modèle employé — celui d'OpenCV — s'écrit

$$\begin{aligned} x_d &= x (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2 (r^2 + 2x^2), \\ y_d &= y (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1 (r^2 + 2y^2) + 2p_2 xy, \end{aligned}$$

où (k_1, k_2, k_3) gouvernent la part radiale et (p_1, p_2) la part tangentielle. Ce sont les coordonnées *distordues* (x_d, y_d) que la caméra forme réellement ; connaître ces coefficients permet de *redresser* l'image avant tout calcul géométrique.

Calibration intrinsèque : la méthode de Zhang. La matrice K et les coefficients de distorsion ne se mesurent pas directement : on les estime par la méthode de *Zhang* [6]. Le principe est de photographier une mire plane connue — un *damier* — sous de nombreuses orientations : chaque vue fournit des contraintes (via l'homographie entre le plan du damier et l'image), et leur ensemble permet de résoudre pour K et la distorsion, puis d'affiner ces paramètres en minimisant l'*erreur de reprojection* — l'écart moyen, en pixels, entre les coins du damier détectés dans l'image

et leur reprojection par le modèle estimé. Cette erreur sert aussi de mesure de qualité de la calibration.

Dans ce travail, la calibration *intrinsèque* de chaque caméra utilise un damier de 7×7 coins internes (carrés de 22,2 mm) présenté sous une quarantaine de poses. Pour `cam_0`, le coefficient radial dominant vaut $k_1 \approx -0,41$ (une distorsion en barillet), les termes tangentiels étant négligeables (capteur bien aligné), et l'erreur de reprojection finale n'est que de 0,19 pixel — une précision *sous-pixellique*. La qualité de calibration des trois caméras est reprise à la section 4.7.

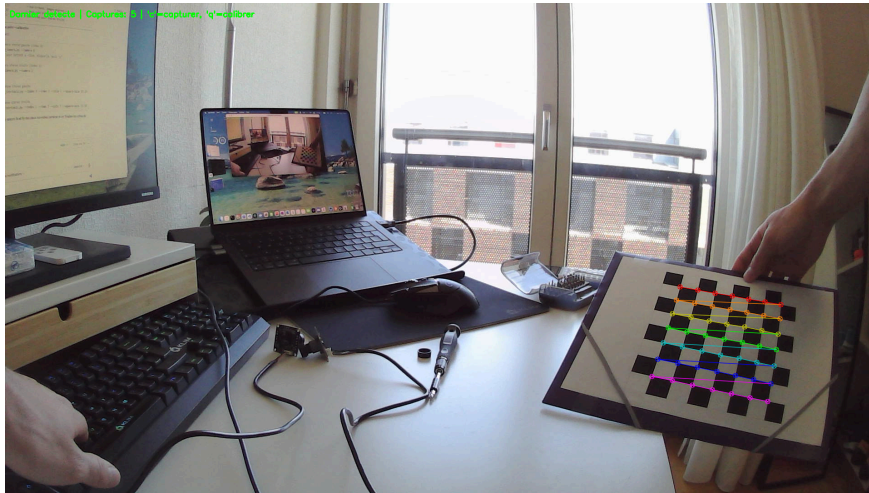
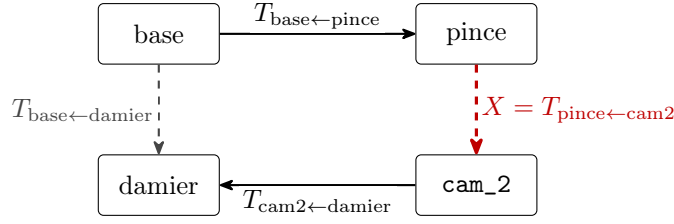


FIGURE 4.2 – Calibration intrinsèque par la méthode de Zhang : le damier est présenté à la caméra sous différentes orientations. La même procédure s'applique aux trois caméras.

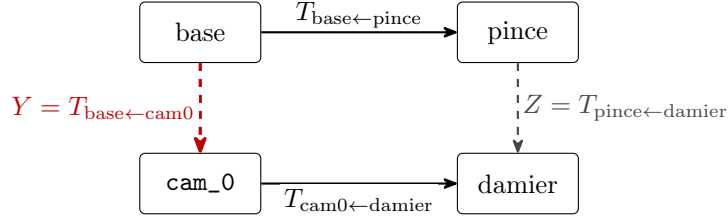
Calibration œil-main : où est la caméra par rapport au robot ? Détecter un objet dans une image ne donne qu'un *pixel* ; pour que le robot agisse dessus, il lui faut savoir *où se trouve la caméra par rapport à lui*. Sans cela, voir l'objet ne suffit pas : on ignorerait dans quelle direction diriger le bras. Déterminer la position et l'orientation exactes de chaque caméra dans le repère de base du robot, c'est l'objet de la calibration *œil-main* (*hand-eye*), dont le principe a été posé au chapitre 2 et que l'on établit ici sur les repères du système.

On ne peut pas relever cette pose à la règle — l'orientation, surtout, exige une précision qu'aucune mesure manuelle n'atteint. On la *calibre* donc, en montrant à la caméra une mire connue — un *damier* — dans de nombreuses poses du bras. Chaque prise fournit alors deux transformations, et deux seulement : la pose du bras $T_{\text{base} \leftarrow \text{pince}}$, donnée par la *cinématique directe* (chapitre 6), et la pose du damier vue par la caméra $T_{\text{cam} \leftarrow \text{damier}}$ — où *cam* désigne la caméra en cours de calibration —, obtenue en détectant les coins du damier dans l'image puis en résolvant le problème de perspective (*PnP*, fonction `solvePnP`). Tout l'enjeu est de combiner ces deux mesures pour en extraire la pose, inconnue, de la caméra ; et la manière de les combiner dépend du *montage* (les deux cas sont schématisés en figure 4.3).

Reste à expliquer comment se *mesure* cette seconde pose, $T_{\text{cam} \leftarrow \text{damier}}$: c'est le modèle sténopé du début de section, appliqué *en sens inverse*. Ce modèle envoyait un point connu du repère caméra vers un pixel ; notons π_K cette projection complète (division perspective, distorsion, puis



(a) *Eye-in-hand* (cam_2) : le damier est fixe sur la table, donc $T_{base←damier}$ est une constante que l'on élimine ; l'unique inconnue cherchée est X .



(b) *Eye-to-hand* (cam_0) : le damier est porté par la pince ; il y a donc *deux* constantes inconnues, la cible Y et l'auxiliaire Z que l'on élimine.

FIGURE 4.3 – Les deux montages vus comme des *boucles de repères*. Chaque flèche de A vers B porte la transformation $T_{A←B}$ (la pose de B dans A , chapitre 3). Dans chaque boucle, les *deux* chemins menant au damier sont égaux : cette égalité, écrite pour deux poses du bras, fournit l'équation $A X = X B$. **Trait noir plein** : grandeur mesurée à chaque pose — $T_{base←pince}$ par cinématique directe, $T_{cam←damier}$ par PnP. **Trait tireté** : transformation *constante* inconnue — en rouge la pose de caméra cherchée, en gris l'auxiliaire éliminée par le calcul.

passage en pixels par K). Ici on fait l'inverse : on connaît les pixels et l'on cherche la pose. Chaque coin du damier a une position 3D connue M_k (la mire est une grille plane régulière), se projette en un pixel m_k que l'on détecte dans l'image, et K a déjà été calibrée ; la seule inconnue est donc la pose qui, reportée dans π_K , ramène les coins projetés sur les pixels observés. On retient celle qui minimise leur écart total — c'est-à-dire exactement l'*erreur de reprojection* déjà rencontrée pour la calibration de Zhang, mais à K figé :

$$T_{cam←damier} = \arg \min_T \sum_k \left\| \pi_K(T M_k) - m_k \right\|^2.$$

Avec $6 \times 9 = 54$ coins pour seulement six inconnues de pose, le problème est très surdéterminé ; c'est le problème classique *Perspective-n-Point* (PnP), que l'on résout par `solvePnP`. On dispose ainsi, à chaque prise, des deux transformations voulues.

Premier montage : la caméra de poignet (*eye-in-hand*). La caméra `cam_2` est montée sur le bras et se déplace avec lui (montage schématisé en figure 4.3a). Nous posons donc le damier *fixe* sur la table, et déplaçons le bras pour le lui faire voir sous des angles variés. L'inconnue cherchée est la pose, *constante*, de la caméra par rapport à la pince, $X = T_{pince←cam_2}$. Pour l'isoler, on écrit qu'à chaque pose i le damier occupe la *même* place dans le repère de base — il ne bouge pas —, en parcourant la chaîne $base \rightarrow pince \rightarrow cam_2 \rightarrow damier$:

$$T_{base←damier} = T_{base←pince}(i) X T_{cam_2←damier}(i),$$

le membre de gauche étant une constante (la place du damier sur la table). En égalant cette relation pour deux poses i et j , cette constante s'élimine ; après réarrangement, il reste

$$\underbrace{T_{\text{base} \leftarrow \text{pince}}(j)^{-1} T_{\text{base} \leftarrow \text{pince}}(i)}_{A : \text{déplacement de la pince}} X = X \underbrace{T_{\text{cam2} \leftarrow \text{damier}}(j) T_{\text{cam2} \leftarrow \text{damier}}(i)^{-1}}_{B : \text{déplacement de la caméra}},$$

c'est-à-dire l'équation de calibration œil-main

$$A X = X B.$$

Son sens géométrique se lit directement : entre les poses i et j , la pince effectue un mouvement A et la caméra un mouvement B ; comme la caméra est rigidement vissée à la pince, c'est le *même* déplacement décrit dans deux repères, et la transformation constante X qui passe de l'un à l'autre est précisément ce qui les relie.

Une seule paire de poses ne suffit pas à fixer X — une transformation rigide compte six degrés de liberté —, et sa rotation n'est contrainte que si les poses tournent autour d'axes *différents*. Nous accumulons donc des dizaines de poses bien variées, ce qui *surdétermine* le système $A X = X B$. La méthode de *Horaud* [3], exposée par la fonction `calibrateHandEye` d'OpenCV, en estime la meilleure solution au sens des moindres carrés en optimisant *conjointement* la rotation et la translation de X , ce qui la rend robuste au bruit de mesure (chapitre 2). Concrètement, on *empile* l'équation $A X = X B$ pour toutes les paires de poses et l'on résout ce système surdéterminé : sa résolution numérique relève de la méthode citée et n'est pas reprise ici, tandis que la *qualité* du X obtenu — l'écart résiduel sur l'ensemble des poses — est, elle, mesurée à la section 4.7. Pour `cam_2`, la méthode renvoie la matrice constante $T_{\text{pince} \leftarrow \text{cam2}}$.

La caméra de poignet bougeant avec le bras, sa pose dans la base *change* à chaque instant ; on la reconstruit à la volée en composant la pose courante de la pince (cinématique directe) avec la transformation X que l'on vient de calibrer :

$$T_{\text{base} \leftarrow \text{cam2}}(t) = T_{\text{base} \leftarrow \text{pince}}(t) \underbrace{T_{\text{pince} \leftarrow \text{cam2}}}_{= X}.$$

Second montage : les caméras fixes (*eye-to-hand*). Les deux caméras de la paire stéréo, `cam_0` et `cam_1`, sont au contraire vissées à la structure : elles ne bougent jamais (montage schématisé en figure 4.3b). Le damier est cette fois fixé *sur la pince*, et c'est le bras qui le déplace devant la caméra immobile. L'inconnue recherchée est alors *directement* la pose de la caméra dans la base, $Y = T_{\text{base} \leftarrow \text{cam0}}$. Les rôles sont simplement échangés par rapport au premier montage : c'est désormais le damier (porté par la pince) qui bouge, tandis que la caméra reste fixe. Deux trajets mènent alors au damier — par la caméra fixe, ou par le bras — et doivent coïncider :

$$\underbrace{Y}_{T_{\text{base} \leftarrow \text{cam0}}} T_{\text{cam0} \leftarrow \text{damier}}(i) = T_{\text{base} \leftarrow \text{pince}}(i) \underbrace{Z}_{T_{\text{pince} \leftarrow \text{damier}}},$$

où Z , la pose du damier sur la pince, est une seconde constante inconnue — auxiliaire, dont on ne veut pas. Pour l'écartier, on l'isole,

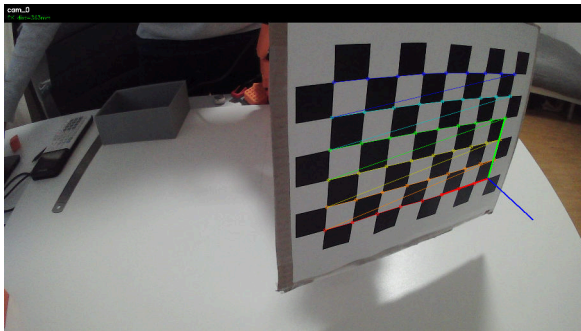
$$Z = T_{\text{base} \leftarrow \text{pince}}(i)^{-1} Y T_{\text{cam0} \leftarrow \text{damier}}(i),$$

puis, Z ne dépendant pas de la pose, on égale ses deux expressions en i et j ; après réarrangement, l'auxiliaire a disparu et il reste

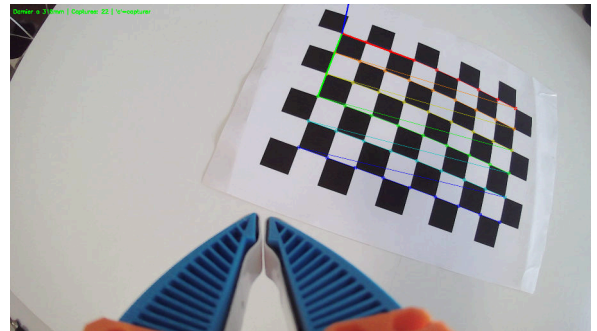
$$\underbrace{T_{\text{base} \leftarrow \text{pince}}(j) T_{\text{base} \leftarrow \text{pince}}(i)^{-1}}_{A'} Y = Y \underbrace{T_{\text{cam0} \leftarrow \text{damier}}(j) T_{\text{cam0} \leftarrow \text{damier}}(i)^{-1}}_{B'}.$$

C'est, terme pour terme, la *même* équation $AX = XB$ que pour la caméra de poignet — à une seule différence : la pose du bras y entre *inversée* (comparer A' à A : $T_{\text{base} \leftarrow \text{pince}}^{-1} = T_{\text{pince} \leftarrow \text{base}}$). Il suffit donc de fournir au même solveur de Horaud les poses du bras *inversées* ; il renvoie alors directement la pose cherchée, $Y = T_{\text{base} \leftarrow \text{cam0}}$. C'est, à la ligne près, ce qu'opère le code (`src/calibration/handeye.py`) : inverser la pose du bras avant l'appel à `calibrateHandEye`, puis relire sa sortie comme une pose caméra-vers-base. Le résultat est une matrice *constante*, réutilisée telle quelle à chaque image. `cam_1` possède, elle, ses propres paramètres *intrinsèques*, mais sa *pose* n'est pas estimée par un hand-eye indépendant : nous résolvons d'abord `cam_0` seule, puis nous *déduisons* celle de `cam_1` en la composant avec la transformation stéréo $T_{\text{cam0} \leftarrow \text{cam1}}$ (mesurée par calibration stéréo du couple). Ce choix garantit la *cohérence* de la paire par construction — essentielle à la triangulation (section 4.7).

Dans ce travail, le damier de calibration extrinsèque est volontairement *asymétrique* — 6×9 coins internes (carrés de 22 mm), soit un nombre *pair* de coins dans un sens et *impair* dans l'autre —, ce qui lève l'ambiguïté d'orientation à 180° d'un damier symétrique et fiabilise l'estimation PnP. Chaque caméra est calibrée par la méthode de Horaud sur plusieurs dizaines de poses du bras (jusqu'à 71 capturées), dont une partie seulement est retenue après rejet automatique des prises mal détectées ; le décompte par caméra et la cohérence des poses obtenues sont reportés à la section 4.7.



(a) *Eye-to-hand* : damier sur la pince.



(b) *Eye-in-hand* : damier sur la table.

FIGURE 4.4 – Calibration œil-main : montage *eye-to-hand* (damier sur la pince) et *eye-in-hand* (damier sur la table). Contrairement à l'intrinsèque, le montage diffère selon le rôle de la caméra.

Composer les deux : la matrice de projection. On dispose maintenant des deux pièces : la matrice intrinsèque K , qui projette un point du repère *caméra* sur un pixel (modèle sténopé, début de section), et la pose extrinsèque $T_{\text{base} \leftarrow \text{cam}}$, qui situe la caméra dans la base (calibration œil-main). En les composant, on obtient un *unique* opérateur qui envoie directement un point du repère *base* sur un pixel — c’est ce dont la reconstruction 3D aura besoin.

Considérons un point $\mathbf{X} = (X, Y, Z, 1)^\top$ du repère base, en coordonnées homogènes. On l’amène d’abord dans le repère caméra par l’inverse de la pose, $T_{\text{cam} \leftarrow \text{base}} = (T_{\text{base} \leftarrow \text{cam}})^{-1}$, puis on le projette par K . Or cette pose est une matrice 4×4 de la forme $\begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$; en n’en gardant que les trois premières lignes — le bloc 3×4 noté $[R \mid t]$ —, les deux étapes se condensent en une seule *matrice de projection* :

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{K [R \mid t]}_P \mathbf{X}, \quad [R \mid t] = T_{\text{cam} \leftarrow \text{base}} \text{ (sans sa dernière ligne)}.$$

La matrice P , de taille 3×4 , résume ainsi *toute* la géométrie d’une caméra — intrinsèque *et* pose — en un seul opérateur. Elle n’en décrit toutefois que la partie *linéaire* : la distorsion, non linéaire (début de section), est retirée des points *avant* d’appliquer P (section 4.5). Pour les deux caméras fixes, P est *constante* ; pour la caméra de poignet, elle se recalcule à chaque instant à partir de la pose courante de la pince (calibration œil-main ci-dessus).

C’est précisément cet opérateur que la reconstruction 3D va *inverser* : avec deux caméras — donc deux matrices P — on remonte d’un couple de pixels à la position 3D d’un point, par *triangulation* (section 4.5).

4.3 Détection 2D — version V1 (HSV)

La détection 2D localise, dans chaque image, les objets connus et en renvoie pour chacun une **Detection2D** — un label, un centre, une boîte englobante, un contour et un score (chapitre 3). Deux détecteurs interchangeables sont étudiés derrière la même interface ; cette section présente le premier, *déterministe* et fondé sur la couleur [11].

Son rôle n’est pas d’être le plus puissant, mais d’*isoler la contribution de la géométrie*. Parce qu’il est reproductible et sans apprentissage, toute erreur de position 3D mesurée en aval ne peut venir que de la calibration ou de la triangulation — jamais d’un réseau dont le comportement varierait d’une exécution à l’autre. C’est donc le détecteur de référence sur lequel reposent la mise au point et la validation (section 4.7).

Pourquoi l’espace HSV. En RGB, la couleur d’un objet se déplace sur les trois canaux à la fois dès que l’éclairage change, ce qui rend tout seuillage fragile. L’espace *HSV* (*teinte*, *saturation*, *valeur*) sépare au contraire la *chrominance* (la teinte H et la saturation S) de la *luminance*, la valeur V . Un objet rouge garde à peu près la même teinte à l’ombre ou en pleine lumière ; seule sa valeur change. Seuiller la couleur en HSV est donc bien plus stable face à l’éclairage qu’en RGB. Un troisième espace, *HSL* (*teinte*, *saturation*, *luminosité*), présente le même avantage — lui aussi isole la teinte —, mais nous lui préférons HSV pour deux raisons.

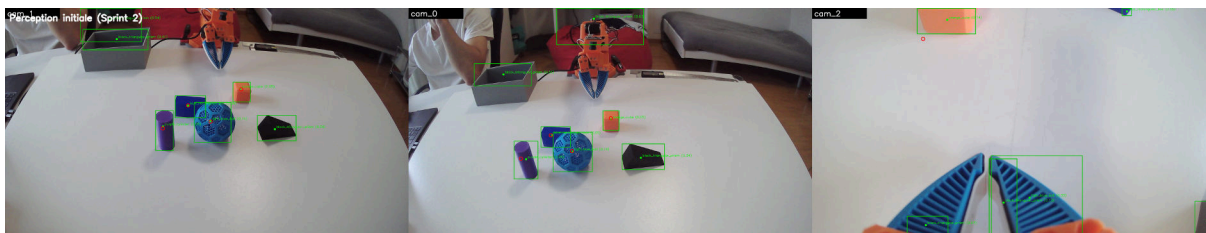


FIGURE 4.5 – Détection HSV, du pixel au contour : seuillage couleur, nettoyage morphologique, puis extraction du contour et de son centre.

D’une part, la *valeur* $V = \max(R, G, B)$ suit directement l’exposition de la caméra, là où la *luminosité* $L = \frac{1}{2}(\max + \min)$ de HSL place une couleur *pure et vive* à mi-échelle ($L \approx 0,5$) et mêle ainsi « vivacité » et « clarté », moins commodes à seuiller séparément. D’autre part, la saturation HSV sépare nettement une couleur vive d’un blanc — un pixel presque blanc y est peu saturé —, tandis que la saturation HSL attribue au contraire une forte valeur aux teintes très *claires*, ce qui brouille le seuillage. Chaque image est donc d’abord convertie en HSV.

De l’image au contour. Chaque classe d’objet est décrite par une `ObjectSpec` : un label (`red_cube`, `green_cylinder...`), une plage HSV et des bornes d’aire. Pour une classe donnée, le détecteur procède en quatre temps. Il construit d’abord un *masque* binaire, qui retient les pixels dont le triplet (H, S, V) tombe dans la plage. Une étape de *morphologie* — une ouverture, qui efface les points isolés, suivie d’une fermeture, qui bouche les trous — nettoie ce masque. On en extrait alors les *contours* des régions connexes, dont on ne garde que celles d’aire comprise entre un minimum (rejet du bruit) et un maximum (rejet des aplats parasites, tel un fond mal éclairé). De chaque contour retenu, enfin, on tire le *centre* — le centroïde, calculé par les moments de l’image —, la boîte englobante et un score (l’aire, normalisée) ; on ne conserve que les k plus grands par classe, par défaut un seul, en supposant un exemplaire visible (figure 4.5).

Le cas des couleurs achromatiques. Un point mérite attention : la teinte n’est *pas définie* pour le noir, le blanc ou le gris. Seuiller un objet noir comme une teinte capterait indistinctement toutes les couleurs sombres. Ces cas sont donc traités à part — le noir par sa seule valeur (V faible, H et S ignorés), le blanc par une faible saturation jointe à une valeur élevée —, ce qui évite des confusions massives entre objets sombres, ou entre objets clairs. De même, le rouge chevauchant la discontinuité de la teinte à $H = 0/179$ (convention OpenCV, où la teinte est codée de 0 à 179), il est décrit par *deux* plages réunies.

Dans ce travail, les plages couleur des primitives sont réglées sous l’éclairage du poste et stockées dans `configs/perception/hsv_specs.json`. Le détecteur autorise des plages *par caméra* : la caméra de poignet (`cam_2`), en vue rapprochée et souvent plus délavée, ne répond pas tout à fait comme la paire stéréo fixe — un même objet y paraît moins saturé —, et distinguer les plages par caméra reste cohérent avec une calibration déjà propre à chacune.

Forces et limites. Ce détecteur est *rapide*, *reproductible* et *sans dépendance d’apprentissage*, ce qui convient à un détecteur de référence. Mais il ne traite que de la couleur : il signale la présence de pixels oranges sans identifier *ce que* c’est. Il ne distingue donc pas le cube orange visé des

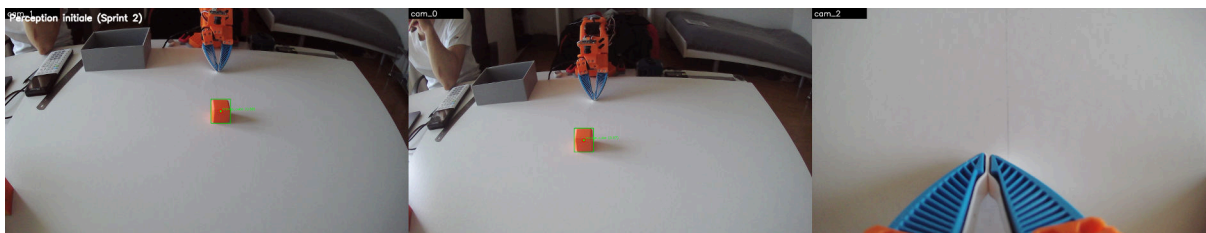


FIGURE 4.6 – Détection à vocabulaire ouvert : le modèle localise chaque objet à partir de sa *description* textuelle et sépare des objets de même couleur par la forme et le contexte.

parties oranges du robot lui-même (d’où le filtrage de scène, section 4.1) ; il exige une calibration des plages sous l’éclairage d’usage ; et il ne tient pas compte de la *forme* ni du *contexte*. Ces limites motivent le second détecteur — *ouvert* et piloté par le langage — présenté à la section suivante.

4.4 Détection 2D — version V2 (*open-vocabulary*)

Le détecteur couleur a une limite : il ne raisonne que sur des pixels, jamais sur le *sens* de ce qu’il voit. Le second détecteur lève cette limite en s’appuyant sur un modèle *vision-langage* pré-entraîné, capable de localiser un objet décrit en *langage naturel* — sans liste de classes figée ni couleur réglée d’avance. C’est la détection à *vocabulaire ouvert* (*open-vocabulary*).

Le principe. On fournit au modèle, en plus de l’image, une *liste de descriptions textuelles* — « *a small bright orange plastic cube* », « *a purple plastic cylinder* »... — et il renvoie, pour chacune, les boîtes englobantes correspondantes assorties d’un score. Entraîné sur des millions de paires image-légende, il a appris à relier les mots à l’apparence : il peut alors séparer le cube orange des *parties oranges du robot* lui-même par la *forme* et le *contexte*, et non par la seule couleur comme le faisait HSV. Changer les objets cibles ne demande alors que de réécrire les descriptions — ni recalibration, ni réentraînement.

Le modèle retenu. Parmi les détecteurs à vocabulaire ouvert recensés au chapitre 2, nous retenons *OWL-ViTv2* [14] (`google/owlv2-base-patch16-ensemble`), servi par la bibliothèque *Transformers* de Hugging Face. Ce choix est cohérent avec l’écosystème du projet : c’est le même *écosystème Hugging Face* (outillage et *Hub* de modèles) que celui des politiques apprises de LeRobot, employées par la seconde approche de ce travail — aucune fragmentation des dépendances.

De la sortie du modèle à une *Detection2D*. Chaque boîte renvoyée au-dessus d’un seuil de confiance devient une *Detection2D* : son centre est le milieu de la boîte, et son « contour » se réduit aux quatre coins de celle-ci. Ce pseudo-contour — faute de segmentation fine — suffit à une prise par le dessus, mais reste trop grossier pour fixer précisément l’orientation du poignet (chapitre 5). La sortie respecte exactement la même interface que le détecteur HSV : tout l’aval du pipeline est identique, qu’on emploie l’un ou l’autre (figure 4.6).

Dans ce travail, OWL-ViT2 (variante *base*, de l'ordre de la centaine de millions de paramètres) tourne sur le GPU intégré du MacBook M4 (*backend* MPS), mais nettement plus lentement que le seuillage HSV — de l'ordre de quelques secondes par image —, ce qui le réserve à une détection *ponctuelle* plutôt qu'en flux continu. Les descriptions sont *enrichies* (couleur, forme, matière) plutôt que minimales, et le seuil de confiance a été porté à 0,25 (dans `configs/perception/hf_specs.json`) : à 0,15, le modèle étiquetait parfois les parties oranges du robot comme le cube orange (score $\sim 0,2$), d'où des reconstructions 3D erronées ; le seuil plus strict écarte ces confusions.

Forces et limites. Ce détecteur est *ouvert*, *sémantique* et *sans calibration couleur* — mais c'est un réseau lourd ($\sim$ une centaine de millions de paramètres, quelques *secondes* par image — bien plus lent que le seuillage HSV), moins transparent qu'un seuillage et porteur des incertitudes propres à un détecteur appris (détections manquées ou parasites selon la formulation et l'éclairage). D'où la complémentarité des deux versions : HSV comme socle reproductible pour *isoler* la géométrie, OWL-ViT2 pour la *robustesse sémantique*. Leur comparaison chiffrée sur la tâche complète figure au chapitre 7.

4.5 Reconstruction 3D

Une détection ne donne qu'un *pixel*, et la projection a perdu la profondeur (le facteur $s = Z$, section 4.2). Pour situer l'objet *en 3D* dans le repère de base, il faut deux points de vue : c'est l'objet de la *reconstruction*, qui *inverse* la matrice de projection P posée plus haut.

Triangulation. À travers sa matrice P , un pixel ne désigne pas un point mais une *droite* : tous les points 3D qui se projettent sur lui. Un même objet vu par les deux caméras fixes (`cam_0`, `cam_1`) définit donc deux droites, qui se coupent — en théorie — en sa position. Le bruit de détection fait qu'elles ne se rencontrent pas tout à fait ; on cherche alors le point qui en minimise l'écart. C'est la *triangulation* par *transformation linéaire directe* (DLT) [10], fournie par `cv2.triangulatePoints` à partir de P_0 et P_1 . Comme P ne décrit que la partie linéaire de la projection, les pixels sont d'abord *dé-distordus* (section 4.2) avant la triangulation.

La mise en équations est directe. Si $\mathbf{p}_1^\top, \mathbf{p}_2^\top, \mathbf{p}_3^\top$ sont les trois lignes de P et (u, v) le pixel observé, le fait que $\mathbf{X}$ se projette en (u, v) s'écrit, après élimination du facteur d'échelle, en deux équations *linéaires* en $\mathbf{X}$:

$$u \mathbf{p}_3^\top \mathbf{X} - \mathbf{p}_1^\top \mathbf{X} = 0, \quad v \mathbf{p}_3^\top \mathbf{X} - \mathbf{p}_2^\top \mathbf{X} = 0.$$

Ces deux relations valent pour *une* caméra ; la seconde en fournit deux autres, de même forme (avec sa propre matrice P et son pixel). Les deux caméras empilent ainsi *quatre* équations pour les trois libertés de $\mathbf{X}$: le système homogène surdéterminé $A \mathbf{X} = \mathbf{0}$ se résout par décomposition en valeurs singulières, le point cherché étant le vecteur singulier de plus petite valeur. La figure 4.7 illustre cette géométrie — et pourquoi la profondeur y est la direction la moins bien contrainte.

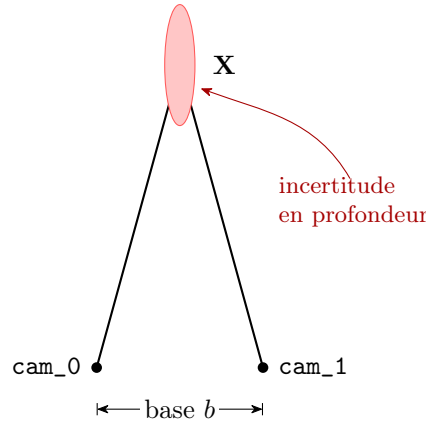


FIGURE 4.7 – Triangulation stéréo. Chaque pixel définit, via sa matrice P , une *ligne de visée* ; leur intersection donne le point $\mathbf{X}$. Comme la base b est petite devant la profondeur, les deux droites se coupent sous un angle *rasant* : la région compatible avec les deux mesures est *allongée* le long de l’axe caméra, si bien que la profondeur est estimée moins précisément que les directions latérales.

Apparier avant de trianguler. Encore faut-il savoir *quelles* détectations des deux caméras correspondent au même objet. L’appariement se fait par *label* (même classe), puis par score ; chaque paire candidate est *validée* en re-projetant le point 3D reconstruit dans les deux images et en vérifiant que l’écart aux pixels détectés reste sous un seuil (quelques dizaines de pixels). Une paire qui échoue — objet hors de l’espace de travail, ou écart trop grand — est écartée au profit de la suivante.

La profondeur, direction la moins bien estimée. La triangulation n’est pas également précise dans toutes les directions. Pour deux caméras de *base* b (l’écart entre leurs centres) et de focale f , une erreur de Δd pixels sur la position relative d’un point se répercute sur la profondeur estimée z selon

$$\Delta z \approx \frac{z^2}{b f} \Delta d,$$

qui croît avec le *carré* de la distance et décroît avec la base. Autrement dit : plus l’objet est loin, ou plus les deux caméras sont rapprochées, plus la profondeur est incertaine — c’est la direction la moins bien contrainte de la reconstruction, celle où les deux droites deviennent presque parallèles.

Dans ce travail, la base stéréo **cam_0–cam_1** vaut environ 103 mm ; aux distances de travail (30 à 50 cm), la profondeur est l’axe le plus bruité. Or la paire fixe *surplombe* la table — sa ligne de visée est inclinée d’environ 48° vers le bas —, si bien que cet axe faible se projette en bonne part sur la *verticale* : la *hauteur* de l’objet est donc l’une des grandeurs les moins précises, ce qui contribue à l’imprécision en hauteur constatée lors des prises (discutée au chapitre 7). Deux mécanismes limitent cette erreur : la *validation par reprojection* ci-dessus, et le *raffinement* de la prise par la caméra de poignet juste avant la descente (chapitre 6). La covariance de position, prévue dans le modèle de données, n’est pas propagée explicitement : la fiabilité d’un point est jugée *a posteriori* par son erreur de reprojection.

Repli monocular. Lorsqu'un objet n'est vu que par une seule caméra, la stéréo est impossible ; un repli *monoculaire* estime alors sa pose par PnP [21] — la même méthode éprouvée que celle de la calibration (section 4.2) —, en exploitant les dimensions *connues* de l'objet. La difficulté ne tient pas à PnP, mais à la géométrie *monoculaire* elle-même : la pose d'un objet quasi *plan* vu de face — une face de cube, un prisme posé — est intrinsèquement *ambiguë*, deux poses symétriques expliquant presque aussi bien l'image, quel que soit le solveur. La stéréo lève cette ambiguïté en croisant deux points de vue ; le repli monocular ne joue donc qu'un rôle secondaire sur les formes étudiées, la reconstruction reposant avant tout sur la paire stéréo.

Le résultat de l'étape est une `ObjectInstance` : la position 3D de l'objet dans le repère de base, son encombrement (boîte englobante 3D) et, le cas échéant, une orientation — les grandeurs que la planification de saisie va consommer (chapitre 5).

4.6 Synchronisation multi-caméras

La triangulation suppose que les deux caméras voient la scène *au même instant* : si l'objet — ou le bras, pour la caméra de poignet — a bougé entre les deux prises, les droites ne se croisent plus au bon endroit et la position 3D est faussée. La capture doit donc être *synchronisée*.

Capture en deux temps. Le pilote `MultiCamera` sépare la *saisie* de l'image de son *décodage* : il déclenche d'abord un `grab` sur les caméras en *rafale serrée*, puis ne décode (`retrieve`) chaque image qu'ensuite. Les `grab` s'enchaînant en quelques millisecondes, les trois prises sont *quasi simultanées* — *indépendamment* de la cadence des caméras —, un écart négligeable devant le mouvement des objets, immobiles au moment de la mesure. Une caméra qui échoue rend simplement une image *absente*, que le pipeline gère sans s'interrompre.

Écarter les images périmées. Les caméras USB *tamponnent* quelques images ; lues telles quelles juste après un déplacement du bras, elles renverraient une vue *périmée*. Avant chaque mesure, le pilote *purge* donc ce tampon, de sorte que la reconstruction travaille sur l'état courant de la scène et non sur une image capturée en plein mouvement.

Dans ce travail, les trois caméras USB — réglées à 15 images/s pour la paire stéréo et 10 pour la caméra de poignet, précisément pour tenir la bande passante et éviter les décrochages — sont réparties sur *deux* concentrateurs (*hubs*) : `cam_0/cam_1` d'un côté, `cam_2` de l'autre. L'écart de saisie obtenu entre `cam_0` et `cam_1`, de quelques millisecondes, suffit pour une scène statique ; il ne conviendrait pas à un suivi d'objet en mouvement rapide, hors du cadre de ce travail.

4.7 Validation expérimentale

Cette section mesure la *qualité géométrique* de la perception — avec quelle précision la calibration situe les caméras et fonde la reconstruction. Elle ne traite pas de la réussite des prises : l'évaluation de la tâche complète, et la comparaison chiffrée des deux détecteurs sur le pick-and-place, relèvent du chapitre 7.

TABLE 4.1 – Cohérence de la calibration œil-main : écart résiduel de la pose reconstruite sur les prises retenues (déviations moyenne et maximale), par caméra.

Caméra	Montage	Poses (ret./capt.)	Transl. moy./max (mm)	Rot. moy./max (°)
cam_0	<i>eye-to-hand</i>	23/71	6,6 / 20,8	1,4 / 2,4
cam_1	<i>eye-to-hand</i> (déduite)	71/71	11,8 / 26,8	2,9 / 7,2
cam_2	<i>eye-in-hand</i>	20/41	2,5 / 4,8	0,4 / 0,8

Calibration intrinsèque. L’erreur de reprojection finale (section 4.2) reste *sous-pixellique* pour les trois caméras — 0,19, 0,17 et 0,14 pixel pour **cam_0**, **cam_1** et **cam_2** respectivement —, chacune sur une quarantaine de vues du damier. Le modèle sténopé et sa distorsion décrivent donc fidèlement l’optique.

Calibration œil-main. La qualité de la calibration extrinsèque se lit dans la *cohérence* de la transformation constante recherchée : la pose du damier reconstruite à travers la chaîne devrait être identique d’une prise à l’autre, et l’écart résiduel mesure ce qui subsiste (table 4.1). Trois lectures s’en dégagent. La caméra de poignet (**cam_2**, *eye-in-hand*) est de loin la plus précise — 2,5 mm et 0,4° en moyenne —, car elle observe le damier de près. Les caméras fixes, plus éloignées, sont moins précises (6,6 mm pour **cam_0**). Enfin **cam_1** a bien ses *intrinsèques* propres, mais sa *pose* est déduite de **cam_0** et de la transformation stéréo (section 4.2) : son résidu cumule alors l’écart de **cam_0** et celui du recalage stéréo, d’où une valeur plus élevée (11,8 mm). Ces résidus individuels, de l’ordre du demi-centimètre, restent inférieurs à la *marge* d’environ 1 cm que le planificateur laisse de *chaque côté* de l’objet (chapitre 5) — d’autant que le raffinement final par la caméra de poignet corrige le reliquat avant la fermeture (chapitre 6).

Reste la question que ces chiffres soulèvent : pourquoi calibrer **cam_1** par *déduction* plutôt qu’*indépendamment* ? Nous l’avons d’abord fait séparément, et c’est instructif. Chaque caméra affichait alors un bon résidu individuel (de l’ordre de 6 mm), mais une *cohérence stéréo* très dégradée : les deux caméras plaçaient un même point 3D à environ 21 mm l’une de l’autre, soit un biais systématique d’environ +40 mm en *Y* à la triangulation. La cause est géométrique — deux calibrations indépendantes portent des erreurs *sans corrélation* qui, sur une base orientée surtout selon *Y*, s’*additionnent* au lieu de se compenser —, alors que pour trianguler, seul l’*accord* entre les deux caméras compte. La calibration *stéréo conjointe* corrige exactement cela : elle mesure la transformation *relative* $T_{\text{cam}_0 \leftarrow \text{cam}_1}$ directement, puis en déduit **cam_1**, de sorte qu’un biais de **cam_0** devient *commun* aux deux caméras et s’*annule* à la triangulation. La cohérence du couple tombe ainsi à environ 2 mm (biais résiduel ramené à environ +14 mm), au prix — assumé — du résidu individuel plus élevé de **cam_1**. Autrement dit, le résidu individuel n’est pas la bonne mesure ici : c’est la cohérence *relative*, seule utile à la triangulation, qui départage les deux méthodes.

Reconstruction 3D. La précision de la triangulation peut se vérifier en comparant la position reconstruite d’un objet à sa position *mesurée au pied à coulisse* depuis la base (outil `scripts/check_perception.py`). Faute d’une campagne de mesures systématique, nous nous en tenons ici à un constat *qualitatif*, conforme à l’analyse de la section 4.5 : l’écart est dominé par la *profondeur* — l’axe caméra —, les directions latérales étant plus fiables. La seule grandeur *chiffrée*

disponible est la *cohérence stéréo* du couple, de l'ordre de 2 mm (ci-dessus) ; les conséquences pratiques de l'incertitude 3D sur la réussite des prises sont examinées avec l'évaluation de la tâche (chapitre 7).

Chapitre 5

Planification de saisie

La planification de saisie fait le lien entre la perception et le contrôle : elle transforme un objet *reconstruit* — une `ObjectInstance`, avec sa position, son encombrement et, quand elle est fiable, l'orientation de son empreinte (chapitre 4) — en une *prise* exécutable — une `GraspPose`, soit trois poses successives de la pince (approche, saisie, retrait) et ses ouvertures (chapitre 3). Conformément au cahier des charges, cette planification est *géométrique et heuristique* [15] : la prise se dérive directement de la géométrie mesurée, par des règles explicites, sans apprentissage. Ce chapitre en pose le problème, puis détaille la stratégie, sa construction géométrique, son périmètre et son interface avec le contrôle.

5.1 Définition du problème et sous-actionnement

Ce qu'il faut décider. Saisir, c'est amener la pince à une configuration — position, orientation, ouverture — où elle enserme l'objet de façon stable, en l'atteignant par une *direction d'approche* dégagée. Le planificateur reçoit une `ObjectInstance` (position 3D dans le repère de base, boîte englobante de dimensions $\ell_x \times \ell_y \times H$, orientation éventuelle) et produit une `GraspPose` : la pose de *saisie* proprement dite, encadrée d'une pose d'*approche* (en retrait, pince ouverte) et d'une pose de *retrait* (levée, pince fermée), avec les ouvertures associées. Tout est exprimé en mètres dans le repère de base (chapitre 3).

La contrainte de fond : un bras sous-actionné. Le bras SO-101 compte *cinq* articulations motorisées, plus la pince. Or amener la pince à une pose *quelconque* (position et orientation libres) en réclamerait *six* : trois pour la position, trois pour l'orientation. Il en manque une, et l'on peut dire précisément laquelle (figure 5.1). L'orientation de la pince (la « tête ») repose sur deux articulations de *poignet* : le *tangage* (`wrist_flex`), qui l'incline vers le haut ou le bas, et le *roulis* (`wrist_roll`), qui la fait tourner autour de son axe d'approche. Un poignet *complet* en aurait une troisième, un *lacet*, mais celui du SO-101 n'en a pas. Le seul lacet du bras est celui de l'*épaule* (`shoulder_pan`) ; or il agit à la *base*, loin de la tête : il fait pivoter le bras *entier* pour l'orienter vers l'objet, sans pouvoir régler à part le *cap* de la pince. Le bras amène donc l'objet en *position* et donne à la tête un tangage et un roulis, mais pas de lacet propre. C'est le sens du *sous-actionnement* pour la tâche de saisie : non que des articulations soient passives (elles sont toutes motorisées), mais qu'il en manque une au regard des six degrés de liberté d'une pose.

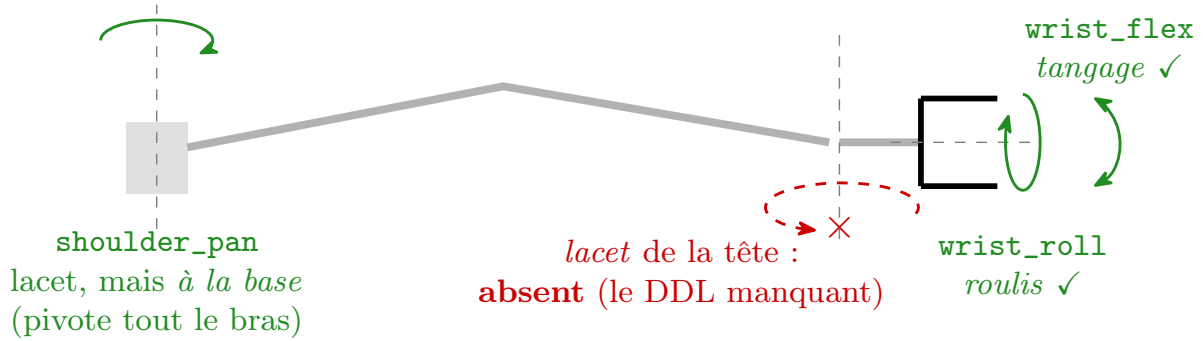


FIGURE 5.1 – Le sous-actionnement du poignet. La tête dispose du *tangage* (*wrist_flex*) et du *roulis* (*wrist_roll*), mais d’aucun *lacet* propre ; seule l’épaule fournit un *lacet*, mais à la base — d’où l’impossibilité de régler librement le cap de la pince à une position donnée.

Cette limitation détermine toute la stratégie : à une position donnée, nous ne choisissons pas une orientation *quelconque* de la pince, seulement son *inclinaison* dans le plan vertical que l’épaule oriente vers l’objet. C’est ce degré de liberté restant, l’*angle d’attaque*, que la stratégie de saisie fait varier.

5.2 Stratégie : saisie géométrique par balayage de l’angle d’attaque

Une stratégie interchangeable. La planification est isolée derrière l’interface *GraspStrategy* (patron *stratégie*, chapitre 3) : elle prend une *ObjectInstance* et rend une *GraspPose*. La stratégie employée est *géométrique* ; la prise verticale (*top-down*) en est un simple cas particulier, obtenu quand l’angle d’attaque est nul.

Ce qui s’adapte, et ce qui ne s’adapte pas. Deux choses souvent confondues méritent d’être distinguées. L’*ouverture* et l’*orientation* de la pince s’adaptent bien à la *géométrie* de l’objet : l’ouverture se règle sur sa largeur mesurée, l’orientation des mâchoires sur son axe principal (section 5.3). En revanche, l’*angle d’attaque* (l’inclinaison de l’approche) n’est *pas* choisi d’après la *forme* de l’objet, mais d’après sa *position* : sa distance au robot et la hauteur de son sommet. C’est une limite que le chapitre 8 rediscute : sur un bras sous-actionné, l’angle réalisable dépend surtout d’où se trouve l’objet, non de ce qu’il est.

Le balayage. La stratégie procède par *balayage*. Pour un objet donné, elle engendre une courte liste d’angles d’attaque *candidats* — 0° (vertical), 45° (diagonal), 90° (de face) —, *ordonnés par préférence* selon la position de l’objet. Chaque candidat géométriquement valide est ensuite soumis au solveur de cinématique *inverse* (chapitre 6), et nous retenons le *premier* que le bras peut effectivement atteindre. La position fixe ainsi la *préférence*, l’atteignabilité fixe la *faisabilité*, et le premier candidat réalisable est retenu. Ce filtrage par *atteignabilité*, qui ne retient que les prises que le bras peut réellement atteindre, rejoint l’esprit des *cartes d’atteignabilité* et de capacité de la littérature [22, 23]. Si aucun candidat n’est atteignable, la prise échoue *proprement* (aucune prise renvoyée) plutôt que de forcer un mouvement dangereux.

La règle de préférence. La préférence se lit sur deux grandeurs : la *distance* $d = \sqrt{x^2 + y^2}$ de l’objet au robot, et la *hauteur de son sommet* $h_{\text{top}} = z + H/2$. Pour un objet *bas*, nous privilégions la prise verticale tant qu’il est à portée, puis la diagonale au-delà ; un objet *haut*, hors d’atteinte par le dessus, s’attaque de face.

Dans ce travail, l’angle préféré suit la hauteur du sommet et la distance : un objet *bas* (sommet < 12 cm) est pris en *top-down* (0°) tant qu’il est à moins de 32 cm, en *diagonale* (45°) au-delà ; un objet de hauteur *intermédiaire* (12 à 18 cm) en diagonale ; un objet *haut* (≥ 18 cm) *de face* (90°). Ces seuils, notamment la zone top-down de 32 cm, sont réglables (option `--zone-topdown`). La prise *de face* d’un objet *bas* a été *retirée* : la caméra de poignet vient alors buter sur la table avant d’atteindre l’objet, et le couple moteur sature (charge moteur relevée à 1004 sur le bus Feetech). À l’autre bout, la branche *de face* pour objet *haut* (90° , sommet ≥ 18 cm, par exemple un objet posé en hauteur) a été vérifiée qualitativement et fonctionne, mais ne figure pas dans les campagnes d’évaluation (chapitre 7).

5.3 Construction géométrique de la prise

L’ouverture, réglée sur la largeur. Les mâchoires se referment dans le plan horizontal ; on prend donc la *plus petite* des deux *largeurs au sol* de l’objet — ses deux dimensions horizontales mesurées ℓ_x et ℓ_y , les côtés de son encombrement au sol —, soit $w = \min(\ell_x, \ell_y)$. On écarte alors les mâchoires de cette largeur *augmentée d’une marge m de chaque côté* :

$$\text{ouverture} = \frac{w + 2m}{O_{\text{max}}} \times 100 \%,$$

bornée à $[20, 100] \%$, où O_{max} est l’ouverture maximale de la pince. La marge, appliquée de *chaque* côté, absorbe l’erreur de visée (cinématique inverse et calibration cumulées, de l’ordre du centimètre) : sans elle, le doigt *fixe* de la pince heurterait l’objet à la descente au lieu de l’encadrer.

Dans ce travail, la marge vaut 10 mm par côté et l’ouverture maximale 150 mm (mesurée sur la pince). Un objet de 30 mm de large commande ainsi une ouverture de $30 + 2 \times 10 = 50$ mm, soit $50/150 \approx 33 \%$ de la course et 10 mm de dégagement de *chaque* côté, et non le maximum : l’ouverture *distingue* ainsi les objets au lieu de s’ouvrir au maximum pour chacun.

L’orientation, alignée sur l’axe principal. Reste à orienter les mâchoires (le *roulis* du poignet). La règle est unique, sans cas particulier par objet : nous alignons les mâchoires *en travers* du *petit* axe de l’empreinte, c’est-à-dire perpendiculairement à son *grand* axe. Ce grand axe est estimé à partir des *moments* du contour de l’objet, par une analyse en composantes principales du plan image :

$$\theta = \frac{1}{2} \text{atan2}(2\mu_{11}, \mu_{20} - \mu_{02}).$$

Quand l’empreinte est nettement *allongée*, cet axe est fiable et fixe l’orientation. Quand elle est *ronde* ou indéterminée (un cube vu de dessus, un disque), le grand axe n’a pas de sens : l’orientation est alors laissée *libre*, et la cinématique inverse choisit l’angle qui fait le *moins* tourner

le poignet depuis sa position courante. C'est sans conséquence pour un objet rond (tout angle le saisit pareil) et cela évite d'imposer une rotation inutile.

Les trois poses. De la pose de *saisie* — position de l'objet, orientation et ouverture calculées —, on déduit les deux autres par de simples décalages. La pose d'*approche* recule le long de la *direction d'attaque* (verticale pour un top-down, inclinée sinon), pince ouverte ; la pose de *retrait* lève l'objet à la verticale, pince fermée.

Dans ce travail, la **GraspPose** encadre la saisie d'une pose d'*approche* (8 cm au-dessus, le long de l'axe d'attaque) et d'un *retrait* (10 cm de levée verticale), sans décalage ajouté à la pose de saisie elle-même. À l'exécution en *boucle fermée*, toutefois, cette pose d'approche n'est qu'un *point de passage* : le bras s'arrête en réalité plus haut, à environ 12 cm au-dessus de l'objet, où la caméra de poignet observe et *raffine* la prise avant la descente directe (chapitre 6). Pour les prises *inclinées*, le roulis appliqué autour de l'axe d'approche est fixé à 90° : un roulis de 45°, essayé, s'est révélé mauvais à 30–35 cm. Ce roulis incliné reste imparfait, car la convention ignore le roulis *réel* de l'objet, d'où une légère asymétrie gauche/droite ; mais 90° est le moins pénalisant à l'usage, et la prise verticale n'est pas concernée.

5.4 Périmètre : balayage sagittal et limite de la prise latérale

Un balayage sagittal, non sphérique. Le balayage n'explore que le *plan sagittal* du bras — le plan vertical que l'épaule oriente vers l'objet —, et seulement vers l'*avant* : 0° (par le dessus), 45° (diagonale) et 90° (*de face*, l'objet attaqué par sa face avant). Ce sont trois *inclinaisons* d'une même approche frontale, non des directions distinctes. Les angles *arrière* (−45°, −90°), qui reviendraient à attaquer la *face arrière* de l'objet, pince tournée vers la base, sont *hors d'atteinte*. Géométriquement, les trois flexions du bras plient dans le même plan : se replier vers l'arrière ne serait envisageable que pour un objet situé *au niveau de la base*, jamais plus haut. Et surtout, un tel retournement ferait *percuter le support de la caméra de poignet* et l'endommagerait. Ces poses ne sont donc jamais retenues, et la cinématique inverse y laisse d'ailleurs des résidus importants. Le balayage n'explore pas non plus les côtés *gauche* et *droit*, objet du paragraphe suivant.

La prise latérale : mal conditionnée, non impossible. À ne pas confondre avec l'approche *de face* à 90° (frontale, ci-dessus), la prise *latérale* (le *side grasp*) saisirait l'objet par le *côté*, en l'abordant perpendiculairement à la ligne robot–objet. Or c'est précisément le *lacet* de la pince qui le permettrait, le degré de liberté qui manque (section 5.1) : sans lui, on ne peut pas tourner les mâchoires sur le côté sans déplacer tout le bras. Cette prise n'est pas strictement *impossible* : le solveur de cinématique inverse, qui part de la posture *courante* du bras, l'atteint depuis certaines configurations de départ. Mais son succès dépend fortement de ce point de départ, car depuis d'autres postures la recherche n'aboutit pas ; ce qui la rend *mal conditionnée* et trop peu fiable pour être retenue. Le balayage reste donc *frontal* : une limite assumée, rediscutée au chapitre 8.

Ce qui n'est pas traité. Enfin, la planification décrite ici ne gère *aucun* évitement d'obstacle : elle suppose l'objet cible dégagé. C'est un choix *délibéré* (l'objectif 3 du cahier des charges), dont

le chapitre 8 détaille le motif et les implications.

5.5 Interface avec le contrôle et validation

De la prise aux moteurs. La *GraspPose* n'est encore qu'une *consigne* géométrique ; le contrôle (chapitre 6) l'exécute. La cinématique *inverse* traduit chacune des trois poses en angles articulaires, et une trajectoire *quintique* les enchaîne de façon lisse. C'est aussi le contrôle qui *filtre* les candidats de saisie par atteignabilité : un angle d'attaque proposé n'est retenu que si la cinématique inverse converge, à une tolérance de l'ordre du centimètre en position et de quelques degrés en orientation ; le premier candidat atteignable dans l'ordre de préférence est exécuté.

Deux garde-fous. Deux sécurités encadrent l'exécution. Avant la descente finale, la caméra de *poignet* (*eye-in-hand*, *cam_2*) corrige la prise *en boucle fermée* sur la vue rapprochée de l'objet, rattrapant le reliquat d'erreur de la reconstruction stéréo avant que la pince ne se referme (chapitre 6). Et une prise dont la cinématique inverse exigerait un *demi-tour* du roulis de poignet est *refusée* : un tel retournement dirigerait la pince vers la table, dans une posture de bras repliée. La tenue de l'objet, enfin, est vérifiée à la levée par la *charge* du moteur de pince, faute de capteur d'effort dédié (chapitre 6).

Une planification transparente. Chaque prise se justifie ici par une règle explicite — telle largeur commande telle ouverture, telle position tel angle —, ce qui la rend vérifiable et discutable, à l'inverse d'une prise apprise de bout en bout. Son évaluation chiffrée, et sa comparaison avec la politique apprise, font l'objet du chapitre 7.

Chapitre 6

Contrôle

La planification a produit une *intention* : une **GraspPose**, trois poses cartésiennes de la pince exprimées dans le repère de base (chapitre 5). Le contrôle est l'étage qui la *réalise* : il traduit ces poses en angles articulaires, puis en mouvements de moteurs, jusqu'à refermer la pince sur l'objet. C'est le maillon qui relie le monde géométrique de la perception au monde physique du bras.

Ce chapitre suit le fil conducteur de tout le travail : les transformations rigides T et les changements de repère (chapitre 3), dont dépendait déjà la perception (chapitre 4). Ici, ces mêmes matrices décrivent la géométrie du bras lui-même. Nous commençons par la *cinématique directe* : où se trouve la pince quand on connaît les angles (section 6.1). Puis vient son inverse, plus délicate : quels angles atteignent une pose voulue (section 6.2). Vient ensuite la *génération de trajectoire*, qui enchaîne ces configurations sans à-coups (section 6.3), puis le *contrôle moteur* qui les envoie au bras et lit son état (section 6.4). La dernière section rassemble le tout dans la boucle fermée du *pick-and-place* (section 6.5).

Un contrôle en position, non en effort. Un choix cadre tout le chapitre : nous pilotons le bras *en position*. À chaque instant, nous commandons aux servomoteurs un *angle* à atteindre, et leur électronique interne s'en charge. Nous ne modélisons ni les forces, ni les couples, ni la dynamique (masses et inerties) du bras. Ce choix se justifie par le matériel : les servomoteurs du SO-101 ont un fort rapport de réduction et un asservissement en position intégré ; aux vitesses lentes que nous employons, ils tiennent leur consigne et supportent le poids du bras sans que nous ayons à calculer le moindre couple. La cinématique, c'est-à-dire la géométrie du mouvement, suffit donc ; la *dynamique* ne deviendrait nécessaire que pour des mouvements rapides ou une commande en effort, hors de notre cadre. La seule grandeur d'effort que nous lisons est la *charge* du moteur de pince, et uniquement comme un *signal* de contact (section 6.4), jamais comme une consigne.

6.1 Cinématique directe

La *cinématique directe* (*forward kinematics*) répond à une question simple : le bras étant dans une configuration articulaire donnée, les cinq angles connus, où se trouve la pince et comment est-elle orientée dans le repère de base ? Elle calcule la pose $T_{\text{base} \leftarrow \text{pince}}$ à partir du vecteur d'angles $q = (q_1, \dots, q_5)$. Ici « pince » désigne le *point de préhension* entre les mâchoires (et non le bout

des doigts), fixé rigidement à la main à une dizaine de centimètres au-delà du poignet, que la planification vient poser sur le centre de l’objet à saisir (chapitre 5). C’est la brique la plus fondamentale du contrôle : la cinématique *inverse* l’appellera des centaines de fois (section 6.2), et la boucle fermée s’en servira pour savoir où pointe la caméra de poignet (section 6.5).

Le bras comme une chaîne de repères. Le SO-101 est une succession de segments rigides reliés par des articulations rotoïdes. À chaque articulation est attaché un repère ; passer de la base à la pince revient à *composer*, de proche en proche, les transformations rigides qui relient chaque repère au suivant — exactement les matrices homogènes $T_{A \leftarrow B}$ du chapitre 3. Une transformation par articulation, mises bout à bout : leur produit donne la pose de la pince. Chaque articulation apporte deux choses : un *montage* fixe (la position et l’orientation de son repère par rapport au précédent, indépendantes de l’angle) et une *rotation* d’un angle q_i autour de son axe propre.

La source du modèle : l’URDF. Cette géométrie — où siège chaque articulation, autour de quel axe elle tourne — n’est pas codée en dur. Elle est lue depuis un fichier *URDF* (*Unified Robot Description Format*), `so101_new_calib.urdf`, généré par TheRobotStudio à partir de la CAO Onshape du bras SO-ARM100. L’URDF est la *seule* source de vérité pour la géométrie : corriger ou changer le modèle, c’est remplacer ce fichier, jamais le code. Chaque articulation y déclare son montage fixe (balise `<origin xyz rpy>` : une translation `xyz` et trois angles `rpy`) et son axe de rotation (`<axis>`).

La transformation d’une articulation. Traduisons cela en matrices. Le montage fixe se lit directement dans l’URDF : la translation $\mathbf{t}$ donnée par `xyz`, et l’orientation par les trois angles *roulis-tangage-lacet* (r, p, y) de `rpy`, selon la convention URDF — rotations successives autour des axes *fixes* X, Y, Z :

$$R_{\text{rpy}} = R_z(y) R_y(p) R_x(r), \quad T_{\text{montage}} = \begin{bmatrix} R_{\text{rpy}} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}.$$

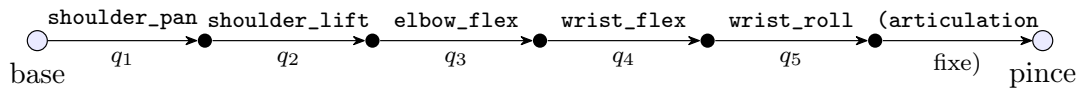
À ce montage s’ajoute la rotation de l’articulation elle-même : une rotation d’angle q_i autour de son axe $\mathbf{k}$ (un vecteur unitaire, exprimé dans le repère de l’articulation). La *formule de Rodrigues* en donne la matrice :

$$R(\mathbf{k}, q_i) = I + \sin q_i [\mathbf{k}]_\times + (1 - \cos q_i) [\mathbf{k}]_\times^2,$$

où $[\mathbf{k}]_\times$ est la matrice antisymétrique du produit vectoriel par $\mathbf{k}$ — celle qui vérifie $[\mathbf{k}]_\times \mathbf{v} = \mathbf{k} \times \mathbf{v}$ pour tout vecteur $\mathbf{v}$. C’est exactement ce que calcule la routine `cv2.Rodrigues` employée dans le code. La transformation complète d’une articulation motorisée est donc son montage suivi de cette rotation, $T_{\text{montage}} R(\mathbf{k}, q_i)$; une articulation *fixe*, sans moteur, se réduit à son seul montage.

La composition : de la base à l’effecteur. La cinématique directe d’un bras série est le produit *ordonné* de ces transformations, de la base vers la pince [17] :

$$T_{\text{base} \leftarrow \text{pince}}(q) = \prod_{i=1}^n T_{\text{montage}, i} R(\mathbf{k}_i, q_i),$$



chaque flèche i : montage fixe $T_{\text{montage},i}$ (lu dans l'URDF) puis rotation $R(\mathbf{k}_i, q_i)$ autour de l'axe $\mathbf{k}_i$
 $\implies T_{\text{base} \leftarrow \text{pince}}(q) = \prod_i T_{\text{montage},i} R(\mathbf{k}_i, q_i)$

FIGURE 6.1 – La chaîne cinématique du bras, vue comme une composition de transformations rigides. Chaque articulation ajoute un montage fixe (lu dans l'URDF) et une rotation autour de son axe ; leur produit, de la base à la pince, donne la cinématique directe.

le produit courant le long de la chaîne, chaque articulation motorisée contribuant son montage *et* sa rotation, chaque articulation fixe son seul montage. Concrètement, on part du repère de base (la matrice identité), on multiplie par la transformation de la première articulation, puis de la deuxième, et ainsi de suite jusqu'à la pince (figure 6.1). Les cinq angles déterminent entièrement et sans ambiguïté la pose résultante : la cinématique directe admet toujours une solution unique, et son calcul est immédiat. C'est son *inverse* qui posera problème (section 6.2).

Dans ce travail, la chaîne compte cinq articulations rotoïdes — `shoulder_pan`, `shoulder_lift`, `elbow_flex`, `wrist_flex`, `wrist_roll` — suivies d'une articulation *fixe* qui porte le repère de la pince. Le moteur de la *mâchoire* n'entre pas dans la chaîne : il ouvre et referme les doigts sans déplacer le repère de la pince, c'est une branche latérale. La configuration *zéro* correspond, par convention de calibration, au milieu de course de chaque articulation (chapitre 4) ; à cette configuration, la cinématique directe place l'effecteur à environ (391, 0, 226) mm du repère de base — bras tendu vers l'avant, à quelque 45 cm de la base. Ce calcul se *vérifie* au passage, par un *test automatique du code*. Comme la cinématique directe enchaîne un long produit de matrices, la moindre erreur de composition (un axe ou un `rpy` mal repris) ou une dérive numérique se trahirait par une « rotation » qui n'en serait plus une. Nous testons donc, sur vingt configurations tirées *au hasard* (et non sur le seul cas favorable de la configuration zéro), que le bloc rotation de la pose reste *orthonormal* et de *déterminant* +1 à la précision numérique — les deux conditions d'une vraie rotation. Le vérifier pour des configurations quelconques est un indice solide que la chaîne est correctement montée.

6.2 Cinématique inverse

La cinématique directe se parcourt dans un sens ; la saisie demande l'autre. Une `GraspPose` fixe où la pince doit se trouver — une pose $T_{\text{base} \leftarrow \text{pince}}$ voulue — et il faut en déduire les cinq angles qui l'y amènent. C'est la *cinématique inverse* (*inverse kinematics*) : passer de la pose aux angles. Autant la directe était immédiate, autant son inverse est délicate, pour deux raisons.

Pourquoi c'est difficile. D'abord, il n'existe pas, pour cette géométrie, de formule explicite donnant les angles à partir de la pose : les rotations s'y composent de façon trop enchevêtrée pour s'inverser à la main. Ensuite — et surtout —, le bras a *cinq* degrés de liberté pour une pose qui en compte *six* (chapitre 5) : la plupart des poses ne sont donc *pas* exactement atteignables. Nous ne chercherons pas la solution exacte — elle n'existe en général pas —, mais la configuration qui

approche au mieux la pose voulue. Cela transforme l'inversion en un problème d'*optimisation* : minimiser l'écart entre la pose atteinte et la pose visée.

Mesurer l'écart : le résidu. Pour minimiser un écart, il faut d'abord le chiffrer. À une configuration q , la cinématique directe donne une pose de position $\mathbf{t}(q)$ et de rotation $R(q)$; on la compare à la cible $(\mathbf{t}_{\text{cible}}, R_{\text{cible}})$ par un *résidu* à six composantes :

$$\mathbf{r}(q) = \begin{bmatrix} \mathbf{t}(q) - \mathbf{t}_{\text{cible}} \\ \alpha \operatorname{vec}(R(q)^\top R_{\text{cible}}) \end{bmatrix}.$$

Ce résidu *empile* deux vecteurs de trois nombres chacun — six en tout. Le bloc du haut, $\mathbf{t}(q) - \mathbf{t}_{\text{cible}}$, est l'erreur de *position* : trois nombres, en mètres. Le bloc du bas est l'erreur d'*orientation* : le produit $R(q)^\top R_{\text{cible}}$ est la rotation qui *reste* à faire pour aligner la pince sur la cible, et $\operatorname{vec}(\cdot)$ l'exprime en *vecteur de rotation* (axe $\times$ angle, par la formule de Rodrigues de la section 6.1), dont la norme vaut l'angle de désalignement — trois nombres de plus, cette fois en radians. Position et orientation cohabitent ainsi dans un *seul* vecteur, ce qui pose un problème d'unités : on ne peut pas additionner des mètres et des radians dans une norme. C'est le rôle du facteur α , exprimé en *mètres par radian* : il *convertit* l'erreur angulaire en son équivalent en mètres, pour que les six composantes se comptent dans la même unité. Nous prenons $\alpha = 0,1 \text{ m/rad}$ — un radian de désalignement « pèse » alors autant que 0,1 m (10 cm) d'erreur de position —, échelle cohérente avec la taille du bras (quelques décimètres). Résoudre l'IK, c'est alors trouver le q qui minimise $\|\mathbf{r}(q)\|^2$.

Descendre vers la solution : Gauss-Newton et Levenberg-Marquardt. On procède par *itérations*, en partant d'une configuration et en l'améliorant pas à pas. Autour de la configuration courante q , on *linéarise* le résidu : $\mathbf{r}(q + \Delta q) \approx \mathbf{r}(q) + J \Delta q$, où $J = \partial \mathbf{r} / \partial q$ est la matrice *jacobienne* (6×5) des dérivées du résidu par rapport aux angles. Chercher le pas Δq qui annulerait au mieux ce résidu linéarisé conduit aux *équations normales* de Gauss-Newton, $J^\top J \Delta q = -J^\top \mathbf{r}$. Mais près des configurations *singulières* — fréquentes sur un bras à cinq axes —, la matrice $J^\top J$ est mal conditionnée et le pas calculé explose. On la *stabilise* par l'astuce de *Levenberg-Marquardt* [24], qui ajoute un terme d'amortissement λ :

$$(J^\top J + \lambda I) \Delta q = -J^\top \mathbf{r}.$$

Le paramètre λ interpole entre deux comportements : *petit*, il redonne Gauss-Newton (grands pas, convergence rapide près de la solution) ; *grand*, il donne un petit pas prudent dans la direction de plus forte descente. Il *s'ajuste* tout seul : après chaque pas, si le résidu a diminué, on accepte le pas et l'on *réduit* λ (on fait davantage confiance au modèle linéaire) ; s'il a augmenté, on *augmente* λ et l'on retente plus prudemment. Deux garde-fous complètent la descente : le pas est *plafonné* (au plus 0,5 rad, pour éviter un bond incontrôlé) et chaque nouvelle configuration est *ramenée dans les plages articulaires* — le solveur ne propose jamais un angle que le servomoteur ne pourrait tenir. On s'arrête quand le résidu passe sous un seuil serré ($\|\mathbf{r}\| < 10^{-3}$) ou au bout de cent itérations.

La jacobienne, par différences finies. Reste à obtenir J . Plutôt que de la dériver analytiquement pour cette géométrie particulière, nous l’estimons *numériquement* : la cinématique directe étant immédiate, il suffit de la « sonder ». Pour chaque articulation k , on décale son angle d’un infime ε dans les deux sens et l’on mesure comment le résidu varie — une *différence finie centrée* :

$$J_{:,k} = \frac{\mathbf{r}(q + \varepsilon e_k) - \mathbf{r}(q - \varepsilon e_k)}{2\varepsilon},$$

où e_k ne déplace que l’articulation k et $\varepsilon = 10^{-4}$ rad. La variante *centrée* — deux évaluations, en $+\varepsilon$ et $-\varepsilon$ — est plus précise que la version à un seul côté (son erreur décroît comme ε^2 au lieu de ε), pour le coût d’une évaluation de plus par articulation. Chaque jacobienne demande ainsi dix appels à la cinématique directe (deux par axe), négligeables vu leur rapidité.

Éviter les mauvais minima : des redémarrages déterministes. La fonction à minimiser n’est pas convexe : partie d’un mauvais endroit, la descente peut se figer dans un minimum *local* sans atteindre la pose. Nous lançons donc la recherche depuis *plusieurs* points de départ : la configuration courante fournie (ou zéro à défaut) ; une amorce heuristique — orienter l’épaule vers la cible, replier grossièrement le bras selon la distance — ; et six configurations *aléatoires* tirées dans les plages articulaires. Point important : le générateur aléatoire est *réinitialisé à l’identique* à chaque appel, si bien qu’une même pose engendre toujours les mêmes redémarrages — donc le *même* mouvement d’une exécution à l’autre. Cette reproductibilité (« même situation, même comportement ») est délibérée : elle évite les mouvements surprises et rejoint les modes d’exécution du chapitre 3.

Choisir parmi les solutions : la continuité. Ces recherches produisent plusieurs solutions convergées. Nous ne retenons ni la première trouvée, ni celle de plus petit résidu, mais celle *la plus proche* de la configuration de départ (au sens de $\|q - q_{\text{init}}\|$). La raison est physique : deux configurations très différentes peuvent atteindre presque la même pose — coude en haut ou en bas, poignet tourné d’un demi-tour —, et choisir celle qui bouge le *moins* donne un mouvement court et lisse, sans reconfiguration brutale entre deux poses successives. Et si *aucune* recherche ne converge, le solveur renvoie tout de même sa meilleure approximation : l’appelant reçoit la pose atteignable la plus proche, plutôt qu’une erreur. C’est exactement le *filtre d’atteignabilité* du chapitre 5 : la planification interroge l’IK pour chaque angle d’attaque candidat et retient le premier réellement atteignable.

La symétrie à demi-tour de la pince. Un dernier raffinement, propre à la saisie. Une pince à mâchoires parallèles referme sur la même ligne, que sa tête soit tournée d’un côté ou de l’autre : la prise à l’orientation R et celle à R *retournée* de 180° autour de l’axe d’approche sont *identiques* pour l’objet. Le solveur essaie donc les *deux* et garde la plus *naturelle* — celle dont le roulis de poignet reste le plus proche de la posture courante —, ce qui empêche la tête de se retrouver à l’envers, plongeant vers la table. C’est la réalisation concrète du garde-fou annoncé au chapitre 5 : une prise qui exigerait un demi-tour du poignet est écartée. Une fenêtre de course *asymétrique* sur le roulis (issue de la calibration) verrouille d’ailleurs structurellement la variante retournée.

Dans ce travail, le solveur emploie une pondération $\alpha = 0,1$, un amortissement initial $\lambda = 10^{-3}$, un pas de différences finies $\varepsilon = 10^{-4}$ rad, six redémarrages aléatoires, et s'arrête à $\|\mathbf{r}\| < 10^{-3}$ ou cent itérations. Quand la pose est réellement atteignable, il est précis : sur un aller-retour cinématique directe $\rightarrow$ inverse — un *test automatique du solveur*, à graine fixe donc reproductible, où l'on part d'angles connus, on en calcule la pose, puis on ré-inverse celle-ci —, l'erreur maximale reste de l'ordre de 0,5 mm en position et $0,4^\circ$ en orientation. Le sous-actionnement, lui, se lit sur les cas limites du même test : une prise verticale à 20 cm devant le robot et 10 cm de haut laisse déjà un résidu de l'ordre du centimètre — le bras ne peut satisfaire *exactement* position et orientation — ; et pour un objet de 18 cm de haut à 20 cm, l'attaque à 45° converge (résidu < 1 mm) là où celle *de face* à 90° échoue (résidu ~ 3 cm), précisément le couplage angle $\times$ distance qu'exploite la planification (chapitre 5). Enfin, parce que ce seuil de convergence strict (10^{-3} , combinant millimètres et radians pondérés) est très exigeant, le pipeline juge en pratique l'atteignabilité sur une tolérance *physique* plus large — une erreur de position sous 8 mm à la pose de saisie, 15 mm à la pose d'approche — : une pose « non convergée » au sens strict reste souvent parfaitement exploitable.

6.3 Génération de trajectoire

La cinématique inverse fournit des *configurations* cibles — les étapes de la prise : configuration courante, approche, saisie, retrait, dépose. Mais connaître les étapes ne dit pas encore *comment* passer de l'une à l'autre : à quelle vitesse, selon quel profil. Envoyer d'un coup la configuration cible, ou s'y rendre à vitesse constante, secouerait le bras — et brouillerait l'image de la caméra de poignet à l'instant précis où l'on en a besoin. La *génération de trajectoire* calcule un chemin *lisse* entre deux configurations.

Relier deux configurations sans à-coups. Entre une configuration de départ q_0 et une cible q_f , il faut définir $q(t)$ pour $t \in [0, T]$. Trois exigences : partir de q_0 et finir en q_f (les extrémités) ; une *vitesse nulle* aux deux bouts (le bras est au repos avant et après) ; et, idéalement, une *accélération nulle* aux deux bouts également — car une accélération qui saute brutalement, c'est une secousse (un *à-coup*). Une simple interpolation *linéaire*, $q(t) = (1 - \frac{t}{T}) q_0 + \frac{t}{T} q_f$, respecte les extrémités mais file à vitesse *constante* : la vitesse bondit de 0 à sa valeur au départ, et retombe d'un coup à l'arrivée — accélération infinie, deux à-coups. Acceptable pour un mouvement très lent, mais non lisse.

Le polynôme quintique. On confie plutôt le profil temporel à un polynôme de degré cinq [17]. Pour chaque articulation,

$$q(t) = q_0 + s(\tau) (q_f - q_0), \quad \tau = \frac{t}{T} \in [0, 1],$$

où $s(\tau)$ est une fonction de *cadence* qui va de 0 à 1. On lui impose *six* conditions — position, vitesse et accélération, à chaque extrémité :

$$s(0) = 0, \quad s(1) = 1, \quad s'(0) = s'(1) = 0, \quad s''(0) = s''(1) = 0.$$

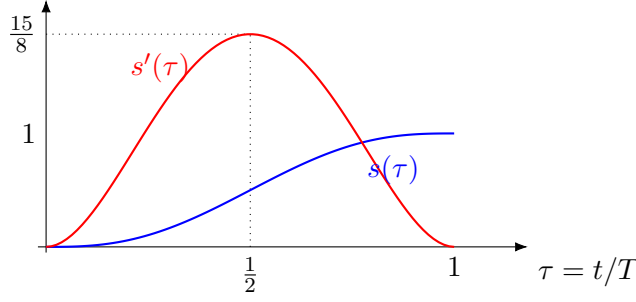


FIGURE 6.2 – Le profil quintique $s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5$ (en bleu) et sa dérivée $s'(\tau)$ (en rouge, la vitesse normalisée). La position monte de 0 à 1 ; la vitesse part de zéro, culmine à $\frac{15}{8}$ de la moyenne au milieu du mouvement, puis revient à zéro — d'où l'absence d'à-coups au départ et à l'arrêt.

Six conditions déterminent exactement les six coefficients d'un polynôme de degré cinq. Leur résolution donne

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5.$$

On le vérifie d'un coup d'œil : $s(0) = 0$ et $s(1) = 10 - 15 + 6 = 1$; la dérivée $s'(\tau) = 30\tau^2 - 60\tau^3 + 30\tau^4$ s'annule en 0 et en 1 (où $30 - 60 + 30 = 0$) ; et l'accélération $s''(\tau) = 60\tau - 180\tau^2 + 120\tau^3$ s'annule elle aussi aux deux bouts. Vitesse et accélération montent donc *en douceur* depuis zéro, puis y reviennent : aucun à-coup au démarrage ni à l'arrêt (figure 6.2).

Une vitesse en cloche. La vitesse articulaire $\dot{q} = \frac{1}{T} s'(\tau) (q_f - q_0)$ suit la courbe en cloche de s' : nulle aux extrémités, maximale au milieu. Ce pic vaut $s'(\frac{1}{2}) = \frac{15}{8}$, soit environ 1,9 fois la vitesse *moyenne* $(q_f - q_0)/T$. Autrement dit, pour rester sous une vitesse maximale donnée, il faut accorder au mouvement une durée un peu plus grande que le simple rapport déplacement / vitesse.

Dimensionner la durée. Reste à choisir T . On le dimensionne à partir du plus grand déplacement : $T = 1,5 \times \Delta_{\max}/v_{\max}$, où $\Delta_{\max}$ est le plus grand déplacement angulaire parmi les articulations et $v_{\max}$ une vitesse maximale *nominale*. Comme le profil quintique culmine à $15/8$ de sa moyenne, cette marge de 1,5 ne fait que *réduire* le dépassement sans l'annuler : la vitesse de pointe atteint en réalité environ $1,25 v_{\max}$. Cela reste prudent vu la faible valeur de $v_{\max}$ retenue. Toutes les articulations partagent le *même* T : elles démarrent et s'arrêtent ensemble (mouvement *synchronisé*), ce qui donne un chemin propre dans l'espace articulaire.

Échantillonner et enchaîner. La courbe continue $q(t)$ est enfin *échantillonnée* à cadence fixe en une suite d'instants et de configurations — une **JointTrajectory** — que le contrôleur rejouera en respectant les horodatages (section 6.4). Une prise complète *enchaîne* plusieurs segments quintiques bout à bout (courant → approche → saisie → retrait → dépose), chacun à vitesse nulle à ses jonctions : le bras marque donc un bref arrêt à chaque étape. Cela tombe bien, puisque c'est lors d'un tel arrêt, *au-dessus* de l'objet, qu'il l'observe pour raffiner la prise (section 6.5). L'ouverture de la pince est interpolée le long du même $s(\tau)$, de sorte qu'elle se referme au rythme du bras.

Dans ce travail, les trajectoires sont échantillonnées à 30 Hz et la vitesse maximale nominale est fixée à 0,5 rad/s (environ 30°/s), prudente pour le SO-101. Une interpolation *linéaire* est aussi disponible, mais c’est le profil *quintique* qui est employé, pour sa douceur ; sa vitesse de pointe, calculée sur le générateur de trajectoire, vaut bien 15/8 de la moyenne.

6.4 Contrôle moteur

La trajectoire est une suite de configurations articulaires horodatées ; reste à les faire exécuter par le bras. Cette section est le pont entre l’*angle* et le *servomoteur* : comment un angle devient une commande d’encodeur, comment la trajectoire est jouée, et comment se lit le seul retour d’effort que nous utilisons — la charge de la pince.

Du radian à l’encodeur. Le bras est mû par six servomoteurs *Feetech STS3215* (cinq pour le bras, un pour la pince), pilotés par le bus série de LeRobot. Chacun porte un encodeur magnétique absolu de 12 bits, soit environ 4096 positions par tour : on lui commande une position-cible en *pas* d’encodeur (registre `Goal_Position`). La conversion d’un angle en pas est l’inverse exact de la calibration qui sert déjà à la perception et à la cinématique directe (chapitre 4) :

$$\text{pas} = \text{centre} + \frac{\theta_{\text{deg}}}{360} N_{\text{tour}}, \quad N_{\text{tour}} \approx 4096,$$

où *centre* est le pas correspondant à l’angle nul — le milieu de course calibré. Un pas vaut donc environ $360/4096 \approx 0,088^\circ$. Certaines articulations tournent en sens inverse (un indicateur de calibration inverse alors le signe), et la valeur obtenue est *bornée* aux plages articulaires, avec une petite tolérance pour qu’un simple arrondi ne déclenche pas une erreur de butée.

Envoyer et suivre la trajectoire. Une fois le *couple activé* — les servomoteurs tiennent alors leur position et obéissent aux consignes —, commander une pose consiste à écrire les positions-cibles des six moteurs *en une seule fois* (écriture synchrone), pour qu’ils démarrent ensemble plutôt que l’un après l’autre. Suivre une trajectoire, c’est répéter cette commande pour chaque échantillon en respectant les horodatages : on envoie la configuration, puis on attend l’instant du point suivant, à la cadence de 30 Hz de la section précédente. Deux précautions encadrent le tout : le couple est *coupé* à la déconnexion et sur interruption (le bras peut alors se manipuler à la main et n’est jamais laissé sous tension inutilement), et la connexion au bus est retentée en cas de paquet corrompu au démarrage.

Le seul retour d’effort : la charge de la pince. Le SO-101 n’a ni capteur de force ni capteur tactile. Mais le servomoteur de pince expose un registre `Present_Load` — son estimation interne de couple, une magnitude sur 10 bits (0 à 1023) accompagnée d’un bit de signe, dont nous ne gardons que la magnitude. C’est une valeur *brute*, sans unité physique — un simple entier sur l’échelle 0–1023. À vide, mâchoires refermées sur rien, cette charge reste basse — de l’ordre de 20 à 30 ; dès que les mâchoires pressent un objet, elle monte. C’est notre unique signal de contact, et il a un avantage sur une vérification par vision : il n’est pas sujet à l’occlusion.

Fermer au contact, sans écraser. Comment la pince sait-elle quand cesser de se refermer ? Pas par une consigne fixe — qui écraserait un objet mou ou raterait un objet fin. Nous refermons *par petits pas* en surveillant la charge : il y a *contact* dès qu'elle dépasse un seuil *adaptatif*, égal à une *référence* plus une marge. La référence est mesurée sur les premières lectures, mâchoires encore libres (le couple à vide) ; la marge vaut 130 unités de charge (sur cette même échelle brute). Le seuil s'établit ainsi vers 150 à 180, ce qui sépare nettement « mâchoires libres » de « mâchoires sur l'objet » — bien mieux qu'une valeur absolue figée. Deux raisons à ce choix. D'une part, un critère fondé sur l'*immobilité* des mâchoires déclenchait de faux contacts au démarrage (le servomoteur ne suit pas sa consigne instantanément). D'autre part, les mâchoires en *TPU compliant* épousent l'objet et le tiennent à un couple *faible*, très en dessous du pic transitoire du contact ; une valeur absolue unique déclarait alors ratées des prises pourtant correctes. Au contact, la consigne est *gelée* puis resserrée d'un cran : la pression est maintenue, tout en gardant un écart consigne $\leftrightarrow$ objet qui permettra de détecter une chute à la levée (section 6.5).

Dans ce travail, les six moteurs sont des Feetech STS3215 (identifiants 1 à 6, la pince étant le sixième), et la charge à vide relevée sur le bus est de l'ordre de 20 à 30. La fermeture asservie referme par pas de 2 % d'ouverture, détecte le contact à *référence* + 130, puis resserre de 4 % pour maintenir la prise. Ces valeurs, calées expérimentalement, sont autant de paramètres réglables.

6.5 Pipeline intégré et boucle fermée

Les sections précédentes ont livré les ingrédients ; il reste à les assembler en une prise complète. Le point clé est que l'exécution n'est pas *en boucle ouverte* — planifier une fois, puis exécuter aveuglément. Entre la planification et la descente finale, la caméra de poignet regarde l'objet de près et *corrige* ; et la prise est *vérifiée* par une levée. Deux boucles fermées se referment ainsi : l'une sur la *position* (avant de saisir), l'autre sur la *tenue* (après avoir saisi). Ce sont les deux garde-fous annoncés au chapitre 5, réalisés ici (figure 6.3).

Phase 1 : monter observer. Depuis sa configuration courante, le bras monte à une pose d'*observation*, à 12 cm *au-dessus* de la position estimée par la stéréo (chapitre 4) — une hauteur choisie pour que l'objet soit dans le champ de *cam_2* et dégagé des mâchoires, qui occupent le bas de l'image. On y monte par une trajectoire quintique (section 6.3), et seulement si la cinématique inverse atteint cette pose dans la tolérance pratique (section 6.2).

Le raffinement *cam_2* : un asservissement visuel. À la pose d'observation, la caméra de poignet voit l'objet *de près* — bien plus précisément que la stéréo oblique. On l'y détecte (le détecteur de la perception, chapitre 4), puis on convertit le pixel détecté en une position 3D par *intersection rayon-plan* : le pixel (corrigé de la distorsion) définit un rayon dans le repère de base — via la matrice intrinsèque et la pose $T_{\text{base} \leftarrow \text{cam}_2}$, elle-même donnée par la cinématique *directe* (section 6.1) — que l'on intersecte avec le plan horizontal à la hauteur de l'objet. On compare cette position à celle que la prise planifiée visait, et l'on en tire une correction (x, y) que l'on applique à la *GraspPose* (la hauteur z , elle, reste celle de la stéréo). C'est une correction

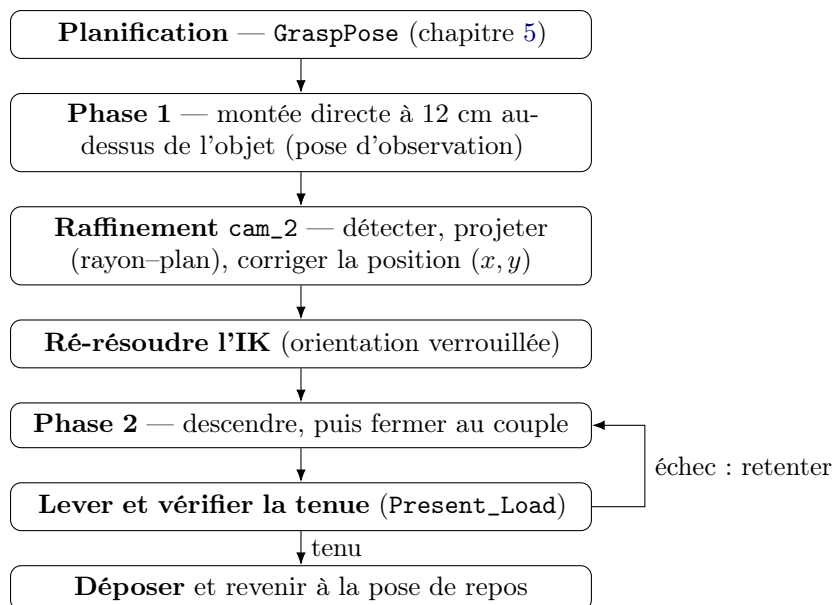


FIGURE 6.3 – La boucle fermée du *pick-and-place*. La position est corrigée par la caméra de poignet avant la saisie (boucle de *position*), et la prise est confirmée par une levée avant la dépose (boucle de *tenue*); un échec de tenue relance une tentative.

ponctuelle de type *look-and-move* — on regarde une fois, on reconstruit la position 3D, on corrige la pose *cartésienne*, puis on descend —, apparentée à l’asservissement visuel [25, 4] sans en être la variante *référéncée image*, qui asservirait en boucle continue sur l’erreur en pixels. Trois garde-fous : la correction n’est appliquée que si la détection est fiable (tache assez grande dans l’image), sous un plafond de sécurité, et si la boîte n’est pas tronquée par un bord disqualifiant. Elle est en outre d’autant plus sûre que **cam_2** regarde l’objet *de haut*. La projection rayon-plan intersecte en effet le rayon avec un plan *horizontal*. Pour une vue plongeante (prise verticale), l’intersection est bien conditionnée. À mesure que l’approche s’incline, le rayon rencontre le plan sous un angle de plus en plus rasant, et une petite erreur de pixel s’amplifie en une grande erreur de position ; à la limite frontale (90°), le rayon *rase* le plan. Une vue oblique, comme la diagonale à 45° , voit de surcroît à la fois le dessus et un côté de l’objet, ce qui biaise le centre détecté. La correction est donc la plus fiable pour la prise *verticale* — de loin la plus fréquente —, un plafond d’inclinaison réglable permettant, au besoin, de la désactiver au-delà d’un angle choisi. Au passage, **cam_2** mesure aussi le grand axe de l’objet, ce qui permet de réaligner les mâchoires.

Ré-résoudre l’IK. La **GraspPose** ayant bougé, on relance la cinématique inverse pour ses trois poses (approche, saisie, retrait), cette fois *orientation verrouillée* : le choix entre les deux symétries de la pince (section 6.2) est déjà fait, on ne le rejoue pas — c’est ce qui évite un demi-tour du poignet d’un calcul à l’autre. La continuité avec la configuration courante garde le mouvement lisse.

Phase 2 : saisir, vérifier, recommencer. La saisie proprement dite est une *boucle de tentatives* (deux au plus). Chaque tentative descend vers la pose de saisie, pince ouverte ; referme la pince *au couple* (section 6.4) ; et vérifie le contact. Puis vient la *vérification par levée*, notre boucle de tenue : le bras remonte à la pose de retrait en tenant la pince, et l’on relit charge *et* position.

L'objet est jugé *tenu* si la charge reste nettement au-dessus du couple à vide, *ou* si les mâchoires demeurent bloquées au-dessus de la consigne de serrage. Pourquoi vérifier *après* la levée ? Parce que le couple à la fermeture peut être trompeur : un effleurement du sommet, un contact sur un bord ou un appui sur la table donnent de faux positifs. Après une dizaine de centimètres de levée, en revanche, un objet perdu voit sa charge retomber à la valeur à vide, et le faux positif est démasqué. Si l'objet est tenu, le bras le *dépose* et revient à sa pose de repos ; sinon, une nouvelle tentative est lancée (re-raffinement puis redescente).

Les deux boucles, réunies. La boucle de *position* rattrape le reliquat d'erreur de la perception avant que la pince ne se referme ; la boucle de *tenue* rattrape les fermetures ratées après coup. Ensemble, elles rendent le pipeline géométrique robuste malgré une perception imparfaite et un bras à cinq degrés de liberté — sans jamais recourir à l'apprentissage. Leur efficacité chiffrée, et la comparaison avec la politique apprise de bout en bout, font l'objet du chapitre 7.

Dans ce travail, la pose d'observation est à 12 cm au-dessus de l'objet (paramètre réglable), et c'est le *seul* arrêt de raffinement : la descente est ensuite directe. La correction `cam_2` ne porte que sur (x, y) et reste plafonnée par sécurité ; elle est la plus fiable en prise verticale, un plafond d'inclinaison réglable pouvant la restreindre aux prises peu inclinées (par défaut, aucune restriction). À la levée, l'objet est déclaré tenu si sa charge dépasse d'au moins 90 unités la valeur à vide (de l'ordre de 24 sur l'échelle 0–1023), ou si l'ouverture reste supérieure d'au moins 3 % à la consigne de serrage. Une seule reprise est autorisée : en cas d'échec, le bras retente une fois puis s'arrête (deux tentatives au plus).

Chapitre 7

Évaluation expérimentale

Les chapitres précédents ont décrit deux manières de résoudre la même tâche : le *pipeline modulaire* — perception, planification, contrôle (chapitres 4 à 6) — et une *politique apprise* de bout en bout par imitation (ACT, chapitre 2). Ce chapitre les met à l'épreuve sur le *même* banc : d'abord le protocole (section 7.1), puis les résultats du pipeline (section 7.2), ceux de la politique apprise (section 7.3), et enfin leur comparaison (section 7.4).

7.1 Protocole expérimental

La tâche et le critère de succès. La tâche est unique : saisir un objet cible posé sur la table, puis le déposer dans une boîte. Un *essai* est une tentative complète. Le *succès* exige que l'objet soit *saisi et déposé*. Le pipeline enregistre le succès comme « tenu après la levée » (la vérification par la charge du chapitre 6), et note à part si l'objet a *visuellement* atterri dans la boîte ; les deux peuvent différer de peu (un objet tenu au décollage mais relâché de travers à la dépose). Pour la politique apprise, le succès est jugé à l'œil (saisi et déposé). Les deux campagnes visent la même issue — un *pick-and-place* mené à terme —, mais ne la mesurent pas tout à fait pareil : le pipeline retient la *tenue* après la levée (un proxy automatique, un peu plus indulgent que le dépôt effectif), la politique apprise le *dépôt* jugé à l'œil. Nous en tenons compte au moment de les comparer (section 7.4).

Les positions. Nous faisons varier la *distance* de l'objet à la base : *proche*, *moyenne* et *lointaine*, soit un rayon d'environ 25, 35 et 42 cm. L'objet est déplacé le long d'un *arc* de ce rayon (et non sur le seul axe avant), à ± 2 cm près. Nous sondons ainsi l'*espace de travail* — souvent très fiable de près, se dégradant de loin — plutôt qu'un unique point favorable.

Les objets. Le plan initial prévoyait cinq objets (figure 7.1). En pratique, la campagne s'est concentrée sur le *cube orange*, l'objet principal, testé avec les *deux* détecteurs à raison de 30 essais chacun. Elle a porté à titre secondaire sur le *cylindre violet* (9 essais chacun), pour sonder l'adaptation à une *autre* géométrie. Les trois autres (balle, boîte, prisme) n'ont pas été *retenus dans la campagne* : pratiqués de façon informelle, ils n'ont pas été mesurés, une réduction de périmètre assumée faute de temps et rediscutée au chapitre 8. Tous les essais portent sur un objet

seul, sans scène encombrée : l'évitement d'obstacles (objectif 3 du cahier des charges, écarté au chapitre 5) n'est pas évalué.

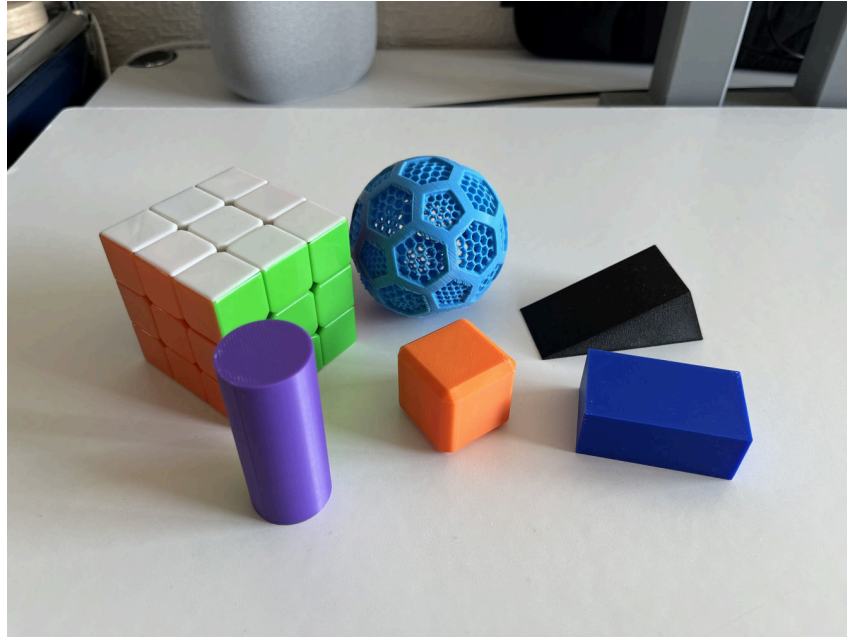


FIGURE 7.1 – Les objets utilisés au cours du projet. La campagne chiffrée porte sur le *cube orange* et le *cylindre violet* ; les autres objets ont été saisis de façon informelle, sans mesure (chapitre 8).

Les métriques. La grandeur principale est le *taux de réussite* $\hat{p} = k/n$ (k succès sur n essais). Mais sur une trentaine d'essais, ce taux n'est qu'une *estimation* du vrai taux : 21/30 pourrait, par chance, provenir d'un procédé à 60 comme à 80 %. Nous l'assortissons donc d'un *intervalle de confiance* à 95 %, la fourchette où se situe, très probablement, le vrai taux. Nous employons l'intervalle de *Wilson* [26], préférable à l'intervalle *naïf* « $\hat{p} \pm \dots$ ». Ce dernier, fondé sur une approximation normale, devient trompeur près de 0 ou 100 % (ses bornes peuvent alors sortir de $[0, 1]$), tandis que celui de Wilson reste bien conditionné pour de petits échantillons. Avec $z = 1,96$, le facteur qui englobe 95 % d'une loi normale,

$$\text{IC}_{95} = \frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}},$$

que nous calculons directement, sans bibliothèque statistique. Au-delà du taux, le pipeline journalise deux grandeurs *automatiques* qui mesurent la précision du contrôle et de la perception. La première est le *résidu de cinématique inverse* (l'erreur de pose atteinte, chapitre 6). La seconde est l'*amplitude de la correction cam_2* : la norme de l'écart *horizontal* entre la position d'abord visée par la stéréo et celle re-estimée de près par la caméra de poignet (la hauteur restant acquise à la stéréo), moyennée sur les essais (chapitre 6). La politique apprise n'expose *aucune* de ces grandeurs, ni cinématique inverse, ni re-planification : ces cases sont *sans objet*. Ce « sans objet » est lui-même un résultat, une différence de *paradigme* plutôt qu'une lacune.

Stades et modes d’échec. Pour voir *où* un essai échoue, nous relevons aussi le *stade atteint* — approche → contact → saisie → levée → transport → dépose — et, en cas d’échec, un *code de cause*. Les causes du pipeline (notées E1–E7) nomment le *module* fautif : perception, cinématique inverse, fermeture à vide. . . Celles de la politique apprise (notées I1–I8) nomment le *comportement* observé : objet éjecté à la fermeture, bras resté à sa position de repos. . . Ces deux grilles (détaillées au tableau A.5) permettent de comparer non seulement *si*, mais *pourquoi* chaque paradigme échoue.

Dans ce travail, le cube est testé sur 30 essais par détecteur (10 à chaque distance), le cylindre sur 9 (3 par distance). La politique apprise est un modèle *ACT* entraîné sur le *seul* cube : le point de contrôle final vient de Google Colab (GPU A100), à partir d’environ 200 démonstrations et 50 000 pas d’entraînement (une version locale antérieure, sur Mac/MPS, utilisait 50 démonstrations et 30 000 pas) ; un *rollout* par essai, sans rattrapage manuel. La politique observe *deux* vues, la caméra fixe **cam_1** et la caméra de poignet **cam_2** (**cam_0**, redondante avec **cam_1**, n’est pas utilisée), et les démonstrations sont enregistrées par téléopération du bras meneur. Les deux campagnes partagent l’opérateur, la session et l’éclairage ; une part du relevé (stade atteint, dépôt visuel, cause d’échec) est *manuelle*.

7.2 Résultats : pipeline modulaire

Le pipeline a été évalué sur le cube (objet principal, les *deux* détecteurs) et sur le cylindre (objet secondaire). Les taux de réussite, par distance et globaux, figurent au tableau 7.1.

TABLE 7.1 – Taux de réussite du pipeline modulaire (tenue après la levée), par distance et global, avec l’intervalle de Wilson à 95 %. « HF » désigne le détecteur appris OWL-ViT2 (chapitre 4).

Objet	Détecteur	Proche	Moyenne	Lointaine	Global
Cube	OWL-ViT2 (HF)	10/10	7/10	6/10	23/30 = 77 % [59, 88]
	HSV	9/10	6/10	5/10	20/30 = 67 % [49, 81]
Cylindre	OWL-ViT2 (HF)	3/3	2/3	2/3	7/9 = 78 % [45, 94]
	HSV	3/3	1/3	2/3	6/9 = 67 % [35, 88]

Vue d’ensemble. Sur le cube, les deux détecteurs réussissent le plus souvent : OWL-ViT2 [14] — le détecteur appris, piloté par le langage (chapitre 4) — atteint 23/30 = 77 %, le seuillage couleur HSV 20/30 = 67 %. L’écart apparent en faveur d’OWL-ViT2 ne doit pas être surinterprété : les intervalles de Wilson se *chevauchent* largement ([59, 88] contre [49, 81]), si bien que, sur 30 essais, la différence n’est *pas* significative. Nous retiendrons que les deux détecteurs soutiennent la tâche à un niveau comparable.

La distance dégrade la réussite — surtout par l’angle. La réussite baisse nettement avec la distance, de 90–100 % de près à 50–60 % au loin, mais la distance n’est pas seule en cause. Le cube étant *bas*, la politique d’angle (chapitre 5) bascule à 32 cm : *en deçà* (position *proche*), l’objet est saisi à *la verticale* (top-down) ; *au-delà* (positions *moyenne* et *lointaine*), la prise passe à 45°.

Deux des trois distances relèvent donc de ce régime incliné : le cube a été *majoritairement* testé à 45° (20 essais sur 30), le top-down ne couvrant que la position *proche*. Or ce régime est plus délicat : la cinématique inverse y est plus contrainte, et le raffinement *cam_2* y perd en fiabilité, car sa projection rayon-plan se dégrade hors de la verticale (chapitre 6). S’y ajoutent deux effets de distance *pure* : la triangulation stéréo, dont l’erreur de profondeur croît avec l’éloignement (chapitre 4), et une cinématique inverse qui peine sur les poses les plus tendues. Ces facteurs se conjuguent au loin, et cela s’est *observé* : au-delà de la zone top-down, le robot devenait plus *irrégulier*, réussissant souvent mais pas toujours. La zone lointaine, en régime incliné, est la moins fiable.

Le cylindre. Sur le cylindre, la réussite reste du même ordre — OWL-ViT2 7/9, HSV 6/9 —, avec de *larges* intervalles vu le petit nombre d’essais : on ne peut en tirer de chiffre fin. Le constat qualitatif, lui, tient : la saisie *géométrique* s’adapte à une *autre* forme sans réglage dédié, en ouvrant les mâchoires sur la largeur mesurée et en les alignant sur l’axe principal (chapitre 5). C’est une illustration concrète de l’adaptation à la géométrie visée par le cahier des charges.

Pourquoi ça échoue. Sur les 78 essais, nous comptons 22 échecs ; 21 ont reçu un code de cause dans le suivi (un essai raté a échappé au relevé), qui se répartissent ainsi : cinématique inverse injoignable (E2) $\times 8$, perception (E1) $\times 6$, fermeture à vide (E3) $\times 5$, perte à la levée (E4) $\times 1$, mauvaise saisie due à l’orientation (E5) $\times 1$. Les deux causes dominantes — pose hors d’atteinte et objet mal localisé — sont précisément celles qui se dégradent au loin, en régime incliné. La « fermeture à vide » (E3), elle, indique une localisation légèrement fausse : la pince se referme à côté de l’objet.

La précision, chiffrée. Le pipeline journalise deux grandeurs à chaque essai. Le *résidu de cinématique inverse* — l’écart entre la pose visée et la pose atteinte — vaut en moyenne 2,0 mm : le bras à cinq degrés de liberté atteint la cible au millimètre près. L’*amplitude de la correction cam_2* vaut en moyenne 13 mm : c’est le reliquat d’erreur de la stéréo que la caméra de poignet rattrape *avant* la descente. Autrement dit, la boucle fermée corrige couramment plus d’un centimètre d’erreur de perception — ce qui justifie son rôle, chiffres à l’appui (chapitre 6).

Tenu n’est pas déposé. Une nuance de mesure, enfin. Le « succès » compté ici est la *tenue après la levée* (le proxy couple/position du chapitre 6). Le *dépôt visuel* dans la boîte est parfois très légèrement inférieur — 19 dépôts pour 20 tenues sur le cube HSV, par exemple, un objet ayant lâché de travers au-dessus de la boîte. L’écart est mineur, mais réel, et nous rapportons la tenue.

7.3 Résultats : apprentissage par imitation (ACT)

La politique apprise (ACT) a été évaluée sur le cube — 30 essais, aux mêmes positions que le pipeline — puis *sondée* sur un objet qu’elle n’avait jamais vu. Ses résultats figurent au tableau 7.2.

TABLE 7.2 – Résultats de la politique apprise (ACT) sur le cube, par distance : stade de *saisie* atteint, *dépôt* réussi, et taux de réussite avec l'intervalle de Wilson à 95 %.

Position	Saisie ($\geq$ grasp)	Dépôt (succès)	Taux [Wilson]
Proche	9/10	6/10	60 % [31, 83]
Moyenne	10/10	9/10	90 % [60, 98]
Lointaine	9/10	6/10	60 % [31, 83]
Global	28/30	21/30	70 % [52, 83]

Vue d'ensemble. Sur le cube, la politique réussit 21/30 = 70 % ([52, 83]) — un niveau global comparable à celui du pipeline. Mais le *profil* est tout autre.

Un pic de réussite au centre. Là où le pipeline décroît avec la distance, la politique est la plus fiable au centre : 90 % à distance moyenne, mais seulement 60 % de près *comme* de loin. La raison tient à l'apprentissage : les démonstrations d'entraînement étaient concentrées autour de la distance moyenne, si bien que la politique est la plus fiable *là où elle a le plus vu d'exemples*, et se dégrade aux bords de cette *distribution*. C'est un comportement typique d'une méthode apprise : fiable dans la distribution d'entraînement, moins fiable en dehors.

Le point limitant n'est pas la perception, mais la fermeture. Le stade atteint est instructif : la politique *saisit* dans 28 essais sur 30 (93 %), mais ne *dépose* que dans 21. Autrement dit, elle approche et attrape presque toujours l'objet — perception et geste d'approche fonctionnent —, puis en perd environ un sur cinq *entre* la prise et la boîte. La cause dominante ($\times 7$) est l'*éjection à la fermeture* : la pince se referme *de travers*, souvent sur une arête ou un coin du cube plutôt qu'à plat, d'où une prise mal assurée — et le serrage éjecte alors le cube hors des mâchoires. Le point faible de la politique est donc l'*exécution* de la prise : elle voit et approche bien l'objet, mais le *referme* mal.

La sonde de généralisation : 0/5. Nous avons enfin *sondé* la politique sur un objet *différent* — le cylindre : 0 succès sur 5. À chaque essai, le bras est resté à sa position de repos ou a oscillé sans agir, sans même atteindre le stade de la saisie. Entraînée sur le *seul* cube, la politique échoue sur une forme qu'elle n'a jamais rencontrée. C'est la limite *structurelle* de l'imitation de bout en bout : elle ne généralise pas au-delà de ce qu'on lui a montré.

Une remarque sur la cadence. La politique tournait à environ 24 images par seconde (le point de contrôle « basse résolution »). Cette cadence conditionne la *réactivité* d'une politique qui agit image après image ; le chemin qui y a mené — et le compromis résolution contre cadence — est rediscuté au chapitre 8.

7.4 Comparaison des deux paradigmes

Les deux approches ayant été mesurées sur le même banc, nous pouvons enfin les comparer — sans désigner de « vainqueur », mais en pointant ce qui les sépare vraiment (tableau 7.3).

TABLE 7.3 – Comparaison des deux paradigmes sur la même tâche de *pick-and-place*.

	Pipeline modulaire	Politique apprise (ACT)
Réussite (cube)	77 % (OWL-ViT2) / 67 % (HSV)	70 %
Profil en distance	décroît avec la distance	pic au centre (zone d’entraînement)
Échec dominant	perception / cinématique inverse (E1, E2)	fermeture, objet éjecté
Généralisation	cylindre 67–78 % ; objets désignés par le langage	cylindre 0/5
Grandeurs internes	résidu IK ~ 2 mm, correction <code>cam_2</code> ~ 13 mm	sans objet (boîte noire)
Transparence	diagnostics par module	aucune

Sur l’objet entraîné : à égalité. Sur le cube, les trois taux — pipeline OWL-ViT2 77 %, HSV 67 %, politique apprise 70 % — ont des intervalles de Wilson qui se *chevauchent* tous largement. À 30 essais, la différence n’est pas significative : sur l’objet et dans la zone d’entraînement, les deux paradigmes se valent. On ne peut écrire ni « l’apprentissage l’emporte », ni l’inverse. La conclusion résiste même à un critère strictement identique : si l’on exige le *dépôt* des deux côtés (le pipeline étant sinon crédité de la seule *tenue*), HSV redescend à $19/30 = 63\%$ tandis qu’OWL-ViT2 reste à 77 % — tous ses objets tenus ayant été déposés — et l’imitation à 70 % ; les intervalles se chevauchent encore.

Des modes d’échec opposés. Ils échouent pourtant *différemment*. Le pipeline bute sur la *perception* et l’*atteignabilité* — objet mal localisé, pose hors d’atteinte —, surtout au loin et en régime incliné. La politique, elle, perçoit et attrape bien (saisie 93 %) mais rate l’*exécution* finale, éjectant le cube à la fermeture. Et leurs profils sont inverses : le pipeline se dégrade *avec la distance* (contrainte géométrique), tandis que la politique culmine au *centre* (sa zone d’entraînement) et faiblit aux bords. L’un est limité par la géométrie, l’autre par ses données.

La différence décisive : la généralisation. La différence décisive n’est pas sur le cube, mais *au-delà*. Le pipeline saisit le cylindre (67–78 %) *sans* ré-entraînement — car il procède par étapes sur la géométrie : mesurer, planifier, saisir — et, via OWL-ViT2, peut viser *n’importe quel* objet désigné par le langage. La politique, sondée sur ce même cylindre, obtient 0/5 : elle n’agit même pas, faute de l’avoir vu à l’entraînement. Le pipeline est *général* là où la politique est *spécialisée*.

Transparence. Enfin, quand le pipeline échoue, il dit *pourquoi* : résidu de cinématique inverse, amplitude de la correction `cam_2`, code de la cause. La politique apprise est une *boîte noire* — des images entrent, des mouvements sortent, sans explication. Pour le débogage, la vérification et la confiance, cette transparence est un atout réel de l’approche modulaire — au prix, précisément, de toute l’ingénierie que la politique apprise, elle, s’épargne.

En somme. Aucun des deux ne *domine* : leurs avantages sont complémentaires. Le pipeline modulaire *généralise* — autres formes, objets désignés par le langage — et se *diagnostique* (il dit *pourquoi* il échoue), mais il a demandé des *semaines* de construction et de réglage, module par

module. La politique apprise, elle, a atteint le même niveau en quelques *jours*, le temps de collecter environ 200 démonstrations puis d’entraîner le modèle. Elle y parvient sans le moindre modèle géométrique : elle est *économe*, en données *comme* en temps de mise en place. En revanche, elle reste *limitée* à l’objet qu’on lui a montré et demeure une *boîte noire*. Leurs voies d’amélioration suivent le même contraste : le pipeline se perfectionne par une ingénierie *ciblée*, module par module, quand la politique s’améliore surtout en lui montrant *plus de données* — plus rapide, mais limité à ce qu’elle a appris. Les deux correspondent aux extrémités d’un même compromis — *effort d’ingénierie et généralité* d’un côté, *rapidité de mise en œuvre et spécialisation* de l’autre.

Chapitre 8

Discussion

Les chapitres précédents ont décrit deux manières de résoudre la même tâche, puis les ont mesurées sur le même banc (chapitre 7). Ce chapitre prend du recul. Plutôt que de dresser un catalogue de fonctionnalités, nous revenons sur ce que la mise en œuvre a réellement *révélé* : les limites que chaque module a fini par exposer, les choix techniques qui se sont imposés une fois confrontés au matériel, et ce que la contrainte de *sous-actionnement* du bras a dicté à toute la stratégie de saisie. Plusieurs de ces points ne sont pas des décisions prises *a priori* : ce sont des conclusions tirées d’essais qui ont d’abord échoué. Nous nous efforçons de les restituer dans l’ordre où ils sont apparus (hypothèse, essai, problème observé, correctif), car c’est là que s’est concentré l’essentiel de l’apprentissage du projet. Nous terminons par les limites que nous avons *choisies*, l’évitement d’obstacles et l’occlusion, puis par les pistes qui prolongeraient le travail.

8.1 Synthèse des limites identifiées

Le pipeline complet étant posé, ses points faibles se laissent regrouper. Chacun a été rencontré *en pratique*, puis corrigé autant que possible ; les chapitres techniques les ont exposés en détail, et nous les rassemblons ici pour en dégager le trait commun.

La détection : deux faiblesses complémentaires. Les deux détecteurs présentent une faiblesse complémentaire (chapitre 4). Le seuillage HSV ne connaît que la couleur : il confond le cube orange avec les parties oranges du robot, échoue sur les teintes *achromatiques* (le blanc et le noir n’ont pas de teinte définie), et doit être recalé à chaque changement d’éclairage. Le détecteur appris OWL-ViT2 lève ces confusions par la sémantique, la forme et le contexte plutôt que la teinte, mais au prix d’une inférence *lente* (quelques secondes par image, d’où une détection ponctuelle et non un suivi continu) et d’une simple *boîte englobante* sans masque : un contour trop grossier pour fixer précisément l’orientation d’un objet rond, tel le cylindre. Aucun des deux ne réunit rapidité, robustesse sémantique et contour précis, un manque que les pistes reprendront (section 8.4).

La précision géométrique : la hauteur est l’axe le moins fiable. La calibration *œil-main* laisse des résidus de l’ordre du demi-centimètre (chapitre 4, table 4.1), qui se *propagent* : l’objet est localisé, puis la pince posée, à quelques millimètres près. Le point à retenir n’est pas ce chiffre

global, mais son *anisotropie* : l'axe le moins fiable est la *hauteur*. La paire fixe surplombe la table, si bien que sa profondeur bruitée, direction la plus incertaine de toute triangulation stéréo (chapitre 4), se projette pour l'essentiel sur la verticale. Or c'est justement l'axe que le raffinement *cam_2* ne corrige pas : il ne rattrape que le plan (x, y) , la hauteur restant celle de la stéréo (chapitre 6). Deux marges en limitent les effets, l'ouverture laissée large de 10 mm de chaque côté (chapitre 5) et la correction *cam_2* qui efface en moyenne 13 mm d'erreur latérale avant la descente (chapitre 7) ; mais pour un objet fin, cette faiblesse en hauteur reste la principale limite de fiabilité.

Un point non résolu : le roulis en prise inclinée. En prise *inclinée* (45°), l'orientation du roulis de poignet (*wrist_roll*) est moins fiable qu'à la verticale. Une part en est comprise : la convention de roulis retenue ignore le roulis *réel* de l'objet, d'où une légère asymétrie gauche/droite (chapitre 5). Mais nous avons aussi observé, à certaines positions, un comportement du roulis plus *erratique* que cette seule asymétrie n'explique, dont nous n'avons pas établi la cause avec certitude. Nous le consignons comme une limite *ouverte* du contrôle en régime incliné plutôt que d'en risquer une explication non vérifiée. Ce régime incliné est, rappelons-le, celui où se concentrent déjà les échecs de loin (chapitre 7).

8.2 Choix discutables et sous-actionnement

Toute limite de perception peut, en principe, se corriger par un meilleur modèle. D'autres choix, en revanche, tiennent à la *cinématique* du bras ou à des arbitrages assumés en cours de projet. Cette section revient sur les plus discutables, à commencer par la manière dont l'*angle de saisie* est décidé, qui découle directement du sous-actionnement introduit au chapitre 5.

L'angle d'attaque : la position, non la forme. C'est, rétrospectivement, le choix le plus discutable de la planification de saisie. Le chapitre 5 l'a posé : l'*ouverture* et l'*orientation* des mâchoires s'adaptent à la géométrie mesurée, mais l'*angle d'attaque*, lui, est dicté par la *position* de l'objet (sa distance, la hauteur de son sommet) et son atteignabilité, non par sa forme. Nous sommes d'abord partis d'un cadre purement *top-down*, la pince à la verticale, puis l'avons généralisé en un *balayage* d'inclinaisons (0° , 45° , 90°) sélectionnées par la position ; ce n'est qu'ensuite que nous avons pleinement mesuré qu'une saisie *réellement* adaptative choisirait l'angle d'après la *forme et la pose* de l'objet. À position et encombrement égaux, en effet, un cube, un cylindre ou une balle reçoivent aujourd'hui le *même* angle d'attaque, quelle que soit leur forme.

Plus fondamentalement, le planificateur raisonne sur une *boîte englobante* (position, encombrement, axe principal), non sur la *forme réelle* de l'objet ni sur ses *prises naturelles*. Une tasse creuse le montre bien : c'est un objet hors de nos primitives, que notre chaîne n'a pas eu à saisir. Le planificateur la traiterai comme une boîte : il ouvrirait les mâchoires à sa largeur et les refermerait sur ses *parois*. Cela *fonctionnerait* sans doute, la pince serrant les bords extérieurs et non le creux, mais sans analyse de la forme : la stratégie ignore que l'*anse* offrirait une prise plus naturelle, et n'évalue pas si une prise vaut mieux qu'une autre. Elle saisit sans analyser l'objet. Cela dit, combler ce manque relève autant de la *perception* que de la planification, car reconnaître une anse ou un bord comme une prise suppose d'analyser l'objet, pas seulement de le localiser.

C'est là, plus que dans le calcul de l'angle lui-même, que se situe l'axe d'amélioration : une saisie qui déciderait à partir de la *forme*, de la *pose* et des *prises offertes* par l'objet, ce vers quoi tend précisément une politique apprise (chapitre 7).

Le sous-actionnement : une donnée du problème, non un défaut. Cet angle piloté par la position n'est pas un défaut de conception : il découle du *sous-actionnement* du bras (chapitre 5). Le SO-101 dispose de cinq articulations motorisées, une de moins que les six degrés de liberté d'une pose *quelconque* : il lui manque le *lacet* de la tête. À une position donnée, on ne peut donc pas régler librement le *cap* de la pince, seulement son *inclinaison* dans le plan vertical que l'épaule oriente vers l'objet. C'est cette contrainte, et non un choix arbitraire, qui réduit le degré de liberté d'orientation à un simple *angle d'attaque*. Elle explique aussi pourquoi la prise *latérale*, saisir l'objet par le flanc, n'a pas été retenue : non qu'elle soit strictement impossible, mais elle est *mal conditionnée* sur cinq articulations (son succès dépend de la posture de départ du solveur), trop peu fiable pour être exploitée (chapitre 5). Un bras à *six* degrés de liberté lèverait la contrainte. Enfin, la branche *de face* (90°) réservée aux objets *hauts* a bien été vérifiée à la main et fonctionne, mais elle ne figure dans *aucune* campagne : nous n'en tirons donc aucun chiffre, seulement le constat qualitatif qu'elle est atteignable (chapitre 7). Cela dessine du même coup une limite de *hauteur* : les prises fiables supposent un objet assez *bas* pour l'approche verticale ou diagonale ; un objet trop haut ne relèverait que de cette branche frontale non évaluée, et sort donc en pratique de l'ensemble des prises éprouvées. Comprendre cette limite mécanique, plutôt que de la combattre, est ce qui a permis de décider *quelles* saisies valaient la peine d'être tentées.

L'open-vocabulaire plutôt qu'un détecteur ré-entraîné. Côté perception, le choix discuté est celui du *type* de détecteur appris. Un détecteur à vocabulaire *fermé* comme YOLO [12], *ré-entraîné* sur nos objets, serait sans doute plus rapide et plus précis sur ces classes, mais au prix d'un jeu d'images *annotées* par objet, soit plusieurs jours de collecte et d'annotation, à refaire pour chaque nouvel objet (chapitre 2). Le détecteur à vocabulaire *ouvert* retenu (OWL-ViT2) supprime ce coût : changer d'objet cible ne demande que de réécrire une *description textuelle*, sans ré-entraînement. Le prix en est une part de précision et une inférence plus lente, un compromis que nous jugeons raisonnable pour un travail dont l'effort porte sur l'*orchestration* du pipeline, non sur l'optimisation d'un détecteur. Nous n'avons pas essayé d'autres détecteurs à vocabulaire ouvert, tel Grounding-DINO [27] : OWL-ViT2 s'inscrit dans le même *écosystème Hugging Face* (outillage et *Hub* de modèles) que les politiques apprises de la seconde approche, une cohérence qui a pesé dans le choix.

La politique apprise : résolution contre cadence. Un dernier arbitrage, propre à la seconde approche. Une politique apprise agit *image par image* : à chaque pas, elle observe puis décide, et sa *réactivité* dépend directement de sa *cadence* d'inférence. Une première version, entraînée en pleine résolution, inférait trop lentement, de l'ordre de 13 images par seconde, pour une boucle réactive : une politique qui décide à chaque image a besoin d'un flux soutenu pour suivre le mouvement. Ré-entraîner puis exporter le modèle en résolution *réduite* a fait remonter la cadence à environ 24 images par seconde (le point de contrôle « basse résolution », chapitre 7), assez pour agir de façon fluide. Le compromis est explicite : on échange du *détail visuel* contre de la

cadence. Ce choix, purement pragmatique, a rendu la politique utilisable en boucle fermée, au prix peut-être d'une part de finesse dans la perception, que nous ne cherchons pas à surinterpréter.

8.3 Limites assumées : évitement d'obstacles et occlusion

Les limites précédentes, nous les avons *subies* puis corrigées autant que possible. D'autres, au contraire, nous les avons *choisies* : faute de temps et pour faire d'abord fonctionner la saisie, certains objectifs du cahier des charges ont été délibérément mis de côté.

L'évitement d'obstacles : un objectif délibérément écarté. Le cahier des charges prévoyait des trajectoires *évitant les obstacles* (objectif 3). Nous l'avons délibérément écarté, non par oubli mais par arbitrage : faire d'abord fonctionner la chaîne perception-planification-contrôle sur un objet *seul* nous a paru prioritaire, et un évitement d'obstacles général aurait demandé un effort hors de portée du temps imparti. En conséquence, la trajectoire (l'enchaînement de segments quintiques du chapitre 6) mène à l'objet *sans* vérifier une représentation de la scène : il n'y a pas de *détection de collision*. La scène de test étant mono-objet, sans distracteur, la question ne se posait pas en pratique ; c'est néanmoins une limite, et la métrique « collisions évitées » de l'objectif 6 reste, par construction, *vide* (chapitre 7). Une planification capable de contourner des obstacles est renvoyée aux perspectives (section 8.4).

L'occlusion : un traitement partiel, et pourquoi. La sélection de points de vue face aux occlusions (objectif 2) n'est, elle, que *partiellement* traitée. La reconstruction 3D repose sur la *triangulation* stéréo, qui exige que les *deux* caméras fixes voient l'objet ; si l'une d'elles en est empêchée (un distracteur, un bord de champ), le pipeline ne peut plus trianguler et s'*arrête* proprement. Un repli monoculaire par PnP existe bien dans le code (`enable_mono_pnp_fallback`), mais il reste en pratique *inopérant* sur nos objets, pour deux raisons qui se cumulent (chapitre 4) : nos détecteurs ne livrent qu'un *contour* grossier, une boîte et non des points d'appui précis, et surtout la pose d'un objet quasi *plan* vu de face est géométriquement *ambiguë* pour une seule caméra, quel que soit le solveur. C'est un résultat *négatif* assumé. La piste, non réalisée, serait de faire *coopérer* les caméras autrement : qu'une caméra fixe voyant encore l'objet *amorce* la caméra de poignet, ou que cette dernière tente seule une localisation lorsque la paire fixe est aveugle. Nous la présentons comme une perspective, non comme un acquis.

Un périmètre d'objets réduit. Enfin, un mot sur le périmètre d'objets. Le plan initial en prévoyait *cinq* ; la *campagne* d'évaluation, elle, s'est concentrée sur le *cube* et, secondairement, sur le *cylindre* (chapitre 7). Les autres (balle, boîte, prisme, et même un *Rubik's cube* visé par le détecteur appris OWL-ViTv2) ont bien été *saisis*, mais de manière *informelle*, sans entrer dans la campagne mesurée, faute de temps. Nous disposons donc d'indices *qualitatifs* que le pipeline s'étend au-delà de ses deux objets mesurés, sans statistiques sur ce périmètre élargi. La conséquence porte sur ce que nous pouvons *affirmer* : la *généralité* du pipeline, saisir d'autres formes ou un objet désigné par le langage, est *étayée* (plusieurs formes pratiquées, le cylindre chiffré) sans être *exhaustivement mesurée*. Le pipeline reste *plus* général que la politique apprise

(chapitre 7), mais l'étendue exacte de cette généralité, au-delà du cube et du cylindre, demande une campagne élargie pour être quantifiée.

8.4 Pistes d'amélioration

Cette mise en perspective ouvre naturellement sur les directions qui prolongeraient le travail. Plusieurs découlent directement des limites ci-dessus.

Perception : segmenter et comprendre l'objet. Un module de *segmentation* (tel SAM 2 [28]) fournirait au planificateur un *masque* précis, et non plus une simple boîte englobante : de quoi affiner l'estimation d'orientation, aujourd'hui dépendante d'un contour parfois grossier (section 8.1), et amorcer une perception des *prises* de l'objet, en repérant l'anse ou le bord d'une tasse au lieu d'en viser le centre (section 8.2). Un mode *hybride*, HSV rapide pour le suivi et détecteur appris pour la validation périodique, concilierait par ailleurs la *cadence* du premier et la *robustesse sémantique* du second.

Planification : contourner les obstacles, décider d'après l'objet. Deux directions se rejoignent ici. D'une part, remplacer l'enchaînement de segments quintiques par un *planificateur par échantillonnage* de type RRT [16], capable de *contourner* des obstacles, ce qui adresserait l'objectif 3 délibérément écarté (section 8.3). D'autre part, faire décider la *saisie* par la forme et la pose de l'objet, et non par sa seule position : choisir l'angle d'attaque *et* le point de prise d'après l'objet lui-même, en s'appuyant sur la perception enrichie ci-dessus (section 8.2).

Perception active et occlusion. Sélectionner *dynamiquement* le point de vue le moins occulté, l'objectif 2 du cahier aujourd'hui seulement effleuré par la coopération stéréo / caméra de poignet, rendrait la reconstruction robuste aux masquages. Dans le même esprit, une véritable *recupération d'occlusion* ferait coopérer les caméras au lieu d'arrêter le pipeline dès qu'une vue fixe est aveugle (section 8.3).

Asservissement visuel continu. La correction *cam_2* actuelle est *ponctuelle* : on regarde une fois, on corrige, on descend (*look-and-move*, chapitre 6). La remplacer par un asservissement visuel *continu*, référencé image [25], corrigerait la trajectoire à chaque image plutôt qu'une seule fois, au prix d'un détecteur assez rapide pour suivre la cadence.

Matériel : un sixième degré de liberté. La limite la plus structurelle n'est ni logicielle ni algorithmique : c'est le *sous-actionnement* du bras (section 8.2). Un bras à *six* degrés de liberté, ou une modification du SO-101 ajoutant un *lacet* au poignet, libérerait l'orientation d'approche et débloquerait les prises *latérales* aujourd'hui mal conditionnées. C'est l'amélioration qui lèverait la contrainte la plus fondamentale du travail, celle que ni la calibration ni le code ne peuvent contourner.

Un retour d'effort. La tenue de l'objet est aujourd'hui vérifiée par un *proxy* : la charge du moteur de pince (chapitre 6), faute de capteur dédié. Un *capteur tactile* ou d'effort à la pince y

substituerait une vraie mesure de contact, de quoi détecter plus tôt un objet qui glisse ou une fermeture à vide, et *doser* le serrage plutôt que de le tenir constant.

Évaluation et apprentissage. Enfin, deux prolongements côté mesure. Élargir la *campagne* formelle à l'ensemble des objets déjà pratiqués (section 8.3) chiffrerait la généralité aujourd'hui seulement étayée. Et, du côté de la politique apprise, l'enrichir de démonstrations *plus nombreuses et plus variées* (d'autres objets, d'autres positions) élargirait sa distribution, là où elle demeure aujourd'hui *spécialisée* (chapitre 7).

Ces pistes supposent toutes un même socle : un pipeline complet, instrumenté et reproductible. La conclusion qui suit en dresse le bilan d'ensemble et le confronte aux objectifs du cahier des charges.

Chapitre 9

Conclusion

Ce travail portait sur une question précise : faire saisir un objet à un bras robotique *low-cost* guidé par la vision. Nous y avons répondu en construisant une *pipeline* complète — perception, planification de saisie, contrôle — sur le SO-101, puis en la comparant à une seconde approche, une politique apprise par imitation évaluée sur le même banc. Ce chapitre en dresse le bilan au regard des objectifs du cahier des charges (section 9.1), puis résume les *contributions* (section 9.2) et les *perspectives* (section 9.3). Une dernière section explicite l’usage fait des outils d’assistance par intelligence artificielle (section 9.4).

9.1 Bilan au regard des objectifs

Les six objectifs du cahier des charges, posés en introduction (section 1.3, table 1.1), servent de grille d’évaluation finale. Là où cette première table en donnait le *statut*, la table 9.1 résume, pour chacun, ce que le travail a *concrètement* produit.

Objectif	Ce qui a été concrètement réalisé
1. Architecture multi-caméras	Pipeline complète perception–planification–contrôle sur trois caméras, et <i>deux</i> paradigmes — modulaire et politique apprise — construits puis comparés sur le même banc.
2. Points de vue vs occlusions	Coopération stéréo et caméra de poignet ; occlusion <i>mesurée</i> et analysée (repli monoculaire inopérant), mais sans sélection <i>active</i> de point de vue.
3. Trajectoires sans obstacles	<i>Écarté</i> délibérément : ni évitement, ni détection de collision.
4. Boucle fermée par la perception	Raffinement cam_2 avant la descente (~ 13 mm d’erreur corrigés en moyenne) et re-tentatives de prise au couple.
5. Saisie adaptée à la géométrie	Prise géométrique : ouverture et orientation des mâchoires réglées sur la forme mesurée ; réussite 67–77 % (cube) et 67–78 % (cylindre) selon le détecteur.
6. Évaluation par métriques	Deux campagnes (78 essais pour le pipeline, 30 pour la politique apprise, hors sonde de généralisation), taux de réussite et intervalles de Wilson ; métrique « collisions évitées » sans objet.

TABLE 9.1 – Bilan : ce que le travail a concrètement réalisé pour chacun des six objectifs (leur *statut* formel figure en table 1.1).

Le bilan d’ensemble est le suivant. La partie centrale du sujet — architecture multi-caméras, saisie adaptée à la géométrie, évaluation chiffrée — est *atteinte et mesurée*. Ce que le travail laisse de côté (sélection active de points de vue, évitement d’obstacles) constitue ses limites, discutées au chapitre 8.

9.2 Contributions

Ce travail comporte quatre contributions, annoncées dès l’introduction et développées au fil des chapitres.

(i) *Un pipeline complet, modulaire et reproductible*. L’élément principal du travail est une chaîne perception–planification–contrôle fonctionnelle (figure 3.1), du pixel à la prise, sur un bras *low-cost* (SO-101) et trois caméras. Chaque module — détection, triangulation, planification de saisie, cinématique inverse, contrôle moteur — y est isolé, analysable et remplaçable (chapitres 3 à 6), et l’ensemble est *open-source*.

(ii) *Une saisie adaptée à la géométrie de l’objet*. La planification de prise règle l’ouverture et l’orientation des mâchoires sur la forme mesurée, et choisit un angle d’attaque par balayage (chapitre 5). Nous en avons aussi cerné la limite : cet angle est piloté par la *position* de l’objet, non par sa *forme* (chapitre 8).

(iii) *La mise en regard de deux paradigmes*. Sur le même bras et la même tâche, le pipeline

explicite a été comparé à une politique apprise de bout en bout (ACT). La comparaison est *expérimentale* et *équitable* — même banc, mêmes positions, mêmes métriques (chapitre 7) — et débouche sur un constat nuancé : à égalité sur l’objet entraîné, le pipeline l’emporte en *généralité*, la politique apprise en *rapidité* de mise en œuvre.

(iv) Une *évaluation à périmètre déclaré*. Les résultats reposent sur deux campagnes chiffrées, assorties d’intervalles de confiance, et surtout d’un périmètre et de limites *explicitement déclarés* — objets, occlusion, obstacles.

9.3 Perspectives

Les prolongements *techniques* du travail ont été détaillés au fil de la discussion (section 8.4) : segmentation fine, planification évitant les obstacles, perception active, asservissement visuel continu, retour tactile, ou encore un bras à six degrés de liberté. Sans y revenir dans le détail, deux directions nous semblent les plus utiles.

La première est *matérielle* : lever le sous-actionnement. Un sixième degré de liberté rendrait possibles les prises que la cinématique interdit aujourd’hui et rendrait l’angle d’attaque réellement libre. C’est la contrainte la plus structurelle du travail, et la seule que ni le code ni la calibration ne peuvent contourner (chapitre 8).

La seconde est *conceptuelle* : combiner les deux paradigmes plutôt que de les opposer. Ce travail les a comparés ; il en ressort qu’ils sont *complémentaires* — le pipeline explicite apporte la transparence, les garanties géométriques et la généralité, la politique apprise la souplesse et la rapidité d’adaptation. Une architecture *hybride* — par exemple un planificateur géométrique guidé par une perception *apprise* des prises de l’objet — combinerait leurs avantages là où chacune, seule, atteint ses limites. C’est la principale direction de travail que suggère ce mémoire.

9.4 Note sur l’utilisation d’outils d’intelligence artificielle

Par souci de transparence, nous déclarons l’usage fait d’outils d’assistance par intelligence artificielle au cours de ce travail. Des assistants de programmation fondés sur de grands modèles de langage ont été employés de façon *substantielle* pour l’*implémentation* du code, ainsi que pour aider à *formuler* et à *documenter* les méthodes mathématiques standard mobilisées — calibration, triangulation, cinématique, planification de trajectoire. Ces outils ont accéléré l’écriture et la mise au point du logiciel, et servi de support pour en expliciter les fondements.

En revanche, la *conception* du projet, le choix et le montage du matériel, la définition du protocole, la conduite des expériences et l’interprétation des résultats relèvent du travail propre de l’auteur — de même que les dérivations de changements de repère qui structurent le mémoire. L’exigence de *traçabilité* — rapporter chaque chiffre à une source vérifiable dans le code ou les mesures — a guidé la rédaction. Les outils d’intelligence artificielle ont été un *moyen* ; les choix, la direction scientifique et la responsabilité du contenu final reviennent à l’auteur.

Bibliographie

- [1] THEROBOTSTUDIO. *SO-ARM100 / SO-101 : Standard Open-Source Robot Arm*. 2024. URL : <https://github.com/TheRobotStudio/SO-ARM100> (visité le 02/07/2026).
- [2] R. CADENE, S. ALIBERT et al. *LeRobot : State-of-the-art Machine Learning for Real-World Robotics in PyTorch*. Hugging Face. 2024. URL : <https://github.com/huggingface/lerobot> (visité le 02/07/2026).
- [3] R. HORAUD et F. DORNAIKA. « Hand-Eye Calibration ». In : *The International Journal of Robotics Research* 14.3 (juin 1995). arXiv :2311.12655 [cs], p. 195-210. ISSN : 0278-3649, 1741-3176. DOI : [10.1177/027836499501400301](https://doi.org/10.1177/027836499501400301). URL : <http://arxiv.org/abs/2311.12655> (visité le 11/01/2026).
- [4] G. FLANDIN, F. CHAUMETTE et E. MARCHAND. « Eye-in-hand/eye-to-hand cooperation for visual servoing ». In : *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*. T. 3. ISSN : 1050-4729. Avr. 2000, 2741-2746 vol.3. DOI : [10.1109/ROBOT.2000.846442](https://doi.org/10.1109/ROBOT.2000.846442). URL : <https://ieeexplore.ieee.org/document/846442/> (visité le 24/03/2026).
- [5] R. TSAI et R. LENZ. « A new technique for fully autonomous and efficient 3D robotics hand/eye calibration ». en. In : *IEEE Transactions on Robotics and Automation* 5.3 (juin 1989), p. 345-358. ISSN : 1042296X. DOI : [10.1109/70.34770](https://doi.org/10.1109/70.34770). URL : <http://ieeexplore.ieee.org/document/34770/> (visité le 11/01/2026).
- [6] Z. ZHANG. « A Flexible New Technique for Camera Calibration ». In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22.11 (2000), p. 1330-1334. DOI : [10.1109/34.888718](https://doi.org/10.1109/34.888718).
- [7] *Understanding Reprojection Error*. en. Août 2022. URL : <https://calib.io/blogs/knowledge-base/understanding-reprojection-errors> (visité le 03/04/2026).
- [8] S. GARRIDO-JURADO, R. MUÑOZ-SALINAS, F. J. MADRID-CUEVAS et M. J. MARÍN-JIMÉNEZ. « Automatic generation and detection of highly reliable fiducial markers under occlusion ». In : *Pattern Recognition* 47.6 (2014), p. 2280-2292. DOI : [10.1016/j.patcog.2014.01.005](https://doi.org/10.1016/j.patcog.2014.01.005).
- [9] L. LI, L. WAN, V. KRUEGER et X. ZHANG. *3D Hand-Eye Calibration for Collaborative Robot Arm : Look at Robot Base Once*. arXiv :2504.21619 [cs]. Mai 2025. DOI : [10.48550/arXiv.2504.21619](https://doi.org/10.48550/arXiv.2504.21619). URL : <http://arxiv.org/abs/2504.21619> (visité le 11/01/2026).
- [10] R. HARTLEY et A. ZISSERMAN. *Multiple view geometry in computer vision*. en. 2. edition, 17.printing. Cambridge : Cambridge Univ. Press, 2018. ISBN : 978-0-521-54051-3.

- [11] D. A. FORSYTH et J. PONCE. *Computer Vision : A Modern Approach*. 2nd. Boston : Pearson, 2012. ISBN : 978-0-13-608592-8.
- [12] J. REDMON, S. DIVVALA, R. GIRSHICK et A. FARHADI. « You Only Look Once : Unified, Real-Time Object Detection ». In : *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, p. 779-788. DOI : [10.1109/CVPR.2016.91](https://doi.org/10.1109/CVPR.2016.91).
- [13] M. MINDERER, A. GRITSENKO et al. « Simple Open-Vocabulary Object Detection with Vision Transformers ». In : *Computer Vision – ECCV 2022*. Springer, 2022, p. 728-755. DOI : [10.1007/978-3-031-20080-9_42](https://doi.org/10.1007/978-3-031-20080-9_42).
- [14] M. MINDERER, A. GRITSENKO et N. HOULSBY. « Scaling Open-Vocabulary Object Detection ». In : *Advances in Neural Information Processing Systems (NeurIPS)*. 2023. URL : <http://arxiv.org/abs/2306.09683>.
- [15] J. BOHG, A. MORALES, T. ASFOUR et D. KRAGIC. « Data-Driven Grasp Synthesis - A Survey ». In : *IEEE Transactions on Robotics* 30.2 (avr. 2014). arXiv :1309.2660 [cs], p. 289-309. ISSN : 1552-3098, 1941-0468. DOI : [10.1109/TR0.2013.2289018](https://doi.org/10.1109/TR0.2013.2289018). URL : <http://arxiv.org/abs/1309.2660> (visité le 28/03/2026).
- [16] S. M. LAVALLE. *Planning Algorithms*. en. 1^{re} éd. Cambridge University Press, mai 2006. ISBN : 978-0-521-86205-9 978-0-511-54687-7. DOI : [10.1017/CB09780511546877](https://doi.org/10.1017/CB09780511546877). URL : <https://www.cambridge.org/core/product/identifier/9780511546877/type/book> (visité le 28/03/2026).
- [17] B. SICILIANO, L. SCIAVICCO, L. VILLANI et G. ORIOLO. *Robotics : Modelling, Planning and Control*. London : Springer, 2009. ISBN : 978-1-84628-641-4.
- [18] T. Z. ZHAO, V. KUMAR, S. LEVINE et C. FINN. « Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware ». In : *Robotics : Science and Systems (RSS)*. 2023. DOI : [10.15607/RSS.2023.XIX.016](https://doi.org/10.15607/RSS.2023.XIX.016). URL : <http://arxiv.org/abs/2304.13705>.
- [19] C. CHI, S. FENG et al. « Visuomotor Policy Learning via Action Diffusion ». en. In : *Robotics : Science and Systems (RSS)* (2023).
- [20] M. SHUKOR, D. AUBAKIROVA et al. *SmolVLA : A Vision-Language-Action Model for Affordable and Efficient Robotics*. arXiv :2506.01844 [cs]. Juin 2025. DOI : [10.48550/arXiv.2506.01844](https://doi.org/10.48550/arXiv.2506.01844). URL : <http://arxiv.org/abs/2506.01844> (visité le 24/03/2026).
- [21] V. LEPETIT, F. MORENO-NOGUER et P. FUA. « EPnP : An Accurate O(n) Solution to the PnP Problem ». In : *International Journal of Computer Vision* 81.2 (2009), p. 155-166. ISSN : 0920-5691, 1573-1405. DOI : [10.1007/s11263-008-0152-6](https://doi.org/10.1007/s11263-008-0152-6).
- [22] F. ZACHARIAS, C. BORST et G. HIRZINGER. « Capturing robot workspace structure : representing robot capabilities ». In : *2007 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2007, p. 3229-3236. DOI : [10.1109/IROS.2007.4399105](https://doi.org/10.1109/IROS.2007.4399105).
- [23] N. VAHRENKAMP, T. ASFOUR et R. DILLMANN. « Robot placement based on reachability inversion ». In : *2013 IEEE International Conference on Robotics and Automation (ICRA)*. 2013, p. 1970-1975. DOI : [10.1109/ICRA.2013.6630839](https://doi.org/10.1109/ICRA.2013.6630839).

- [24] D. W. MARQUARDT. « An Algorithm for Least-Squares Estimation of Nonlinear Parameters ». In : *Journal of the Society for Industrial and Applied Mathematics* 11.2 (1963), p. 431-441. DOI : [10.1137/0111030](https://doi.org/10.1137/0111030).
- [25] F. CHAUMETTE et S. HUTCHINSON. « Visual servo control, Part I : Basic approaches ». en. In : *IEEE Robotics & Automation Magazine* 13.4 (2006), p. 82-90. DOI : [10.1109/MRA.2006.250573](https://doi.org/10.1109/MRA.2006.250573).
- [26] E. B. WILSON. « Probable inference, the law of succession, and statistical inference ». In : *Journal of the American Statistical Association* 22.158 (1927), p. 209-212.
- [27] S. LIU, Z. ZENG et al. « Grounding DINO : Marrying DINO with Grounded Pre-Training for Open-Set Object Detection ». In : *Computer Vision – ECCV 2024*. Springer, 2024. URL : <http://arxiv.org/abs/2303.05499>.
- [28] N. RAVI, V. GABEUR et al. « SAM 2 : Segment Anything in Images and Videos ». In : *arXiv preprint arXiv :2408.00714* (2024). URL : <http://arxiv.org/abs/2408.00714>.

Annexe A

Détails techniques et reproductibilité

Cette annexe rassemble les informations pratiques que le corps du mémoire ne détaille pas, mais dont un lecteur a besoin pour reproduire le travail : l'organisation du code, la mesure du repère de base, la procédure de calibration, la campagne de dimensionnement de la scène, puis les paramètres et les fichiers des campagnes d'évaluation. Le code, les configurations et les modèles imprimés en 3D sont regroupés dans un unique dépôt, github.com/machanier/tfe-lerobot-so101, qui constitue la référence exécutable de ce qui est décrit ici.

A.1 Structure du code

Le dépôt s'organise en dossiers de premier niveau. Une bibliothèque `src/` contient le cœur réutilisable ; des points d'entrée `scripts/` l'assemblent en ligne de commande pour une tâche précise ; des fichiers `configs/` conservent les calibrations et les paramètres ; `tests/` regroupe les tests automatiques ; `hardware/` rassemble les modèles imprimés en 3D (STL) et `results/` les données brutes des quatre campagnes. Cette découpe reflète l'architecture du chapitre 3 : chaque module de `src/` correspond à une brique du pipeline perception → planification → contrôle, et les scripts ne font que les orchestrer. Le tableau A.1 associe chaque module de la bibliothèque à son rôle et au chapitre qui le décrit.

Points d'entrée (`scripts/`). Les scripts se répartissent en quatre familles. La *calibration* regroupe `calibrate_intrinsic.py` (paramètres intrinsèques par caméra), `recalibrate_handeye.py` et `recalibrate_handeye_stereo.py` (*hand-eye*), ainsi que `calibrate_hsv.py` et `calibrate_grasp_threshold.py` (réglage des seuils couleur et de fermeture). L'*exécution* se fait par `pick_and_place.py` (la tâche complète décrite au chapitre 6) et `run_perception.py` (perception seule, sans mouvement). Les *campagnes* d'évaluation sont pilotées par `experiment_campaign.py` (pipeline modulaire) et `experiment_campaign_il.py` (imitation), ce dernier s'appuyant sur `record_dataset.py`, `train.py` et `eval_policy.py` pour la collecte, l'entraînement et l'évaluation de la politique. Enfin, une famille de *vérification* (`check_perception.py`, `check_calibration.py`, `compare_detectors.py`) sert à contrôler la chaîne sans lancer de saisie.

Configurations (`configs/`). Toutes les grandeurs mesurées ou réglées sont externalisées dans ce dossier, jamais codées en dur : les intrinsèques (`calibration_cam_0/1/2.json`), les

Fichier	Rôle
<code>pipeline.py</code>	Orchestration d'un cycle de saisie complet : capture, détection, reconstruction 3D, planification, contrôle et boucle fermée (chapitres 3 et 6).
<i>perception/ — chapitre 4</i>	
<code>camera_io.py</code>	Capture synchronisée des trois caméras USB ; chargement des paramètres intrinsèques.
<code>detector.py</code>	Détection 2D : interface <code>ObjectDetector</code> et ses deux implémentations, <code>HSVDetector</code> (seuillage couleur) et <code>HFDetector</code> (détecteur appris OWL-ViT2).
<code>pose_estimator.py</code>	Reconstruction 3D des détections dans le repère de base (triangulation stéréo, repli PnP monoculaire).
<code>robot_state.py</code>	Lecture des moteurs et calcul de la pose de l'effecteur par cinématique directe.
<code>scene.py</code>	Types de données échangés dans le pipeline (<code>Frame</code> , <code>Detection2D</code> , <code>ObjectInstance</code>).
<i>planning/ — chapitre 5</i>	
<code>grasp.py</code>	Stratégies de saisie : interface <code>GraspStrategy</code> , cas de base <code>TopDownGrasp</code> et stratégie géométrique <code>AdaptiveGrasp</code> (choix de l'angle d'attaque, de l'ouverture et de l'orientation).
<i>control/ — chapitre 6</i>	
<code>ik.py</code>	Cinématique inverse (Gauss–Newton / Levenberg–Marquardt, jacobienne numérique, redémarrages déterministes).
<code>trajectory.py</code>	Génération de trajectoires articulaires quintiques.
<code>motor_controller.py</code>	Interface haut niveau du bus Feetech (consigne de position, lecture de couple, sécurités).
<code>closed_loop.py</code>	Raffinement de la pose de saisie par la caméra de poignet <code>cam_2</code> .
<i>calibration/ — sous-basement des chapitres 4 et 6</i>	
<code>forward_kinematics.py</code>	Cinématique directe à partir de la chaîne décrite dans le fichier URDF.
<code>handeye.py</code>	Calibration <i>hand-eye</i> via <code>cv2.calibrateHandEye</code> .
<code>motor_to_angle.py</code>	Conversion entre valeur d'encodeur (12 bits, circulaire) et angle articulaire.
<i>utils/</i>	
<code>transforms.py</code>	Opérations sur les transformations rigides $SE(3)$, représentées par des matrices 4×4 homogènes.

TABLE A.1 – Modules de la bibliothèque `src/`, leur rôle et le chapitre qui les décrit.

hand-eye (`handeye_cam_*.json`), le modèle cinématique (`so101_new_calib.urdf`), la géométrie de la scène (`scene.json`), et les paramètres de perception (`perception/hsv_specs.json`, `perception/hf_specs.json`, `perception/bias_correction*.json`). Cette centralisation est ce qui rend l'exécution reproductible : rejouer le système sur un autre montage revient à remplacer ces fichiers, sans toucher au code.

Tests (`tests/`). Le dossier contient les tests automatiques cités dans le corps du mémoire, notamment la validation du pipeline de perception (`perception/test_pipeline.py`) et de la sélection de saisie (`planning/test_adaptive_grasp_selection.py`). Ce sont ces tests, exécutés sur graine fixe, qui produisent les grandeurs déterministes rapportées aux chapitres 4 et 6.

A.2 Repère de base : mesure expérimentale

Toutes les positions du pipeline sont exprimées dans le repère de base `base_link`, dont les axes sont : X vers l'avant du robot, Y vers la gauche (positif ; droite négative), Z vers le haut. Par convention, la plaque sur laquelle reposent les objets définit $z = 0$ (paramètre `table_z_m`) : un objet de hauteur H posé sur la table a donc son centre à $z = H/2$ (un cube de 30 mm $\rightarrow +15$ mm).

Localisation de l'origine. L'origine $(0,0,0)$ n'est pas un point matériel évident : elle ne coïncide pas avec le centre du moteur de lacet (`shoulder_pan`), mais s'en trouve décalée de quelques centimètres — en retrait selon X , en dessous selon Z , et vers la droite selon Y . Faute de repère physique où poser un instrument, la vérification des mesures s'est appuyée surtout sur X , clairement lisible sur le bâti du robot : nous mesurons au mètre la distance de l'origine à l'objet posé, puis nous la comparons à la position reconstruite. Les axes Y et Z , plus difficiles à rattacher à un point tangible, n'ont pas servi de contrôle direct.

Hauteur de la table. La stéréo *sous-estime* légèrement la hauteur : un cube de 30 mm posé sur la plaque, dont le centre est donc à +15 mm, se reconstruit à un z légèrement *négatif* (autour de -5 mm). Le Z reconstruit étant trop bruité pour qu'on cherche à le corriger directement, deux dispositions suffisent. D'une part, nous fixons $z = 0$ au niveau de la plaque (paramètre `table_z_m`) et nous abaissons la borne basse de l'espace de travail à $z_{\min} = -20$ mm : un simple seuil d'*acceptation*, pour qu'un objet réellement posé (donc reconstruit un peu bas) ne soit pas rejeté comme aberrant. D'autre part, la hauteur de prise n'est pas lue sur ce z , mais *ancrée* sur l'objet reposant sur la plaque : le plan $z = 0$ plus la demi-hauteur connue de l'objet.

Biais systématique de la stéréo. Indépendamment de la hauteur, la reconstruction stéréo présente un biais systématique, caractérisé par une procédure simple (fichier `configs/perception/bias_correction.json`) : nous posons l'objet à des distances *mesurées au mètre* depuis l'origine, nous lisons la position reconstruite (`check_perception.py`), et nous prenons, pour chaque axe, l'écart signé $\text{biais} = \text{reconstruit} - \text{réel}$, moyenné sur plusieurs positions (tableau A.2). L'erreur *globale* de position est, elle, la norme euclidienne de cet écart, comme la précision rapportée au chapitre 7. Ce biais est celui de la *stéréo* (caméras fixes) : la caméra de poignet `cam_2` est, elle, précise en X , si bien que la correction en profondeur n'est appliquée qu'aux caméras fixes. Le biais en X vaut -33 mm à 300 mm et -31 mm à 350 mm : *le même* aux deux distances, donc indépendant de l'éloignement. C'est un décalage *fixe* que nous annulons en le soustrayant tel quel. Le biais en Y est plus erratique (détection de petits amas de pixels, sensible au bruit) : nous le laissons à zéro en perception et nous corrigeons l'écart latéral une seule fois, plus tard, à la saisie, pour que deux corrections ne se cumulent pas. Enfin `cam_2`, sur sa *propre* calibration, porte son propre biais latéral, distinct de celui de la stéréo ($\Delta y = -7$ mm, fichier `bias_correction_cam2.json`).

Axe	Biais mesuré	Traitement
X (profondeur)	≈ -32 mm, constant	soustrait (stéréo, caméras fixes); <code>cam_2</code> précise en X
Y (latéral)	+16 à +24 mm, variable	laissé à 0; corrigé à la saisie
Z (hauteur)	sous-lecture ≈ 20 mm	non corrigé; borne basse -20 mm, prise ancrée sur la plaque

TABLE A.2 – Biais systématique de la reconstruction stéréo, mesuré sur le montage, et son traitement.

A.3 Procédure de calibration

Deux calibrations s’enchaînent pour placer une détection 2D dans le repère de base : le modèle *intrinsèque* de chaque caméra, puis la transformation *hand-eye* qui relie la caméra au robot. Leurs fondements mathématiques (méthode de Zhang, équation $AX = XB$ de Horaud) sont exposés au chapitre 4; nous ne donnons ici que la procédure pratique et les décomptes. Les deux étapes reposent sur un damier imprimé (`generate_chessboard.py`), et le corps du mémoire en illustre le déroulement (figures 4.2 et 4.4).

Deux damiers. La calibration *intrinsèque* utilise un damier de 7×7 coins internes (carrés de 22,2 mm). Ce damier est *symétrique*, ce qui ne gêne pas l’intrinsèque, mais introduit pour le *hand-eye* une ambiguïté d’orientation à 180° (la pose du damier vue par la caméra peut être confondue avec sa version retournée). Après plusieurs essais infructueux d’extrinsèque, nous sommes donc passés, pour cette étape, à un damier *asymétrique* de 9×6 coins internes (carrés de 22 mm) : le nombre pair de coins dans un sens et impair dans l’autre lève l’ambiguïté et fiabilise l’estimation de pose.

Intrinsèque. Pour chaque caméra, nous présentons le damier sous des angles et des distances variés et nous capturons les poses au clavier (`calibrate_intrinsic.py`); OpenCV en déduit la matrice K (focales f_x, f_y , point principal c_x, c_y) et les coefficients de distorsion $(k_1, k_2, p_1, p_2, k_3)$, enregistrés dans `calibration_cam_<i>.json`. La littérature recommande un minimum d’une quinzaine de poses; nous en avons capturé davantage — une quarantaine par caméra, dont 41, 41 et 43 retenues — pour une erreur moyenne de reprojection sous-pixellique de 0,19 / 0,17 / 0,14 px respectivement.

Hand-eye. La transformation caméra $\leftrightarrow$ robot est estimée par `cv2.calibrateHandEye` (méthode de Horaud), via `recalibrate_handeye_stereo.py`. Pour la paire fixe (*eye-to-hand*), le damier est tenu par la pince à travers de nombreuses poses du bras (le montage extérieur est illustré figure A.1); pour la caméra de poignet (*eye-in-hand*), le damier est fixe et c’est le bras qui bouge. Jusqu’à 71 poses sont capturées, dont une partie seulement est retenue après rejet automatique des prises mal détectées : 23/71 (`cam_0`, sous-ensemble robuste), 71/71 (`cam_1`, déduite de `cam_0`) et 20/41 (`cam_2`). Une prise est écartée lorsque sa pose s’éloigne trop de la solution

consensuelle : après un premier calcul, nous mesurons pour chaque prise son résidu — l'écart de sa transformation à la solution commune — et nous retirons celles dont le résidu dépasse une tolérance de quelques millimètres et degrés, avant de recalculer sur les prises restantes ; une détection floue ou trop oblique du damier y ressort en résidu élevé. La paire fixe donne une base stéréo de 103,35 mm, ses centres optiques se situant à (654, 92, 235) mm et (655, -11, 229) mm dans le repère de base.

Seuils couleur (HSV) et éclairage. Le détecteur couleur demande une calibration d'une autre nature que les précédentes : les seuils HSV ne dépendent pas de la géométrie mais de l'*éclairage*, dont la teinte et l'intensité déplacent les valeurs mesurées. Or le travail s'est étalé sur plusieurs mois (de mars à juin), pendant lesquels la lumière naturelle a changé. Il a donc fallu *fixer* un éclairage reproductible et re-calibrer les seuils juste avant la campagne finale, pour que tous les essais partagent la même lumière. Nous avons pour cela mené une courte étude (`results/calibration_hsv/lighting_study/`) : la scène est figée — mêmes objets, mêmes cadrages — et nous ne faisons varier que la lumière d'une condition à l'autre (jour seul, jour + lampes, lampes seules, en plusieurs réglages). Pour chaque condition, nous capturons trois vues par caméra (l'objet à gauche, au centre, à droite de l'axe) et nous mesurons, avec le détecteur du pipeline lui-même, l'exposition *par caméra* et la couleur de l'objet.

La caméra la plus sensible est celle de *poignet* : proche de l'objet, elle sature vite. Avec le jour combiné à des lampes trop fortes, plus de 60 % de ses pixels ressortaient « cramés » (alors que les caméras fixes restaient quasi épargnées), ce qui explique sa réponse plus délavée et sa plage HSV propre. En ajustant l'éclairage, cette surexposition retombe à quelques pour-cent. La configuration retenue est *la lumière du jour complétée par la lampe du plafond* : c'est celle qui, dans nos comparaisons, ressortait la mieux exposée et la plus lisible en couleur sur les trois caméras. Les seuils ont alors été calibrés sous cette condition (`calibrate_hsv.py`, caméra par caméra), puis *gardés inchangés* pour toute la perception, les tests de saisie et la campagne. Un éclairage stable n'est donc pas un détail : c'est une condition du bon fonctionnement du seuillage couleur (chapitres 4 et 8).



FIGURE A.1 – Montage *eye-to-hand* : le damier est fixé sur une planche de carton à l’aide de scotch, le tout fixé perpendiculairement sur le bout des pinces, que le bras promène devant les caméras fixes (visibles au fond, sur leurs montants). Cette vue extérieure complète les vues caméras du chapitre 4 (figure 4.4).

A.4 Campagne de mesure de la structure d’environnement

La scène n’a pas été improvisée : la structure qui porte les caméras a été dimensionnée par un modèle géométrique explicite (feuille de calcul `results/mesures_structure/enclosure_design.xlsx`) *avant* l’impression, pour que la paire stéréo voie l’ensemble de l’espace de travail avec une précision de profondeur suffisante. Le modèle repose sur quatre paramètres de conception et sur quelques grandeurs optiques dérivées des intrinsèques calibrées, définis au tableau A.3.

Trois relations, dont les expressions figurent au tableau A.3, imposent chacune une contrainte. La *couverture* L doit englober tout l’espace de travail atteignable — un disque de 43 cm de rayon, soit 86 cm de diamètre. La *précision de profondeur* de la stéréo ΔZ doit rester de l’ordre de 2 mm à la distance de travail, ce qui pousse la base B vers le haut et rapproche les caméras. Enfin la *visibilité verticale* (bords x_{near} , x_{far}) doit contenir à la fois la base du bras et l’ensemble de l’espace de travail. Un candidat (D, B, h, θ) n’est retenu que s’il satisfait les trois *simultanément* ; le balayage des plages du tableau A.3 produit 636 combinaisons faisables, la région faisable s’élargissant quand D croît (davantage d’inclinaisons rentrent dans le champ vertical).

Le montage finalement construit se situe dans cette région : une base stéréo de 10 cm (confirmée par la calibration, 103 mm, section A.3), des caméras à environ 65 cm devant la base et 23 cm

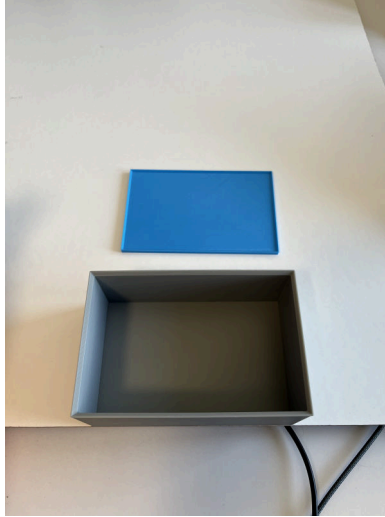
Symbole	Signification	Plage ou expression
<i>Paramètres de conception (balayés)</i>		
D	distance des caméras au centre de l'espace de travail	55–65 cm
B	base stéréo (écart entre les deux caméras fixes)	8–15 cm
h	hauteur des caméras au-dessus de la table	28–40 cm
θ	inclinaison des caméras vers le bas	36–50°
<i>Grandeurs optiques (issues de la calibration)</i>		
f	distance focale moyenne de la paire	≈ 1219 px
HFOV, VFOV	champs de vue horizontal et vertical	76°, 48°
Z	distance caméra–objet le long de l'axe optique	(variable)
L	largeur de scène couverte à la distance Z	$2Z \tan(\text{HFOV}/2)$
ΔZ	incertitude de profondeur de la stéréo à la distance Z	$Z^2/(fB)$
$x_{\text{near}}, x_{\text{far}}$	bords proche et lointain du champ au sol	fonction de h, θ, VFOV

TABLE A.3 – Paramètres et grandeurs du modèle de dimensionnement de la structure (feuille `enclosure_design.xlsx`).

au-dessus de la table (centres optiques mesurés, section A.3), inclinées d'environ 48° vers le bas (angle de fixation des caméras sur la structure). La structure a été imprimée en 3D, en PLA+ comme les pièces du robot, en quatre pièces distinctes et en six impressions (deux pièces tirées en double) : deux montants verticaux qui portent les caméras fixes, deux rails, une base côté caméras et une base côté robot, assemblés par vis et boulons. L'ensemble est maintenu fermement sur la table par des serre-joints prenant appui sur la base côté robot (conçue et imprimée sur mesure), le robot y étant lui-même vissé et boulonné. Le bras *leader* servant aux démonstrations d'imitation (chapitre 7) est fixé de la même façon. Par-dessus vient se poser la plaque de la scène (60 × 90 cm), maintenue par des aimants (cinq côté caméras, trois côté robot). La boîte de dépôt, également imprimée, est garnie d'un matelas en TPU (le rembourrage bleu visible sur les photos) qui amortit la chute de l'objet (figure A.2). Les modèles STL sont fournis dans le dépôt (dossier `hardware/`) ; le balayage complet des combinaisons reste consultable dans la feuille de calcul.



(a) Structure porte-caméras.



(b) Boîte de dépôt.



(c) Plaque de travail.

FIGURE A.2 – La scène assemblée. (a) Les deux montants portent la paire stéréo fixe (*eye-to-hand*), assemblés aux rails et aux bases par vis et boulons, et les rails indépendants au sol sont des chutes d'impression servant de cales. (b) La boîte de dépôt, garnie du matelas en TPU bleu. (c) La plaque de travail (60×90 cm), maintenue par des aimants.

A.5 Capture des données et paramètres

Cette dernière section liste les fichiers de résultats bruts derrière les chiffres du chapitre 7 (tableau A.4) et les paramètres d'exécution qui fixent le comportement du système (tableau A.6). Les deux campagnes sont journalisées automatiquement, un essai par ligne, au format CSV ; pour le pipeline, il s'agit de la feuille de suivi des essais, exportée depuis le classeur de travail.

Chaque essai est en outre annoté, en cas d'échec, d'un code de cause (chapitre 7) : le tableau A.5 en donne la grille complète pour les deux campagnes, le décompte par code figurant au chapitre 7.

Campagne	Fichier(s)	Contenu
Pipeline	<code>results/saisie_pipeline/campagne_donnees.csv</code>	78 essais (cube et cylindre, détecteurs HSV et OWL-ViT2, trois distances)
IL cube	<code>results/saisie_imitation/campaign_il_20260627_*.csv</code>	30 essais, trois sessions (proche / moyen / loin)
IL cylindre (sonde)	<code>results/saisie_imitation/campaign_il_20260628_113734.csv</code>	5 essais (généralisation géométrique)

TABLE A.4 – Manifeste des fichiers de campagne, regroupés dans le dossier `results/` du dépôt.

Code	Signification
<i>Pipeline modulaire — module ou étape fautive</i>	
E1	Perception : objet mal vu ou non détecté
E2	Cinématique inverse : aucun angle d’attaque atteignable
E3	Fermeture à vide : la pince se referme à côté de l’objet
E4	Perte à la levée : l’objet retombe pendant la montée
E5	Saisie ratée par l’orientation, l’objet étant pourtant bien localisé
E6	Dépôt raté : objet saisi et levé, mais non déposé dans la boîte
E7	Matériel : défaillance d’un composant (<code>cam_2</code> USB, moteur...)
<i>Politique apprise (ACT) — comportement observé</i>	
I1	N’approche pas : le bras reste à sa position de repos
I2	Approche mais mauvais alignement (objet hors des mâchoires)
I3	Fermeture à vide
I4	Saisit puis lâche l’objet en transit
I5	Rate la dépose (hors de la boîte)
I6	Matériel : caméra gelée ou défaillance (essai biaisé)
I7	Comportement erratique, oscillation
I8	Saisit mais l’objet est éjecté à la fermeture, par la force de serrage, avant le transit

TABLE A.5 – Codes de cause d’échec des deux campagnes (chapitre 7). Les codes E# nomment le module ou l’étape fautive du pipeline ; les codes I# décrivent le comportement observé de la politique apprise.

Paramètre	Valeur
<i>Détection</i>	
Détecteur appris	OWL-ViTv2 (<code>google/owlv2-base-patch16-ensemble</code>), seuil de confiance 0,25
Détecteur couleur	seuils HSV par caméra (<code>hsv_specs.json</code>)
<i>Politique d'angle d'attaque (<code>grasp.py</code>)</i>	
Top-down (0°)	sommet < 12 cm <i>et</i> distance ≤ 32 cm
Diagonale (45°)	sommet 12–18 cm, ou distance > 32 cm
Frontal (90°)	sommet ≥ 18 cm (hors campagne)
Rejet en top-down pur	sommet > 12 cm
<i>Saisie et boucle fermée</i>	
Ouverture de la pince	(largeur + 2×10 mm)/150 mm, bornée à [20, 100] %
Décalage latéral des mâchoires	automatique = $\frac{1}{2}$ largeur + 5 mm
Décalage de convention (lacet)	+ 90° (mesuré sur le montage)
Observation / raffinement cam_2	12 cm au-dessus de l'objet
Point de passage d'approche	8 cm
Biais soustraits	stéréo $\Delta x = -32$ mm ; cam_2 $\Delta y = -7$ mm
<i>Imitation (ACT)</i>	
Entraînement final	200 démonstrations, 50 000 pas (Colab, GPU A100)
Entraînement local	50 démonstrations, 30 000 pas (Mac, MPS)
Point de reprise évalué	version basse résolution, ~ 24 images/s

TABLE A.6 – Principaux paramètres d'exécution du pipeline et de l'imitation.